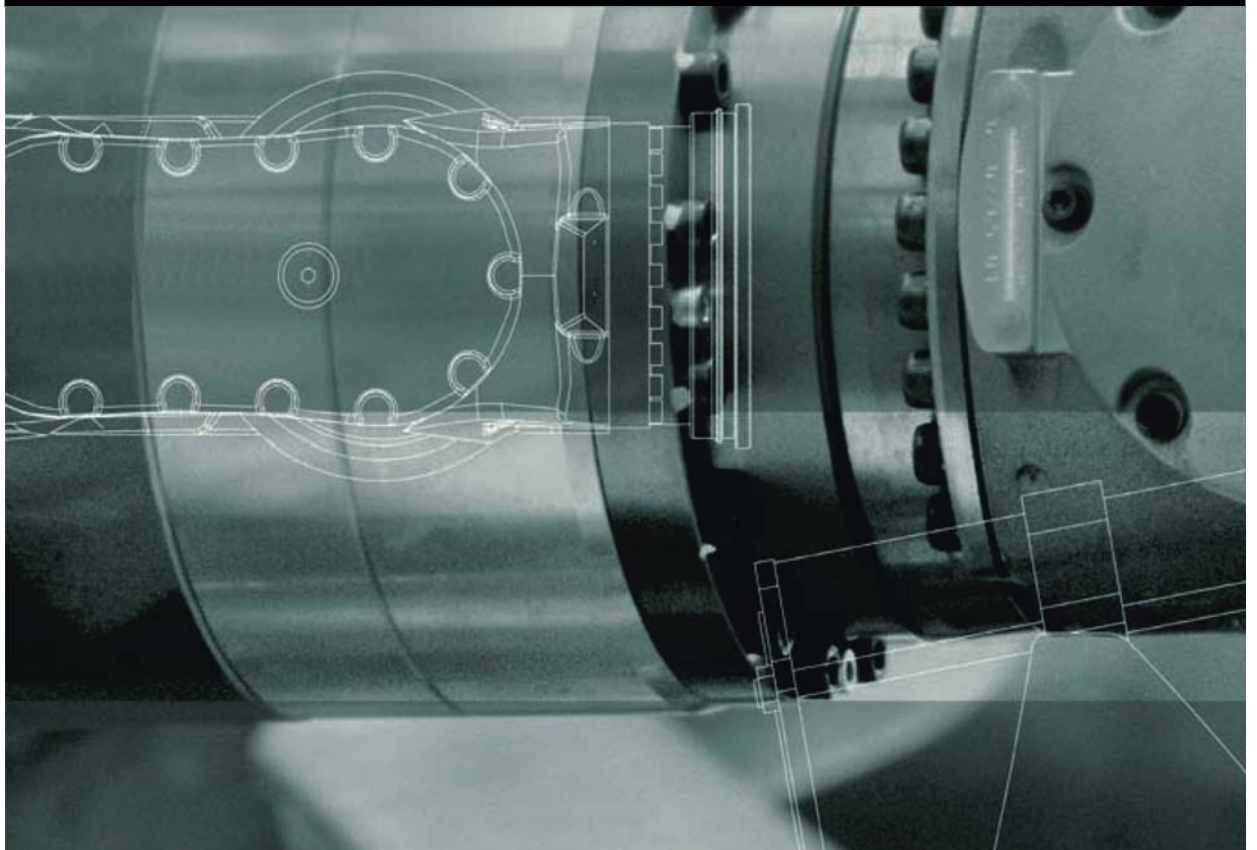


KUKA.Ethernet KRL 3.0

Für KUKA System Software 8.5



Stand: 06.03.2017

Version: KST Ethernet KRL 3.0 V3

© Copyright 2017

KUKA Roboter GmbH
Zugspitzstraße 140
D-86165 Augsburg
Deutschland

Diese Dokumentation darf – auch auszugsweise – nur mit ausdrücklicher Genehmigung der KUKA Roboter GmbH vervielfältigt oder Dritten zugänglich gemacht werden.

Es können weitere, in dieser Dokumentation nicht beschriebene Funktionen in der Steuerung lauffähig sein. Es besteht jedoch kein Anspruch auf diese Funktionen bei Neulieferung oder im Servicefall.

Wir haben den Inhalt der Druckschrift auf Übereinstimmung mit der beschriebenen Hard- und Software geprüft. Dennoch können Abweichungen nicht ausgeschlossen werden, so dass wir für die vollständige Übereinstimmung keine Gewähr übernehmen. Die Angaben in dieser Druckschrift werden jedoch regelmäßig überprüft und notwendige Korrekturen sind in der nachfolgenden Auflage enthalten.

Technische Änderungen ohne Beeinflussung der Funktion vorbehalten.

Original-Dokumentation

KIM-PS5-DOC

Publikation:	Pub KST Ethernet KRL 3.0 (PDF) de
Buchstruktur:	KST Ethernet KRL 3.0 V3.1
Version:	KST Ethernet KRL 3.0 V3

Inhaltsverzeichnis

1	Einleitung	7
1.1	Zielgruppe	7
1.2	Dokumentation des Industrieroboters	7
1.3	Darstellung von Hinweisen	7
1.4	Verwendete Begriffe	8
1.5	Marken	9
1.6	Lizenzen	9
2	Produktbeschreibung	11
2.1	Übersicht Ethernet KRL	11
2.2	Konfiguration einer Ethernet-Verbindung	11
2.2.1	Verhalten bei Verbindungsverlust	11
2.2.2	Überwachen einer Verbindung	12
2.3	Datenaustausch	12
2.4	Datenspeicherung	13
2.5	Client-Server-Betrieb	14
2.6	Protokollarten	14
2.7	Ereignismeldungen	15
2.8	Fehlerbehandlung	15
2.9	Bestimmungsgemäße Verwendung	16
3	Sicherheit	17
4	Installation	19
4.1	Systemvoraussetzungen	19
4.2	Installation über WorkVisual	19
4.2.1	Ethernet KRL installieren oder updaten	19
4.2.2	Ethernet KRL deinstallieren	19
4.3	Installation über smarthMI	20
4.3.1	Ethernet KRL installieren oder updaten	20
4.3.2	Ethernet KRL deinstallieren	21
5	Konfiguration	23
5.1	Netzwerkverbindung über das KLI der Robotersteuerung	23
6	Programmierung	25
6.1	Ethernet-Verbindung konfigurieren	25
6.1.1	XML-Struktur für Verbindungseigenschaften	25
6.1.2	XML-Struktur für den Datenempfang	28
6.1.3	XML-Struktur für den Datenversand	31
6.1.4	Konfiguration nach XPath-Schema	31
6.2	Funktionen für den Datenaustausch	32
6.2.1	Programmiertipps	33
6.2.2	Initialisieren und Löschen einer Verbindung	34
6.2.3	Öffnen und Schließen einer Verbindung	36
6.2.4	Senden von Daten	36
6.2.5	Auslesen von Daten	39
6.2.6	Löschen von Datenspeichern	41

6.2.7	EKI_STATUS – Struktur für funktionspezifische Rückgabewerte	42
6.2.8	Konfigurieren von Ereignismeldungen	43
6.2.9	Empfang vollständiger XML-Datensätze	43
6.2.10	Verarbeiten unvollständiger Datensätze	44
6.2.11	EKI_CHECK() – Funktionen auf Fehler prüfen	44
7	Beispielkonfigurationen und -programme	47
7.1	Server-Programm und Beispiele einbinden	47
7.2	Bedienoberfläche Server-Programm	48
7.2.1	Kommunikationsparameter im Server-Programm einstellen	49
7.3	Beispiel "BinaryFixed"	50
7.4	Beispiel "BinaryStream"	52
7.5	Beispiel "XmlTransmit"	53
7.6	Beispiel "XmlServer"	54
7.7	Beispiel "XmlCallback"	55
8	Diagnose	59
8.1	Diagnosedaten anzeigen	59
9	Meldungen	61
9.1	Fehlerprotokoll (EKI-Logbuch)	61
9.2	Informationen zu den Meldungen	61
9.3	Systemmeldungen aus Modul: EthernetKRL (EKI)	61
9.3.1	EKI00002	61
9.3.2	EKI00003	63
9.3.3	EKI00006	64
9.3.4	EKI00007	65
9.3.5	EKI00009	66
9.3.6	EKI00010	67
9.3.7	EKI00011	69
9.3.8	EKI00012	70
9.3.9	EKI00013	71
9.3.10	EKI00014	72
9.3.11	EKI00015	74
9.3.12	EKI00016	75
9.3.13	EKI00017	78
9.3.14	EKI00018	81
9.3.15	EKI00019	83
9.3.16	EKI00020	84
9.3.17	EKI00021	87
9.3.18	EKI00022	89
9.3.19	EKI00023	90
9.3.20	EKI00024	91
9.3.21	EKI00027	92
9.3.22	EKI00512	93
9.3.23	EKI00768	94
9.3.24	EKI01024	95
9.3.25	EKI01280	95
9.3.26	EKI01536	98

9.3.27	EKI01792	99
9.3.28	EKI02048	100
10	Anhang	101
10.1	Erweiterte XML-Struktur für Verbindungseigenschaften	101
10.2	Speicher erhöhen	101
10.3	Meldungsausgabe und Loggen von Meldungen deaktivieren	102
10.4	Befehlsreferenz	102
10.4.1	Verbindung initialisieren, öffnen, schließen und löschen	102
10.4.2	Daten senden	103
10.4.3	Daten schreiben	104
10.4.4	Daten auslesen	106
10.4.5	Funktion auf Fehler prüfen	109
10.4.6	Datenspeicher löschen, sperren, entsperren und prüfen	110
11	KUKA Service	113
11.1	Support-Anfrage	113
11.2	KUKA Customer Support	113
	Index	121

1 Einleitung

1.1 Zielgruppe

Diese Dokumentation richtet sich an Benutzer mit folgenden Kenntnissen:

- Fortgeschrittene KRL-Programmierkenntnisse
- Fortgeschrittene Systemkenntnisse der Robotersteuerung
- Fortgeschrittene XML-Kenntnisse
- Fortgeschrittene Netzwerkkennnisse



Für den optimalen Einsatz unserer Produkte empfehlen wir unseren Kunden eine Schulung im KUKA College. Informationen zum Schulungsprogramm sind unter www.kuka.com oder direkt bei den Niederlassungen zu finden.

1.2 Dokumentation des Industrieroboters

Die Dokumentation zum Industrieroboter besteht aus folgenden Teilen:

- Dokumentation für die Robotermechanik
- Dokumentation für die Robotersteuerung
- Bedien- und Programmieranleitung für die System Software
- Anleitungen zu Optionen und Zubehör
- Teilekatalog auf Datenträger

Jede Anleitung ist ein eigenes Dokument.

1.3 Darstellung von Hinweisen

Sicherheit

Diese Hinweise dienen der Sicherheit und **müssen** beachtet werden.



Diese Hinweise bedeuten, dass Tod oder schwere Verletzungen sicher oder sehr wahrscheinlich eintreten **werden**, wenn keine Vorsichtsmaßnahmen getroffen werden.



Diese Hinweise bedeuten, dass Tod oder schwere Verletzungen eintreten **können**, wenn keine Vorsichtsmaßnahmen getroffen werden.



Diese Hinweise bedeuten, dass leichte Verletzungen eintreten **können**, wenn keine Vorsichtsmaßnahmen getroffen werden.



Diese Hinweise bedeuten, dass Sachschäden eintreten **können**, wenn keine Vorsichtsmaßnahmen getroffen werden.



Diese Hinweise enthalten Verweise auf sicherheitsrelevante Informationen oder allgemeine Sicherheitsmaßnahmen.
Diese Hinweise beziehen sich nicht auf einzelne Gefahren oder einzelne Vorsichtsmaßnahmen.

Dieser Hinweis macht auf Vorgehensweisen aufmerksam, die der Vorbeugung oder Behebung von Not- oder Störfällen dienen:

**SICHERHEITS-
ANWEISUNGEN**

Die folgende Vorgehensweise genau einhalten!

Mit diesem Hinweis gekennzeichnete Vorgehensweisen **müssen** genau eingehalten werden.

Hinweise

Diese Hinweise dienen der Arbeitserleichterung oder enthalten Verweise auf weiterführende Informationen.



Hinweis zur Arbeitserleichterung oder Verweis auf weiterführende Informationen

1.4 Verwendete Begriffe

Begriff	Beschreibung
Datenstrom	Kontinuierliche Abfolgen von Datensätzen, deren Ende nicht im Voraus abzusehen ist. Die einzelnen Datensätze sind von beliebigem, aber festem Typ. Die Menge der Datensätze pro Zeiteinheit (Datenrate) kann variieren. Es ist nur ein sequentieller Zugriff auf die Daten möglich.
EKI	Ethernet KRL Interface
EOS	End Of Stream (Endzeichenfolge) Zeichenfolge, die das Ende eines Datensatzes kennzeichnet
Ethernet	Ethernet ist eine Datennetztechnologie für lokale Datennetze (LANs). Sie ermöglicht den Datenaustausch in Form von Datenrahmen zwischen den verbundenen Teilnehmern.
FIFO	Verfahren, nach denen ein Datenspeicher abgearbeitet werden kann
LIFO	
	■ First In First Out: Elemente, die zuerst gespeichert wurden, werden zuerst wieder aus dem Speicher entnommen. ■ Last In First Out: Elemente, die zuletzt gespeichert wurden, werden zuerst wieder aus dem Speicher entnommen.
KLI	KUKA Line Interface Linienbus zur Integration der Anlage in das Kunden-netz
KR C	KUKA Robot Controller KR C ist die KUKA Robotersteuerung.
KRL	KUKA Robot Language KRL ist die KUKA Roboter Programmiersprache.
smarHMI	smart Human-Machine Interface KUKA smarHMI ist die Bedienoberfläche der KUKA System Software.
Socket	Software-Schnittstelle, die IP-Adressen und Port-Nummern miteinander verbindet

Begriff	Beschreibung
TCP/IP	Transmission Control Protocol Protokoll über den Datenaustausch zwischen den Teilnehmern eines Netzwerks. TCP stellt einen virtuellen Kanal zwischen 2 Endpunkten einer Netzwerkverbindung her. Auf diesem Kanal können in beide Richtungen Daten übertragen werden.
UDP/IP	User Datagram Protocol Verbindungsloses Protokoll über den Datenaustausch zwischen den Teilnehmern eines Netzwerks
IP	Internet Protocol Das Internet-Protokoll hat die Aufgabe, Subnetze über physikalische MAC-Adressen zu definieren.
XML	Extensible Markup Language Standard zur Erstellung maschinen- und menschenlesbarer Dokumente in Form einer vorgegebenen Baumstruktur
XPath	XML Path Language Sprache, um Teile eines XML-Dokuments zu beschreiben und zu lesen

1.5 Marken

.NET Framework ist eine Marke der Microsoft Corporation.

Windows ist eine Marke der Microsoft Corporation.

1.6 Lizenzen

Die KUKA Lizenzbedingungen und die Lizenzbedingungen verwendeter Open-Source-Software sind in folgenden Ordnern zu finden:

- Auf dem Datenträger mit den Installationsdateien der KUKA Software unter `.LICENSE`
- Nach der Installation auf der Robotersteuerung unter `D:\KUKA_OPT\Optionspaketname\LICENSE`
- Nach der Installation in WorkVisual im Fenster **Kataloge** in der Registerkarte **Optionen** in dem Lizenzenordner unter dem Namen des Optionspakets



Weitere Informationen zu Open-Source-Lizenzen können unter folgender Adresse angefordert werden: opensource@kuka.com

2 Produktbeschreibung

2.1 Übersicht Ethernet KRL

Funktionen	Ethernet KRL ist ein nachladbares Optionspaket mit folgenden Funktionen: ■ Datenaustausch über das EKI ■ Empfangen von XML-Daten eines externen Systems ■ Senden von XML-Daten an ein externes System ■ Empfangen von Binärdaten eines externen Systems ■ Senden von Binärdaten an ein externes System
Eigenschaften	■ Robotersteuerung und externes System als Client oder Server ■ Konfiguration von Verbindungen über XML-basierte Konfigurationsdatei ■ Konfiguration von "Ereignismeldungen" ■ Überwachen von Verbindungen durch einen Ping auf das externe System ■ Lesen und Schreiben von Daten aus dem Submit-Interpreter ■ Lesen und Schreiben von Daten aus dem Roboter-Interpreter
Kommunikation	Daten werden über das TCP/IP-Protokoll übertragen. Die Verwendung des UDP/IP-Protokolls ist möglich, wird aber nicht empfohlen (verbindungsloses Netzwerkprotokoll, z. B. kein Erkennen von Datenverlust).

2.2 Konfiguration einer Ethernet-Verbindung

Die Ethernet-Verbindung wird über eine XML-Datei konfiguriert. Für jede Verbindung muss im Verzeichnis C:\KRC\ROBOTER\Config\User\Common\EthernetKRL der Robotersteuerung eine Konfigurationsdatei definiert sein. Die Konfiguration wird beim Initialisieren einer Verbindung eingelesen.

Ethernet-Verbindungen können vom Roboter- oder Submit-Interpreter angelegt und bedient werden. Die Kanäle sind über Kreuz verwendbar, z. B. kann ein im Submit-Interpreter geöffneter Kanal auch vom Roboter-Interpreter bedient werden.

Das Löschen einer Verbindung kann an Roboter- und Submit-Interpreter-Aktionen oder Systemaktionen gekoppelt sein.

Bei einer Verwendung über Kreuz müssen bestimmte Richtlinien beachtet werden, um unerwartetes Programmverhalten zu vermeiden.

(>>> 6.2.1 "Programmiertipps" Seite 33)

2.2.1 Verhalten bei Verbindungsverlust

Folgende Eigenschaften und Funktionen des EKI stellen sicher, dass empfangene Daten zuverlässig bearbeitet werden können:

- Bei Erreichen des Limits eines Datenspeichers wird eine Verbindung automatisch geschlossen.
- Wenn ein Fehler beim Datenempfang auftritt, wird eine Verbindung automatisch geschlossen.
- Bei geschlossener Verbindung werden die Datenspeicher weiterhin ausgelesen.
- Bei Verbindungsverlust kann ohne Einfluss auf gespeicherte Daten die Verbindung wiederhergestellt werden.
- Ein Verbindungsverlust kann angezeigt werden, z. B. über ein Flag.

- Zum Fehler, der zu einem Verbindungsverlust geführt hat, kann die Fehlermeldung auf der smartHMI ausgegeben werden.

2.2.2 Überwachen einer Verbindung

Eine Verbindung kann durch einen Ping auf das externe System überwacht werden. (Element <ALIVE.../> in der Verbindungskonfiguration)

Bei erfolgreicher Verbindung kann je nach Konfiguration ein Flag oder ein Ausgang gesetzt werden. Solange der Ping regelmäßig gesendet wird und die Verbindung zum externen System aktiv ist, ist der Ausgang oder das Flag gesetzt. Wenn die Verbindung zum externen System abbricht, wird der Ausgang oder das Flag gelöscht.

2.3 Datenaustausch

Übersicht

Über KUKA.Ethernet KRL kann die Robotersteuerung sowohl Daten von einem externen System empfangen als auch Daten an ein externes System senden.

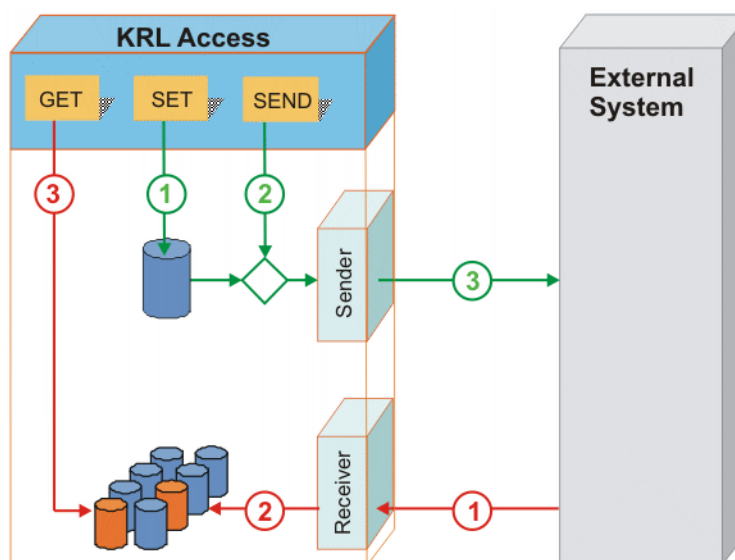


Abb. 2-1: Systemübersicht

Datenempfang

Prinzipieller Ablauf (Rot markiert) (>>> Abb. 2-1):

1. Das externe System sendet Daten, die über ein Protokoll übertragen und vom EKI empfangen werden.
2. Die Daten werden strukturiert in einem Datenspeicher abgelegt.
3. Aus einem KRL-Programm heraus wird strukturiert auf die Daten zugegriffen. Mithilfe von KRL-Anweisungen werden die Daten gelesen und in KRL-Variablen kopiert.

Datenversand

Prinzipieller Ablauf (Grün markiert) (>>> Abb. 2-1):

1. Mithilfe von KRL-Anweisungen werden die Daten strukturiert in einen Datenspeicher geschrieben.
2. Mit einer KRL-Anweisung werden die Daten aus dem Speicher gelesen.
3. EKI sendet die Daten über ein Protokoll an das externe System.



Es ist möglich Daten direkt zu versenden, ohne dass die Daten zuvor in einem Speicher abgelegt werden.

2.4 Datenspeicherung

Beschreibung Alle empfangenen Daten werden automatisch gespeichert und stehen damit KRL zur Verfügung. Beim Speichern werden XML- und Binärdaten unterschiedlich behandelt.

Jeder Datenspeicher ist als Stapelspeicher realisiert. Die einzelnen Speicher werden im FIFO- oder LIFO-Modus ausgelesen.

XML-Daten Die empfangenen Daten werden extrahiert und typrichtig in verschiedene Speicher abgelegt (pro Wert ein Speicher).

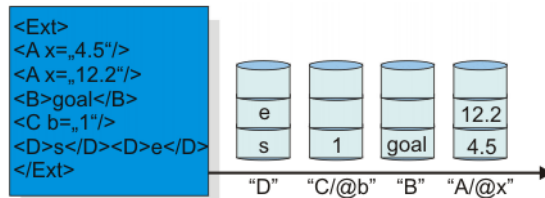


Abb. 2-2: XML-Daten-Speicher

Binärdaten Die empfangenen Daten werden nicht extrahiert oder interpretiert. Für eine Verbindung im Binärmodus existiert nur ein Speicher.

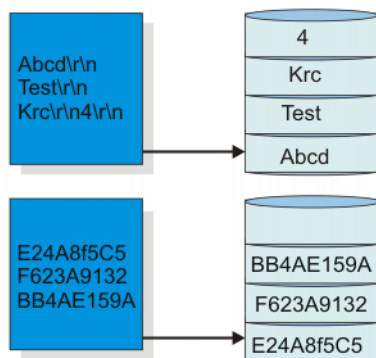


Abb. 2-3: Binäre Datenspeicher

Ausleseverfahren Datenelemente werden in der Reihenfolge aus dem Speicher entnommen, in der sie dort abgelegt wurden (FIFO). Das umgekehrte Verfahren, mit dem das zuletzt im Speicher abgelegte Datenelement zuerst entnommen wird, ist konfigurierbar (LIFO).

Jedem Speicher wird ein gemeinsames Limit maximal speicherbarer Daten zugeteilt. Wird das Limit überschritten, wird die Ethernet-Verbindung sofort geschlossen, um den Empfang weiterer Daten zu verhindern. Die aktuell empfangenen Daten werden noch gespeichert und der Speicher um "1" erhöht. Die Speicher können weiter abgearbeitet werden. Die Verbindung kann über die Funktion EKI_OPEN() wieder geöffnet werden.

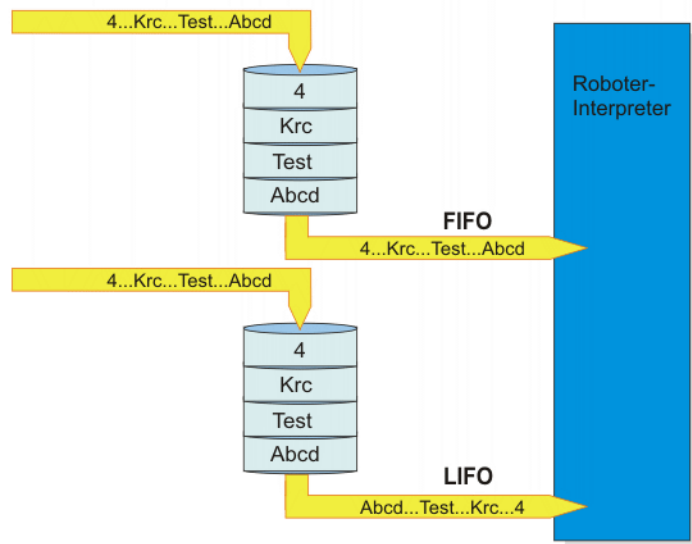


Abb. 2-4: Ausleseverfahren Übersicht

2.5 Client-Server-Betrieb

Beschreibung

Robotersteuerung und externes System verbinden sich als Client und Server. Dabei kann das externe System Client oder Server sein. Die Anzahl aktiver Verbindungen ist auf 16 beschränkt.

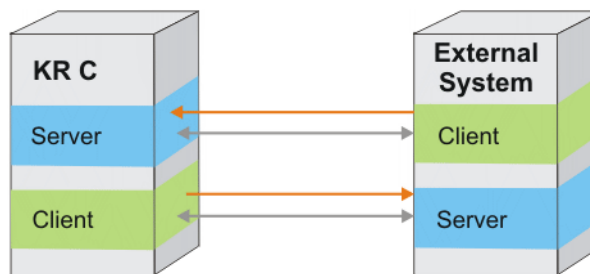


Abb. 2-5: Client-Server-Betrieb

Wenn das EKI als Server konfiguriert wird, kann sich nur ein einzelner Client mit dem Server verbinden. Werden mehrere Verbindungen benötigt, sind auch mehrere EKI-Server anzulegen. Es ist möglich mehrere Clients und Server gleichzeitig innerhalb des EKI zu betreiben.

2.6 Protokollarten

Beschreibung

Die übertragenen Daten können in verschiedene Formate verpackt werden. Folgende Formate werden unterstützt:

- XML-Struktur frei konfigurierbar
- Binärdatensatz mit fester Länge
- Binärdatensatz variabel mit Endzeichenfolge

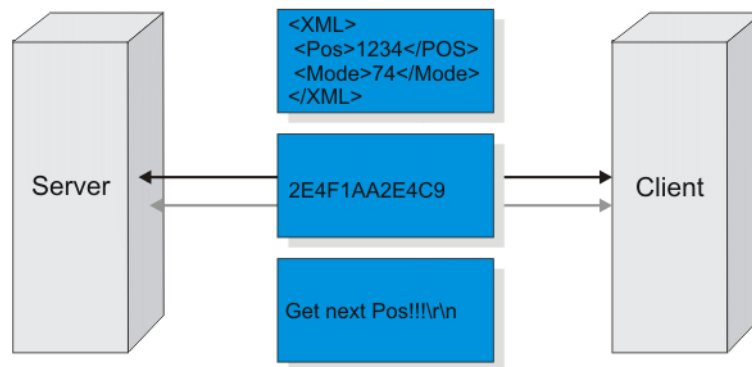


Abb. 2-6: Protokollarten

Die beiden Binärdatenformate können nicht gleichzeitig auf einer Verbindung betrieben werden.

Folgende Kombinationen sind möglich:

Verbindung Vx	V1	V2	V3	V4	V5
Binär fest	✓	✗	✓	✗	✗
Binär variabel	✗	✗	✗	✓	✓
XML	✓	✓	✗	✗	✓

Beispiele



Abb. 2-7: Binärdaten mit fester Länge (10 Byte)

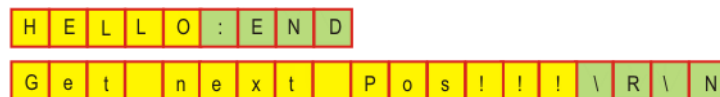


Abb. 2-8: Binärdaten variabel mit Endzeichenfolge

2.7 Ereignismeldungen

Über das Setzen eines Ausgangs oder Flags können folgende Ereignisse gemeldet werden:

- Verbindung ist aktiv.
- Ein einzelnes XML-Element ist angekommen.
- Eine vollständige XML-Struktur oder ein vollständiger Binärdatensatz ist angekommen.

(>>> 6.2.8 "Konfigurieren von Ereignismeldungen" Seite 43)

2.8 Fehlerbehandlung

Beschreibung

Für den Datenaustausch zwischen Robotersteuerung und externem System stellt KUKA.Ethernet KRL Funktionen zur Verfügung.

Jede dieser Funktionen gibt Werte zurück. Diese Rückgabewerte können im KRL-Programm abgefragt und ausgewertet werden.

Abhängig von der Funktion werden folgende Werte zurückgegeben:

- Fehlernummer

- Anzahl der Elemente, die sich nach dem Zugriff noch im Speicher befinden
- Anzahl der Elemente, die aus dem Speicher gelesen wurden
- Information, ob eine Verbindung besteht
- Zeitstempel des Datenelements, das aus dem Speicher entnommen wurde

(>>> 6.2.7 "EKI_STATUS – Struktur für funktionsspezifische Rückgabewerte" Seite 42)

Zu jedem Fehler wird eine Meldung auf der smartHMI und im EKI-Logbuch ausgegeben. Die automatische Ausgabe von Meldungen kann deaktiviert werden.

2.9 Bestimmungsgemäße Verwendung

Verwendung Das Optionspaket KUKA.Ethernet KRL ist für den Datenaustausch zwischen der Robotersteuerung und einem externen System über Ethernet zur Programmlaufzeit von Submit- oder Roboter-Interpreter vorgesehen.

Fehlanwendung Alle von der bestimmungsgemäßen Verwendung abweichenden Anwendungen gelten als Fehlanwendung und sind unzulässig. Für hieraus resultierende Schäden haftet der Hersteller nicht. Das Risiko trägt allein der Betreiber.

Zu den Fehlanwendungen zählen z. B.:

- Die Kommunikation über Ethernet ist kein vollständiger Ersatz für ein Feldbussystem und ist nicht zur Bedienung der Automatik-Extern-Schnittstelle vorgesehen.
- KUKA.Ethernet KRL garantiert kein deterministisches Zeitverhalten, d. h. es kann nicht davon ausgegangen werden, dass abgerufene Daten mit einer konstanten Verzögerung zur Verfügung stehen.
- KUKA.Ethernet KRL ist nicht geeignet für den Aufbau einer zyklischen Kommunikation im Echtzeittakt der Robotersteuerung.

3 Sicherheit

Diese Dokumentation enthält Sicherheitshinweise, die sich spezifisch auf die hier beschriebene Software beziehen.

Die grundlegenden Sicherheitsinformationen zum Industrieroboter sind im Kapitel "Sicherheit" der Bedien- und Programmieranleitung für Systemintegratoren oder der Bedien- und Programmieranleitung für Endanwender zu finden.



Das Kapitel "Sicherheit" in der Bedien- und Programmieranleitung der System Software muss beachtet werden. Tod von Personen, schwere Verletzungen oder erheblicher Sachschaden können sonst die Folge sein.

4 Installation

Das Optionspaket kann entweder über die smartHMI oder über WorkVisual auf der Robotersteuerung installiert werden.

4.1 Systemvoraussetzungen

Hardware	■ KR C4
	■ Externes System
Software	■ KUKA System Software 8.5
	■ WorkVisual 5.0

4.2 Installation über WorkVisual

4.2.1 Ethernet KRL installieren oder updaten

Beschreibung Das Optionspaket wird in WorkVisual installiert und dem Projekt hinzugefügt. Beim Übertragen des Projekts wird das Optionspaket automatisch auf der Robotersteuerung installiert.

 Es wird empfohlen, vor dem Update einer Software alle zugehörigen Daten zu archivieren.

Voraussetzung

- Benutzergruppe Experte
- Betriebsart T1 oder T2
- Es ist kein Programm angewählt.
- Netzwerkverbindung zwischen PC und Robotersteuerung
- Optionspaket liegt als KOP-Datei vor.

Vorgehensweise

1. Das Optionspaket **EthernetKRL** in WorkVisual installieren.
2. Das aktive Projekt von der Robotersteuerung laden.
3. Das Optionspaket **EthernetKRL** in das Projekt einfügen.
4. Das Projekt von WorkVisual auf die Robotersteuerung übertragen und aktivieren.
5. Auf der smartHMI wird die Sicherheitsabfrage *Wollen Sie die Aktivierung des Projektes [...] zulassen?* angezeigt. Bei der Aktivierung wird das aktive Projekt überschrieben. Wenn kein relevantes Projekt überschrieben wird: Die Abfrage mit **Ja** bestätigen.
6. Auf der smartHMI wird eine Übersicht mit den Änderungen und eine Sicherheitsabfrage angezeigt. Diese mit **Ja** beantworten. Das Optionspaket wird installiert und die Robotersteuerung führt einen Neustart durch.

 Informationen zu Abläufen in WorkVisual sind in der Dokumentation zu WorkVisual zu finden.

LOG-Datei Es wird eine LOG-Datei unter C:\KRC\ROBOTER\LOG erstellt.

4.2.2 Ethernet KRL deinstallieren

 Es wird empfohlen, vor der Deinstallation einer Software alle zugehörigen Daten zu archivieren.

- Voraussetzung**
- Benutzergruppe Experte
 - Betriebsart T1 oder T2
 - Es ist kein Programm angewählt.
 - Netzwerkverbindung zwischen PC und Robotersteuerung
- Vorgehensweise**
1. Das Projekt von der Robotersteuerung laden.
 2. Das Optionspaket **EthernetKRL** aus dem Projekt entfernen. Ein Fenster mit Änderungen wird angezeigt.
 3. Das Projekt von WorkVisual auf die Robotersteuerung übertragen und aktivieren.
 4. Die Sicherheitsabfrage auf der smartHMI *Wollen Sie die Aktivierung des Projektes [...] zulassen?* mit **Ja** bestätigen.
 5. Auf der smartHMI wird eine Übersicht mit den Änderungen und eine Sicherheitsabfrage angezeigt. Diese mit **Ja** beantworten. Das Optionspaket wird deinstalliert und die Robotersteuerung führt einen Neustart durch.

 Informationen zu Abläufen in WorkVisual sind in der Dokumentation zu WorkVisual zu finden.

LOG-Datei Es wird eine LOG-Datei unter C:\KRC\ROBOTER\LOG erstellt.

4.3 Installation über smartHMI

4.3.1 Ethernet KRL installieren oder updaten

 Es wird empfohlen, vor dem Update einer Software alle zugehörigen Daten zu archivieren.

- Voraussetzung**
- Benutzerrechte: Funktionsgruppe **Allgemeine Konfiguration**
 - Betriebsart T1 oder T2
 - Es ist kein Programm angewählt.
 - USB-Stick mit dem Optionspaket (KOP-Datei)

HINWEIS Es wird empfohlen, einen KUKA-USB-Stick zu verwenden. Wenn ein Stick eines anderen Herstellers verwendet wird, können Daten verloren gehen.

- Vorgehensweise**
1. Den USB-Stick an der Robotersteuerung oder am smartPAD anstecken.
 2. Im Hauptmenü **Inbetriebnahme > Zusatzsoftware** wählen.
 3. Auf **Neue Software** drücken: In der Spalte **Name** muss der Eintrag **EthernetKRL** angezeigt werden und in der Spalte **Pfad** das Laufwerk **E:** oder **K:**.
Wenn nicht, auf **Aktualisieren** drücken.
 4. Wenn die genannten Einträge jetzt angezeigt werden, weiter mit Schritt 5.
Wenn nicht, muss erst der Pfad, von dem installiert werden soll, konfiguriert werden:
 - a. Auf die Schaltfläche **Konfigurieren** drücken.
 - b. Im Bereich **Installationspfade für Optionen** eine Zeile markieren.
Hinweis: Wenn die Zeile bereits einen Pfad enthält, wird dieser überschrieben.
 - c. Auf **Pfadauswahl** drücken. Die vorhandenen Laufwerke werden angezeigt.

- d. Wenn der Stick an der Robotersteuerung angesteckt ist: **E:** markieren.
Wenn der Stick am smartPAD angesteckt ist: **K:** statt **E:**
- e. Auf **Speichern** drücken. Der Bereich **Installationspfade für Optionen** wird wieder angezeigt. Er enthält jetzt den neuen Pfad.
- f. Die Zeile mit dem neuen Pfad markieren und nochmal auf **Speichern** drücken.
5. Bei **EthernetKRL** das Häkchen setzen und auf **Installieren** drücken. Den Installationshinweis mit **OK** bestätigen.
6. Die Sicherheitsabfrage *Wollen Sie die Aktivierung des Projektes [...] zulassen?* wird angezeigt. Bei der Aktivierung wird das aktive Projekt überschrieben. Wenn kein relevantes Projekt überschrieben wird: Die Abfrage mit **Ja** bestätigen.
7. Eine Übersicht mit den Änderungen und eine Sicherheitsabfrage wird angezeigt. Diese mit **Ja** beantworten. Das Optionspaket wird installiert und die Robotersteuerung führt einen Neustart durch.
8. Den Stick abziehen.

LOG-Datei Es wird eine LOG-Datei unter C:\KRC\ROBOTER\LOG erstellt.

4.3.2 Ethernet KRL deinstallieren



Es wird empfohlen, vor der Deinstallation einer Software alle zugehörigen Daten zu archivieren.

- Voraussetzung**
- Benutzerrechte: Funktionsgruppe **Allgemeine Konfiguration**
 - Betriebsart T1 oder T2
 - Es ist kein Programm angewählt.

- Vorgehensweise**
1. Im Hauptmenü **Inbetriebnahme** > **Zusatzsoftware** wählen.
 2. Bei **EthernetKRL** das Häkchen setzen und auf **Deinstallieren** drücken. Die Sicherheitsabfrage mit **Ja** beantworten.
 3. Die Sicherheitsabfrage *Wollen Sie die Aktivierung des Projektes [...] zulassen?* mit **Ja** bestätigen.
 4. Eine Übersicht mit den Änderungen und eine Sicherheitsabfrage wird angezeigt. Diese mit **Ja** beantworten. Das Optionspaket wird deinstalliert und die Robotersteuerung führt einen Neustart durch.

LOG-Datei Es wird eine LOG-Datei unter C:\KRC\ROBOTER\LOG erstellt.

5 Konfiguration

5.1 Netzwerkverbindung über das KLI der Robotersteuerung

Für den Datenaustausch über Ethernet muss eine Netzwerkverbindung über das KLI der Robotersteuerung hergestellt werden.



Weitere Informationen zur KLI-Netzwerkconfiguration sind in der Bedien- und Programmieranleitung für Systemintegratoren der System Software zu finden.

Je nach Spezifikation stehen an der Kundenschnittstelle der Robotersteuerung folgende Ethernet-Schnittstellen als Option zur Verfügung:

- Schnittstelle X66 (1 Steckplatz)
- Schnittstelle X67.1-3 (3 Steckplätze)



Weitere Informationen zu den Ethernet-Schnittstellen sind in der Betriebs- oder Montageanleitung der Robotersteuerung zu finden.

6 Programmierung

6.1 Ethernet-Verbindung konfigurieren

Übersicht

Eine Ethernet-Verbindung wird über eine XML-Datei konfiguriert. Für jede Verbindung muss im Verzeichnis C:\KRC\ROBOTER\Config\User\Common\EthernetKRL der Robotersteuerung eine Konfigurationsdatei definiert sein.



XML-Dateien sind "case sensitive". Die Groß-/Kleinschreibung muss beachtet werden.

Der Name der XML-Datei ist gleichzeitig der Zugriffsschlüssel in KRL.

Beispiel: ... \EXT.XML → EKI_INIT("EXT")

```
<ETHERNETKRL>
  <CONFIGURATION>
    <EXTERNAL></EXTERNAL>
    <INTERNAL></INTERNAL>
  </CONFIGURATION>
  <RECEIVE>
    <ELEMENTS></ELEMENTS>
  </RECEIVE>
  <SEND>
    <ELEMENTS></ELEMENTS>
  </SEND>
</ETHERNETKRL>
```

Abschnitt	Beschreibung
<CONFIGURATION> ... </CONFIGURATION>	Konfiguration der Verbindungsparameter zwischen externem System und EKI (>>> 6.1.1 "XML-Struktur für Verbindungseigenschaften" Seite 25)
<RECEIVE> ... </RECEIVE>	Konfiguration der Empfangsstruktur, die von der Robotersteuerung empfangen wird (>>> 6.1.2 "XML-Struktur für den Datenempfang" Seite 28)
<SEND> ... </SEND>	Konfiguration der Sendestructur, die von der Robotersteuerung gesendet wird (>>> 6.1.3 "XML-Struktur für den Datenversand" Seite 31)

6.1.1 XML-Struktur für Verbindungseigenschaften

Beschreibung

Im Abschnitt <EXTERNAL> ... </EXTERNAL> werden die Einstellungen für das externe System definiert:

Element	Beschreibung
TYPE	Legt fest, ob das externe System als Server oder als Client mit der Robotersteuerung kommuniziert (optional) ■ Server: Externes System ist ein Server. ■ Client: Externes System ist ein Client. Default-Wert: Server
IP	IP-Adresse des externen Systems, wenn dieses als Server definiert ist (TYPE = Server) Wenn TYPE = Client, wird die IP-Adresse ignoriert.
PORT	Port-Nummer des externen Systems, wenn dieses als Server definiert ist (TYPE = Server) ■ 1 ... 65534 Hinweis: Bei der Wahl des Ports ist darauf zu achten, dass dieser nicht durch andere Dienste, z. B. vom Betriebssystem verwendet wird. In diesem Fall kann keine Verbindung über diesen Port aufgebaut werden. Wenn TYPE = Client, wird die Port-Nummer ignoriert.

Im Abschnitt <INTERNAL> ... </INTERNAL> werden die Einstellungen für das EKI definiert:

Element	Attribut	Beschreibung
ENVIRONMENT	_____	Löschen der Verbindung an Aktionen koppeln (optional) ■ Program: Löschen nach Aktionen des Roboter-Interpreters ■ Programm zurücksetzen. ■ Programm abwählen. ■ E/As rekonfigurieren. ■ Submit: Löschen nach Aktionen des Submit-Interpreters ■ Submit-Interpreter abwählen. ■ E/As rekonfigurieren. ■ System: Löschen nach Systemaktionen ■ E/As rekonfigurieren. Default-Wert: Program
BUFFERING	Mode	Verfahren, nach dem alle Datenspeicher abgearbeitet werden (optional) ■ FIFO: First In First Out ■ LIFO: Last In First Out Default-Wert: FIFO
	Limit	Maximale Anzahl an Datenelementen, die ein Datenspeicher aufnehmen kann (optional) ■ 1 ... 512 Default-Wert: 16

Element	Attribut	Beschreibung
BUFFSIZE	Limit	Maximale Anzahl an Bytes, die empfangen werden können, ohne dass sie interpretiert werden (optional) ■ 1 ... 65534 Bytes Default-Wert: 16384 Bytes
TIMEOUT	Connect	Zeit, nach der der Versuch eine Verbindung aufzubauen abgebrochen wird (optional) Einheit: ms ■ 0 ... 65534 ms Default-Wert: 2000 ms
ALIVE	Set_Out	Setzen eines Ausgangs oder eines Flags bei erfolgreicher Verbindung (optional) Nummer des Ausgangs: ■ 1 ... 4096 Nummer des Flags: ■ 1 ... 1024 Solange eine Verbindung zum externen System aktiv ist, ist der Ausgang oder das Flag gesetzt. Wenn die Verbindung zum externen System abbricht, wird der Ausgang oder das Flag gelöscht.
	Set_Flag	
	Ping	Intervall für das Senden eines Pings, um die Verbindung zum externen System zu überwachen (optional) ■ 1 ... 65534 s
IP	—	IP-Adresse des EKI, wenn dieses als Server definiert ist (EXTERNAL/TYPE = Client) Wenn EXTERNAL/TYPE = Server, wird die IP-Adresse ignoriert.
PORT	—	Port-Nummer des EKI, wenn dieses als Server definiert ist (EXTERNAL/TYPE = Client) ■ 54600 ... 54615 Wenn EXTERNAL/TYPE = Server, wird die Port-Nummer ignoriert.
PROTOCOL	—	Übertragungsprotokoll (optional) ■ TCP ■ UDP Default-Wert: TCP Es wird empfohlen, immer das TCP/IP-Protokoll zu verwenden.

Element	Attribut	Beschreibung
MESSAGES	Logging	Schreiben von Meldungen in EKI-Logbuch deaktivieren (optional). ■ warning: Warnmeldungen und Fehlermeldungen werden geloggt. ■ error: Nur Fehlermeldungen werden geloggt. ■ disabled: Logging ist deaktiviert. Default-Wert: error
	Display	Meldungsausgabe auf smartHMI deaktivieren (optional). ■ error: Meldungsausgabe ist aktiv. ■ disabled: Meldungsausgabe ist deaktiviert. Default-Wert: error
	(>>> 10.3 "Meldungsausgabe und Loggen von Meldungen deaktivieren" Seite 102)	

Beispiel

```

<CONFIGURATION>
  <EXTERNAL>
    <IP>172.1.10.5</IP>
    <PORT>60000</PORT>
    <TYPE>Server</TYPE>
  </EXTERNAL>
  <INTERNAL>
    <ENVIRONMENT>Program</ENVIRONMENT>
    <BUFFERING Mode="FIFO" Limit="10"/>
    <BUFFSIZE Limit="16384"/>
    <TIMEOUT Connect="60000"/>
    <ALIVE Set_Out="666" Ping="200"/>
    <IP>192.1.10.20</IP>
    <PORT>54600</PORT>
    <PROTOCOL>TCP</PROTOCOL>
    <MESSAGES Logging="error" Display="disabled"/>
  </INTERNAL>
</CONFIGURATION>

```

6.1.2 XML-Struktur für den Datenempfang

Beschreibung

Die Konfiguration ist abhängig davon, ob XML-Daten oder Binärdaten empfangen werden.

- Für den Empfang von XML-Daten muss eine XML-Struktur definiert werden: <XML> ... </XML>
- Für den Empfang von Binärdaten müssen Rohdaten definiert werden: <RAW> ... </RAW>

Attribute in den Elementen der XML-Struktur <XML> ... </XML>:

Element	Attribut	Beschreibung
ELEMENT	Tag	Name des Elements Hier wird die XML-Struktur für den Datenempfang definiert (XPath). (>>> 6.1.4 "Konfiguration nach XPath-Schema" Seite 31)
ELEMENT	Type	Datentyp des Elements ■ STRING ■ REAL ■ INT ■ BOOL ■ FRAME Hinweis: Optional, wenn das Tag nur für Ereignismeldungen verwendet wird. In diesem Fall wird kein Speicherplatz für das Element reserviert. Beispiel für Ereignis-Flag: <ELEMENT Tag="Ext" Set_Flag="56"/>
ELEMENT	Set_Out	Setzen eines Ausgangs oder eines Flags, wenn das Element empfangen wurde (optional) Nummer des Ausgangs: ■ 1 ... 4096 Nummer des Flags: ■ 1 ... 1024 Hinweis: Wenn ein Ausgang oder Flag durch Set_Out/Set_Flag gesetzt wird, muss die zugehörige Systemvariable \$OUT[]/\$FLAG[] im Programmcode wieder zurückgesetzt werden.
	Set_Flag	
ELEMENT	Mode	Verfahren, nach dem ein Datensatz im Datenspeicher abgearbeitet wird ■ FIFO: First In First Out ■ LIFO: Last In First Out Nur relevant, wenn einzelne Datensätze anders behandelt werden sollen wie unter BUFFERING für das EKI konfiguriert.

Attribute für das Element in den Rohdaten <RAW> ... </RAW>:

Element	Attribut	Beschreibung
ELEMENT	Tag	Name des Elements
ELEMENT	Type	Datentyp des Elements ■ BYTE: Binärdatensatz mit fester Länge ■ STREAM: Binärdatensatz mit variabler Endzeichenfolge und variabler Länge

Element	Attribut	Beschreibung
ELEMENT	Set_Out	Setzen eines Ausgangs oder eines Flags, wenn das Element empfangen wurde (optional) Nummer des Ausgangs: ■ 1 ... 4096 Nummer des Flags: ■ 1 ... 1024 Hinweis: Wenn ein Ausgang oder Flag durch Set_Out/Set_Flag gesetzt wird, muss die zugehörige Systemvariable \$OUT[]/\$FLAG[] im Programmcode wieder zurückgesetzt werden.
	Set_Flag	
ELEMENT	EOS	Endzeichenfolge einer elementaren Information (nur relevant, wenn Type = "STREAM") ■ ASCII-Kodierung: 1 ... 32 Zeichen ■ Alternatives Ende wird durch das Zeichen "I" getrennt. Beispiele: ■ <ELEMENT ... EOS="123,134,21"/> ■ <ELEMENT ... EOS="123,134,21I13,10"/>
ELEMENT	Size	Feste Größe der Information, wenn Type = "BYTE" ■ 1 ... 3600 Bytes

Beispiele

```
<RECEIVE>
<XML>
  <ELEMENT Tag="Ext/Str" Type="STRING"/>
  <ELEMENT Tag="Ext/Pos/XPos" Type="REAL" Mode="LIFO"/>
  <ELEMENT Tag="Ext/Pos/YPos" Type="REAL"/>
  <ELEMENT Tag="Ext/Pos/ZPos" Type="REAL"/>
  <ELEMENT Tag="Ext/Temp/Cpu" Type="REAL" Set_Out="1"/>
  <ELEMENT Tag="Ext/Temp/Fan" Type="REAL" Set_Flag="14"/>
  <ELEMENT Tag="Ext/Integer/AState" Type="INT"/>
  <ELEMENT Tag="Ext/Integer/BState" Type="INT"/>
  <ELEMENT Tag="Ext/Boolean/CState" Type="BOOL"/>
  <ELEMENT Tag="Ext/Frames/Frame1" Type="FRAME"/>
  <ELEMENT Tag="Ext/Attributes/@A1" Type="STRING"/>
  <ELEMENT Tag="Ext/Attributes/@A2" Type="INT"/>
  <ELEMENT Tag="Ext" Set_Flag="56"/>
</XML>
</RECEIVE>
```

```
<RECEIVE>
<RAW>
  <ELEMENT Tag="RawData" Type="BYTE" Size="1408"
    Set_Flag="14"/>
</RAW>
</RECEIVE>
```

```
<RECEIVE>
<RAW>
  <ELEMENT Tag="MyStream" Type="STREAM" EOS="123,134,21"
    Size="836" Set_Flag="14"/>
</RAW>
</RECEIVE>
```

```
</RAW>
</RECEIVE>
```

6.1.3 XML-Struktur für den Datenversand

Beschreibung Die Konfiguration ist abhängig davon, ob XML-Daten oder Binärdaten gesendet werden.

- Für das Senden von XML-Daten muss eine XML-Struktur definiert werden: `<XML> ... </XML>`



Die XML-Struktur wird beim Senden in der Reihenfolge angelegt, in der sie konfiguriert ist.

- Das Senden von Binärdaten wird direkt in der KRL-Programmierung realisiert. Es muss keine Konfiguration angegeben werden.

Attribut in den Elementen der XML-Struktur `<XML> ... </XML>`:

Attribut	Beschreibung
Tag	Name des Elements Hier wird die XML-Struktur für den Datenversand definiert (XPath).

Beispiel

```
<SEND>
  <XML>
    <ELEMENT Tag="Robot/Data/ActPos/@X" />
    <ELEMENT Tag="Robot/Data/ActPos/@Y" />
    <ELEMENT Tag="Robot/Data/ActPos/@Z" />
    <ELEMENT Tag="Robot/Data/ActPos/@A" />
    <ELEMENT Tag="Robot/Data/ActPos/@B" />
    <ELEMENT Tag="Robot/Data/ActPos/@C" />
    <ELEMENT Tag="Robot/Status" />
    <ELEMENT Tag="Robot/Mode" />
    <ELEMENT Tag="Robot/Complex/Tickcount" />
    <ELEMENT Tag="Robot/RobotType/Robot/Type" />
  </XML>
</SEND>
```

6.1.4 Konfiguration nach XPath-Schema

Beschreibung Wenn XML verwendet wird um Daten auszutauschen, ist es notwendig, dass die ausgetauschten XML-Dokumente nach demselben Schema aufgebaut sind. Zum Beschreiben und Lesen der XML-Dokumente verwendet KUKA.Ethernet KRL das XPath-Schema.

Nach XPath sind folgende Fälle zu unterscheiden:

- Beschreiben und Lesen von Elementen
- Beschreiben und Lesen von Attributen

Elementschreibweise

- Gespeichertes XML-Dokument für den Datenversand:

```
<Robot>
  <Mode>...</Mode>
  <RobotLamp>
    <GrenLamp>
      <LightOn>...</LightOn>
    </GrenLamp>
```

```
</RobotLamp>
</Robot>
```

- Konfigurierte XML-Struktur für den Datenversand:

```
<SEND>
<XML>
  <ELEMENT Tag="Robot/Mode" />
  <ELEMENT Tag="Robot/RobotLamp/GrenLamp/LightOn" />
</XML>
<SEND />
```

Attributschreibweise

- Gespeichertes XML-Dokument für den Datenversand:

```
<Robot>
  <Data>
    <ActPos X="...">
  </ActPos>
    <LastPos A="..." B="..." C="..." X="..." Y="..." Z="...">
  </LastPos>
  </Data>
</Robot>
```

- Konfigurierte XML-Struktur für den Datenversand:

```
<SEND>
<XML>
  <ELEMENT Tag="Robot/Data/LastPos/@X" />
  <ELEMENT Tag="Robot/Data/LastPos/@Y" />
  ...
  <ELEMENT Tag="Robot/Data/ActPos/@X" />
</XML>
<SEND />
```

6.2 Funktionen für den Datenaustausch

Übersicht

Für den Datenaustausch zwischen Robotersteuerung und externem System stellt KUKA.Ethernet KRL Funktionen zur Verfügung.

Die genaue Beschreibung der Funktionen ist im Anhang zu finden.
(>>> 10.4 "Befehlsreferenz" Seite 102)

Verbindung initialisieren, öffnen, schließen und löschen

```
EKI_STATUS = EKI_Init(CHAR[])
EKI_STATUS = EKI_Open(CHAR[])
EKI_STATUS = EKI_Close(CHAR[])
EKI_STATUS = EKI_Clear(CHAR[])
```

Daten senden

```
EKI_STATUS = EKI_Send(CHAR[], CHAR[], INT)
```

Daten schreiben

```
EKI_STATUS = EKI_SetReal(CHAR[], CHAR[], REAL)
EKI_STATUS = EKI_SetInt(CHAR[], CHAR[], INT)
EKI_STATUS = EKI_SetBool(CHAR[], CHAR[], BOOL)
EKI_STATUS = EKI_SetFrame(CHAR[], CHAR[], FRAME)
EKI_STATUS = EKI_SetString(CHAR[], CHAR[], CHAR[])
```


Daten auslesen
EKI_STATUS = EKI_GetBool(CHAR[], CHAR[], BOOL)
EKI_STATUS = EKI_GetBoolArray(CHAR[], CHAR[], BOOL[])
EKI_STATUS = EKI_GetInt(CHAR[], CHAR[], INT)
EKI_STATUS = EKI_GetIntArray(CHAR[], CHAR[], INT[])
EKI_STATUS = EKI_GetReal(CHAR[], CHAR[], REAL)
EKI_STATUS = EKI_GetRealArray(CHAR[], CHAR[], REAL[])
EKI_STATUS = EKI_GetString(CHAR[], CHAR[], CHAR[])
EKI_STATUS = EKI_GetFrame(CHAR[], CHAR[], FRAME)
EKI_STATUS = EKI_GetFrameArray(CHAR[], CHAR[], FRAME[])
Funktion auf Fehler prüfen
EKI_CHECK(EKI_STATUS, EKrIMsgType, CHAR[])
Datenspeicher löschen, sperren, entsperren und prüfen
EKI_STATUS = EKI_Clear(CHAR[])
EKI_STATUS = EKI_ClearBuffer(CHAR[], CHAR[])
EKI_STATUS = EKI_Lock(CHAR[])
EKI_STATUS = EKI_Unlock(CHAR[])
EKI_STATUS = EKI_CheckBuffer(CHAR[], CHAR[])

6.2.1 Programmiertipps

- Wenn eine Verbindung im Submit-Interpreter angelegt wird, sind folgende Punkte zu beachten:
 - In der Verbindungskonfiguration muss über das Element <ENVIRONMENT> angegeben werden, dass es sich um einen Submit-Kanal handelt.
 - Ein im Submit-Interpreter geöffneter Kanal kann auch vom Roboter-Interpreter angesprochen werden.
 - Wird der Submit-Interpreter abgewählt, wird die Verbindung per Konfiguration automatisch gelöscht.
- Wenn Submit-Interpreter und Roboter-Interpreter parallel auf einen Kanal zugreifen, sind folgende Punkte zu beachten:
 - Werden Daten über EKI_Get...() ausgelesen, wird der Inhalt dieses Speichers gelöscht. Aus diesem Grund ist es nicht sinnvoll, Daten parallel über Submit-Interpreter und Roboter-Interpreter auszulesen.
 - Submit-Interpreter und Roboter-Interpreter können mit EKI_Set...() schreibend auf den gleichen Kanal zugreifen. Dabei können sich die Aufrufe gegenseitig die Daten überschreiben, wenn auf dasselbe Ziel-element zugegriffen wird und nicht zwischendurch EKI_Send() aufgerufen wird.
 - Die Ausführungszeitpunkte in verschiedenen Interpretern können sich zueinander verändern, so dass sich bei konkurrierenden Schreib- oder Lesezugriffen das Verhalten unerwartet ändern kann.



Es wird empfohlen, für jede Sende- und Empfangsaufgabe einen separaten Kanal zu öffnen.



Das beschriebene Sende- und Empfangsverhalten trifft auch zu, wenn KUKA.MultiSubmitInterpreter verwendet wird.



EKI-Anweisungen werden im Vorlauf ausgeführt!

- Wenn eine EKI-Anweisung im Hauptlauf ausgeführt werden soll, müssen Anweisungen verwendet werden, die einen Vorlaufstopp auslösen, z. B. WAIT SEC.
- Da jeder Zugriff auf den Speicher Zeit kostet, wird empfohlen große Datenmengen mit den Feldzugriffsfunktionen EKI_Get...Array() abzurufen.
- KUKA.Ethernet KRL kann über EKI_Get...Array() auf maximal 512 Feldelemente zugreifen. In KRL können zwar größere Felder angelegt werden, z. B. myFrame[1000], lesbar sind aber immer nur maximal 512 Elemente.
- Es gibt verschiedene Möglichkeiten auf Daten zu warten:
 - Über das Setzen eines Flags oder eines Ausgangs kann angezeigt werden, dass ein bestimmtes Datenelement oder ein vollständiger Datensatz empfangen wurde. (Element Set_Flag oder Set_Out in der XML-Struktur für den Datenempfang).
Beispiel: Interrupt des KRL-Programms über WAIT FOR \$FLAG[x] (>>> 7.7 "Beispiel "XmlCallback"" Seite 55)
 - Über die Funktion EKI_CheckBuffer() kann zyklisch abgefragt werden, ob neue Datenelemente im Speicher vorhanden sind.

6.2.2 Initialisieren und Löschen einer Verbindung

Beschreibung

Eine Verbindung muss mit der Funktion EKI_Init() angelegt und initialisiert werden. Die in der Funktion angegebene Verbindungskonfiguration wird dabei eingelesen. Eine Verbindung kann sowohl im Roboter- als auch im Submit-Interpreter angelegt werden.

Eine Verbindung kann im Roboter- oder Submit-Interpreter über die Funktion EKI_Clear() wieder gelöscht werden. Zusätzlich kann das Löschen einer Verbindung an Roboter- und Submit-Interpreter-Aktionen oder Systemaktionen gekoppelt sein. (Konfigurierbar über das Element <ENVIRONMENT> in der Verbindungskonfiguration)

Konfiguration "Program"

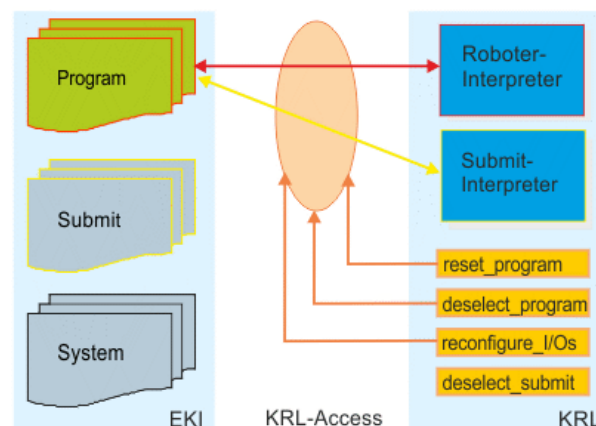


Abb. 6-1: Verbindungskonfiguration "Program"

Bei dieser Konfiguration wird eine Verbindung nach folgenden Aktionen gelöscht.

- Programm zurücksetzen.
- Programm abwählen.
- E/As rekonfigurieren.



Beim Rekonfigurieren der E/As wird der Treiber neu geladen und alle Initialisierungen gelöscht.

Konfiguration "Submit"

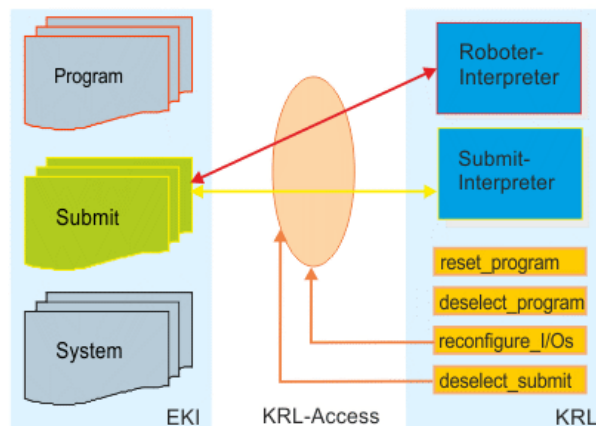


Abb. 6-2: Verbindungskonfiguration "Submit"

Bei dieser Konfiguration wird eine Verbindung nach folgenden Aktionen gelöscht.

- Submit-Interpreter abwählen.
- E/As rekonfigurieren.



Wenn KUKA.MultiSubmitInterpreter verwendet wird, kann die Verbindung nur nach dem Abwählen des System-Submit-Interpreters beendet werden. Submit-Interpreter, die zusätzlich zum System-Submit-Interpreter zur Verfügung stehen, haben keinen Einfluss auf das Beenden der Verbindung.



Beim Rekonfigurieren der E/As wird der Treiber neu geladen und alle Initialisierungen gelöscht.

Konfiguration "System"

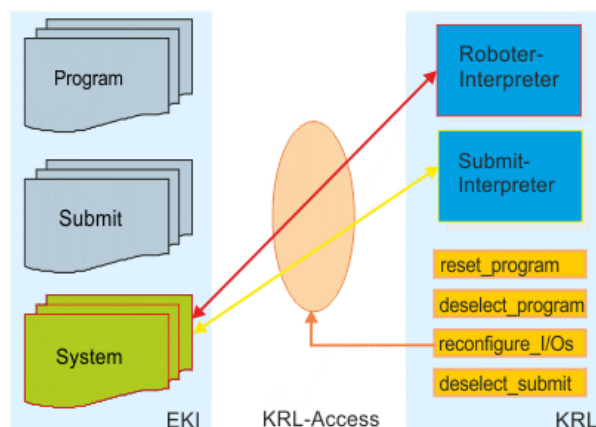


Abb. 6-3: Verbindungskonfiguration "System"

Bei dieser Konfiguration wird eine Verbindung nach folgenden Aktionen gelöscht.

- E/As rekonfigurieren.



Beim Rekonfigurieren der E/As wird der Treiber neu geladen und alle Initialisierungen gelöscht.

6.2.3 Öffnen und Schließen einer Verbindung

Beschreibung

Die Verbindung zum externen System wird über ein KRL-Programm hergestellt. Die meisten KRL-Programme sind wie folgt aufgebaut:

```

1 DEF Connection()
2 ...
3 RET=EKI_Init("Connection")
4 RET=EKI_Open("Connection")
5 ...
6 Write data, send data or get received data
7 ...
8 RET=EKI_Close("Connection")
9 RET=EKI_Clear("Connection")
10 ...
11 END

```

Zeile	Beschreibung
3	EKI_Init() initialisiert den Kanal, über den sich das EKI mit dem externen System verbindet.
4	EKI_Open() öffnet den Kanal.
6	KRL-Anweisungen, um Daten in den Speicher zu schreiben, Daten zu senden oder auf empfangene Daten zuzugreifen
8	EKI_Close() schließt den Kanal.
9	EKI_Clear() löscht den Kanal.

Bei der Programmierung ist zu beachten, ob das EKI als Server oder als Client konfiguriert ist.

Server-Betrieb

Wenn das externe System als Client konfiguriert ist, versetzt EKI_Open() das EKI (= Server) in einen Abhörzustand. Der Server wartet auf die Verbindungsanfrage eines Clients, ohne dass der Programmablauf unterbrochen wird. Wenn in der Konfigurationsdatei das Element <TIMEOUT Connect="..."/> nicht beschrieben wird, wartet der Server so lange, bis ein Client eine Verbindung anfordert.

Eine Verbindungsanfrage durch einen Client wird durch Zugriff auf das EKI oder durch eine Ereignismeldung signalisiert, z. B. über das Element <ALIVE SET_OUT="..."/>.

Wenn der Programmablauf unterbrochen werden soll solange der Server die Verbindungsanfrage erwartet, muss ein Ereignis-Flag oder -Ausgang programmiert werden, z. B. WAIT FOR \$OUT[...].



Es wird empfohlen, EKI_Close() im Server-Betrieb nicht zu verwenden. Im Server-Betrieb wird der Kanal vom externen Client aus geschlossen.

Client-Betrieb

Wenn das externe System als Server konfiguriert ist, unterbricht EKI_Open() den Programmablauf bis die Verbindung zum externen System aktiv ist. EKI_Close() schließt die Verbindung zum externen Server.

6.2.4 Senden von Daten

Beschreibung

Je nach Konfiguration und Programmierung können folgende Daten mit EKI_Send() gesendet werden:

Bei Kommunikation über XML-Struktur:

- Vollständige XML-Struktur
- Teil einer XML-Struktur

- Beliebige Zeichenkette variabler Länge

Bei Kommunikation über Rohdaten:

- Bei Binärdatensätzen mit fester Länge (Attribut `Type = "BYTE"`): Beliebige Zeichenkette mit fester Länge
 - Die Größe der Zeichenkette in Bytes muss genau dem konfigurierten Attribut `Size` entsprechen. Bei Überschreitung wird eine Fehlermeldung, bei Unterschreitung eine Warnmeldung ausgegeben.
 - Binärdatensätze fester Länge müssen im KRL-Programm mit `CAST_TO()` eingelesen werden. Es sind nur Daten vom Typ `REAL` (4 Bytes) lesbar, kein `Double`.



Detaillierte Informationen zum Befehl `CAST_TO()` sind in der Dokumentation `CREAD/CWRITE` zu finden.

- Bei Binärdatensätzen mit variabler Endzeichenfolge (Attribut `Type = "STREAM"`): Beliebige Zeichenkette mit variabler Länge
Die Endzeichenfolge wird automatisch mitgesendet.

Beispiele

Senden von XML-Daten:

Senden einer vollständigen XML-Struktur:

- Gespeicherte XML-Struktur für den Datenversand:

```
<Robot>
  <ActPos X="1000.12"></ActPos>
  <Status>12345678</Status>
</Robot>
```

- Programmierung:

```
DECL EKI_STATUS RET
RET=EKI_Send("Channel_1", "Robot")
```

- Gesendete XML-Struktur:

```
<Robot>
  <ActPos X="1000.12"></ActPos>
  <Status>12345678</Status>
</Robot>
```

Senden eines Teils der XML-Struktur:

- Gespeicherte XML-Struktur für den Datenversand:

```
<Robot>
  <ActPos X="1000.12"></ActPos>
  <Status>12345678</Status>
</Robot>
```

- Programmierung:

```
DECL EKI_STATUS RET
RET=EKI_Send("Channel_1", "Robot/ActPos")
```

- Gesendete XML-Struktur:

```
<Robot>
  <ActPos X="1000.12"></ActPos>
</Robot>
```

Direktes Senden einer Zeichenkette:

- Gespeicherte XML-Struktur für den Datenversand (wird beim direkten Senden nicht verwendet):

```
<Robot>
  <ActPos X="1000.12"></ActPos>
  <Status>12345678</Status>
</Robot>
```

■ **Programmierung:**

```
DECL EKI_STATUS RET
RET=EKI_Send("Channel_1", "<POS><XPOS>1</XPOS></POS>")
```

■ **Gesendete Zeichenkette:**

```
<POS><XPOS>1</XPOS></POS>
```

Senden von Binärdaten:

Senden eines Binärdatensatzes fester Länge (10 Byte):

■ **Konfigurierte Rohdaten:**

```
<RAW>
  <ELEMENT Tag="Buffer" Type="BYTE" Size="10" />
</RAW>
```

■ **Programmierung:**

```
DECL EKI_STATUS RET
CHAR Bytes[10]
OFFSET=0
CAST_TO(Bytes[], OFFSET, 91984754, 913434.2, TRUE, "X")
RET=EKI_Send("Channel_1", Bytes[])
```

■ **Gesendete Daten:**

```
"r?{ ? _I X"
```

Senden eines Binärdatensatzes mit Endzeichenfolge und variabler Länge:

■ **Konfigurierte Rohdaten:**

```
<RAW>
  <ELEMENT Tag="Buffer" Type="STREAM" EOS="65,66" />
</RAW>
```

■ **Programmierung:**

```
DECL EKI_STATUS RET
CHAR Bytes[64]
Bytes[]="Stream ends with:"
RET=EKI_Send("Channel_1", Bytes[])
```

■ **Gesendete Daten:**

```
"Stream ends with:AB"
```

Senden eines auf ein Maximum an Zeichen begrenzten Binärdatensatzes mit Endzeichenfolge und variabler Länge:

■ **Konfigurierte Rohdaten:**

```
<RAW>
  <ELEMENT Tag="Buffer" Type="STREAM" EOS="65,66" />
</RAW>
```

■ **Programmierung:**

```
DECL EKI_STATUS RET
CHAR Bytes[64]
Bytes[]="Stream ends with:"
RET=EKI_Send("Channel_1", Bytes[], 6)
```

- Gesendete Daten:

```
"StreamAB"
```

6.2.5 Auslesen von Daten



Zum Auslesen von Daten müssen die zugehörigen KRL-Variablen initialisiert sein, z. B. durch die Zuweisung von Werten.

Beschreibung

Beim Speichern und Auslesen der Daten werden XML- und Binärdaten unterschiedlich behandelt:

- XML-Daten werden vom EKI extrahiert und typrichtig in verschiedene Speicher abgelegt. Es ist möglich auf jeden gespeicherten Wert einzeln zuzugreifen.
Um XML-Daten auszulesen, können alle Zugriffsfunktionen EKI_Get...() verwendet werden.
- Binärdatensätze werden vom EKI nicht interpretiert und als Ganzes in einem Speicher abgelegt.
Um einen Binärdatensatz aus einem Speicher zu lesen, muss die Zugriffsfunktion EKI_GetString() verwendet werden. Binärdatensätze werden als Zeichenfolgen aus dem Speicher gelesen.
Binärdatensätze fester Länge müssen im KRL-Programm mit CAST_FROM() wieder in einzelne Variablen aufgeteilt werden.



Detaillierte Informationen zum Befehl CAST_FROM() sind in der Dokumentation CREAD/CWRITE zu finden.

Beispiele

Auslesen von XML-Daten:

XML-Struktur für den Datenempfang:

```
<RECEIVE>
  <XML>
    <ELEMENT Tag="Sensor/Message" Type="STRING" />
    <ELEMENT Tag="Sensor/Status/IsActive" Type="BOOL" />
  </XML>
</RECEIVE>
```

Programmierung:

```
; Declaration
INT i
DECL EKI_STATUS RET
CHAR valueChar[256]
BOOL valueBOOL

; Initialization
FOR i=(1) TO (256)
  valueChar[i]=0
ENDFOR
valueBOOL=FALSE

RET=EKI_GetString("Channel_1", "Sensor/Message", valueChar[])
RET=EKI_GetBool("Channel_1", "Sensor/Status/IsActive", valueBOOL)
```

Mit Sensordaten beschriebenes XML-Dokument (empfangene Daten):

```
<Sensor>
  <Message>Example message</Message>
```

```
<Status>
  <IsActive>1</IsActive>
</Status>
</Sensor>
```

Auslesen eines Binärdatensatzes fester Länge (10 Byte):

■ Konfigurierte Rohdaten:

```
<RECEIVE>
  <RAW>
    <ELEMENT Tag="Buffer" Type="BYTE" Size="10" />
  </RAW>
</RECEIVE>
```

■ Programmierung:

```
; Declaration
INT i
INT OFFSET
DECL EKI_STATUS RET
CHAR Bytes[10]
INT valueInt
REAL valueReal
BOOL valueBool
CHAR valueChar[1]

; Initialization
FOR i=(1) TO (10)
  Bytes[i]=0
ENDFOR

OFFSET=0
valueInt=0
valueBool=FALSE
valueReal=0
valueChar[1]=0

RET=EKI_GetString("Channel_1", "Buffer", Bytes[])

OFFSET=0
CAST_FROM(Bytes[],OFFSET,valueReal,valueInt,valueChar[],valueBool)
```

Auslesen eines Binärdatensatzes mit Endzeichenfolge:

■ Konfigurierte Rohdaten:

```
<RECEIVE>
  <RAW>
    <ELEMENT Tag="Buffer" Type="STREAM" EOS="13,10" />
  </RAW>
</RECEIVE>
```

■ Programmierung:

```
; Declaration
INT i
DECL EKI_STATUS RET
CHAR Bytes[64]

; Initialization
FOR i=(1) TO (64)
  Bytes[i]=0
ENDFOR
```



```
RET=EKI_GetString("Channel_1", "Buffer", Bytes[])
```

6.2.6 Löschen von Datenspeichern

Beschreibung

Beim Löschen von Datenspeichern sind folgende Fälle zu unterscheiden:

- Löschen über EKI_Clear(): Löschen aller Datenspeicher und Beenden der Ethernet-Verbindung
- Löschen über EKI_ClearBuffer(): Löschen von bestimmten Datenspeichern ohne Beenden der Ethernet-Verbindung

EKI_ClearBuffer()

Welcher Datenspeicher mit dem Befehl EKI_ClearBuffer() gelöscht werden kann, ist abhängig davon, ob über Rohdaten oder eine XML-Struktur kommuniziert wird.

Bei Kommunikation über Rohdaten:

- Der Datenspeicher für empfangene Daten kann gelöscht werden.

Bei Kommunikation über XML-Struktur:

- Über die Angabe des XPath-Ausdrucks eines Elements aus der konfigurierten XML-Struktur kann dessen Datenspeicher gezielt gelöscht werden. Dies gilt sowohl für empfangene, als auch für zu sendende Daten.



XML-Daten werden vom EKI extrahiert und typrichtig in verschiedene Speicher abgelegt. Beim Löschen einzelner Speicher muss sichergestellt sein, dass keine zusammengehörigen Daten verlorengehen.

Beispiele

Kommunikation über Rohdaten:

Löschen des Datenspeichers für empfangene Daten:

- Konfigurierte Rohdaten:

```
<RECEIVE>
  <RAW>
    <ELEMENT Tag="RawData" Type="BYTE" Size="1408" Set_Flag="14"/>
  </RAW>
</RECEIVE>
```

- Programmierung:

```
DECL EKI_STATUS RET
RET=EKI_ClearBuffer("Channel_1", "RawData")
```

Kommunikation über XML-Struktur:

Löschen des Datenspeichers zum Tag <Flag>:

- Konfigurierte XML-Struktur für den Datenempfang:

```
<RECEIVE>
  <XML>
    <ELEMENT Tag="Ext/Activ/Value" Type="REAL"/>
    <ELEMENT Tag="Ext/Activ/Flag" Type="BOOL"/>
    <ELEMENT Tag="Ext/Activ/Flag/Message" Type="STRING"/>
  </XML>
</RECEIVE>
```

Der Datenspeicher zum Tag <Message> wird ebenfalls gelöscht, da dieses Tag dem Tag <Flag> untergeordnet ist.

- Programmierung:

```
DECL EKI_STATUS RET
RET=EKI_ClearBuffer("Channel_1", "Ext/Activ/Flag")
```

6.2.7 EKI_STATUS – Struktur für funktionsspezifische Rückgabewerte

Beschreibung Jede Funktion des EKI gibt funktionsspezifische Werte zurück. EKI_STATUS ist die globale Strukturvariable, in die diese Werte geschrieben werden.

Syntax GLOBAL STRUC EKI_STATUS INT Buff, Read, Msg_No, BOOL Connected, INT Counter

Erläuterung der Syntax

Element	Beschreibung
Buff	Anzahl der Elemente, die sich nach einem Zugriff noch im Speicher befinden
Read	Anzahl der Elemente, die aus dem Speicher gelesen wurden
Msg_No	Fehlernummer des Fehlers, der beim Aufruf einer Funktion oder beim Datenempfang aufgetreten ist Wenn die automatische Meldungsausgabe deaktiviert wurde, kann mit EKI_CHECK() die Fehlernummer ausgelesen und die Fehlermeldung auf der smartHMI ausgegeben werden.
Connected	Gibt an, ob eine Verbindung besteht ■ TRUE = Verbindung vorhanden ■ FALSE = Verbindung unterbrochen
Counter	Zeitstempel für empfangene Datenpakete Im Speicher eintreffende Datenpakete werden fortlaufend nummeriert, und zwar in der Reihenfolge, in der sie im Speicher abgelegt werden. Wenn einzelne Daten gelesen werden, wird das Strukturelement <i>Counter</i> mit dem Zeitstempel des Datenpakets belegt, aus dem das Datenelement stammt. (>>> 6.2.10 "Verarbeiten unvollständiger Datensätze" Seite 44)

Rückgabewerte Abhängig von der Funktion werden folgende Elemente der Struktur EKI_STATUS beschrieben:

Funktion	Buff	Read	Msg_No	Connected	Counter
EKI_Init()	✗	✗	✓	✗	✗
EKI_Open()	✗	✗	✓	✓	✗
EKI_Close()	✗	✗	✓	✗	✗
EKI_Clear()	✗	✗	✓	✗	✗
EKI_Send()	✗	✗	✓	✓	✗
EKI_Set...()	✗	✗	✓	✓	✗
EKI_Get...()	✓	✓	✓	✓	✓
EKI_ClearBuffer()	✗	✗	✓	✓	✗
EKI_Lock()	✗	✗	✓	✓	✗
EKI_UnLock()	✗	✗	✓	✓	✗
EKI_CheckBuffer()	✓	✗	✓	✓	✓

6.2.8 Konfigurieren von Ereignismeldungen

Über das Setzen eines Ausganges oder Flags können folgende Ereignisse gemeldet werden:

- Verbindung ist aktiv.
- Ein einzelnes XML-Element ist angekommen.
- Eine vollständige XML-Struktur oder ein vollständiger Binärdatensatz ist angekommen.

Ereignis-Ausgang

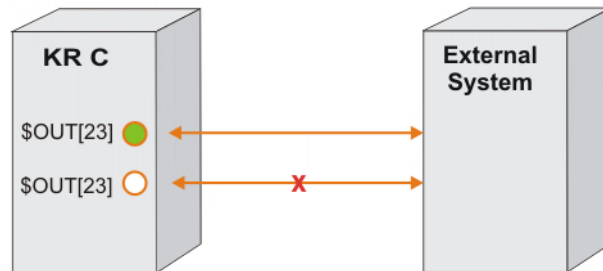


Abb. 6-4: Ereignis-Ausgang (Verbindung aktiv)

\$OUT[23] ist gesetzt, solange die Verbindung zum externen System aktiv ist. Wenn die Verbindung nicht mehr aktiv ist, wird \$OUT[23] zurückgesetzt.



Die Verbindung kann nur mit der Funktion EKI_OPEN() wiederhergestellt werden.

Ereignis-Flag

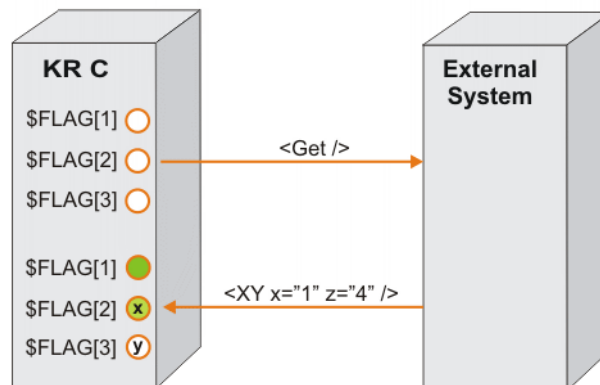


Abb. 6-5: Ereignis-Flag (Vollständige XML-Struktur)

Die XML-Struktur `<XY />` enthält die Datenelemente "XY/@x" und "XY/@z". \$FLAG[1] wird gesetzt, da die vollständige XML-Struktur angekommen ist. \$FLAG[2] wird gesetzt, da das Element "x" in "XY" enthalten ist. \$FLAG[3] wird nicht gesetzt, da das Element "y" nicht übermittelt wurde.

Beispiel

(>>> 7.7 "Beispiel "XmlCallback"" Seite 55)

6.2.9 Empfang vollständiger XML-Datensätze

Beschreibung

Die Zugriffsfunktionen EKI_Get...() sind solange gesperrt bis alle Daten eines XML-Datensatzes im Speicher liegen.

Wenn LIFO konfiguriert ist und 2 oder mehr XML-Datensätze direkt hintereinander ankommen, ist nicht mehr sichergestellt, dass ein Datensatz zusammenhängend aus dem Speicher geholt wird. Es kann z. B. vorkommen, dass die Daten des zweiten Datensatzes bereits im Speicher abgelegt werden, ob-

wohl der erste Datensatz noch nicht vollständig abgearbeitet ist. Da im LIFO-Betrieb immer zuerst auf die zuletzt gespeicherten Daten zugegriffen wird, ist der in KRL verfügbare Datensatz inkonsistent.

Um die Fragmentierung von Datensätzen im LIFO-Betrieb zu verhindern, muss die Verarbeitung neu empfangener Daten gesperrt werden bis alle zusammengehörigen Daten aus dem Speicher geholt wurden.

Beispiel

```
...
RET=EKI_Lock("MyChannel")
RET=EKI_Get...()
RET=EKI_Get...()
...
RET=EKI_Get...()
RET=EKI_Unlock("MyChannel")
...
```

6.2.10 Verarbeiten unvollständiger Datensätze

Es kann vorkommen, dass ein externes System unvollständige Datensätze sendet. Einzelne XML-Elemente sind entweder leer oder fehlen ganz, so dass in einer Speicherschicht Daten aus verschiedenen Datenpaketen liegen.

Wenn die Datensätze zusammenhängend in KRL vorhanden sein müssen, kann das Strukturelement Counter der Variablen EKI_STATUS benutzt werden. Bei der Verwendung von EKI_Get...Array-Funktionen werden zeitlich nicht zusammenhängende Daten daran erkannt, dass Counter = 0 zurückgegeben wird.

6.2.11 EKI_CHECK() – Funktionen auf Fehler prüfen

Beschreibung

KUKA.Ethernet KRL gibt bei jedem Fehler eine Meldung auf der smarHMI aus. Die automatische Ausgabe von Meldungen kann deaktiviert werden.

(>>> 10.3 "Meldungsausgabe und Loggen von Meldungen deaktivieren" Seite 102)

Wenn die automatische Meldungsausgabe deaktiviert worden ist, wird empfohlen mit EKI_CHECK() zu prüfen, ob beim Ausführen einer Funktion ein Fehler aufgetreten ist:

- Die Fehlernummer wird ausgelesen und die zugehörige Meldung auf der smarHMI ausgegeben.
- Wird in EKI_CHECK() ein Kanalname angegeben, wird beim Datenempfang abgefragt, ob Fehler vorliegen.

(>>> 10.4.5 "Funktion auf Fehler prüfen" Seite 109)

Bei jedem Aufruf von EKI_CHECK() wird das Programm KRC:\R1\TP\EthernetKRL\EthernetKRL_USER.SRC aufgerufen. In diesem Programm können benutzerspezifische Fehlerreaktionen programmiert werden.

Beispiel

Eine Verbindung wird immer geschlossen, wenn ein Fehler im Empfang auftritt. Als Fehlerstrategie kann für den Fall, dass die Ethernet-Verbindung abbricht, ein Interrupt programmiert werden.

- In der Konfigurationsdatei XmlTransmit.XML ist definiert, dass bei erfolgreicher Verbindung FLAG[1] gesetzt wird. Bei Verbindungsverlust wird FLAG[1] zurückgesetzt.

```
<ALIVE Set_Flag="1"/>
```

- Im KRL-Programm wird der Interrupt deklariert und eingeschaltet. Wird FLAG[1] zurückgesetzt, wird das Interrupt-Programm ausgeführt.

```
;FOLD Define callback
  INTERRUPT DECL 89 WHEN $FLAG[1]==FALSE DO CON_ERR()
  INTERRUPT ON 89
;ENDFOLD
```

- Im Interrupt-Programm wird mit EKI_CHECK() abgefragt, was für ein Fehler aufgetreten ist, und dann die Verbindung wieder geöffnet.

```
DEF CON_ERR()
  DECL EKI_STATUS RET
  RET={Buff 0,Read 0, Msg_no 0, Connected false}
  EKI_CHECK(RET,#Quit,"XmlTransmit")
  RET=EKI_OPEN("XmlTransmit")
END
```


7 Beispielkonfigurationen und -programme

Im Lieferumfang von KUKA.Ethernet KRL sind ein Server-Programm sowie verschiedene Beispielkonfigurationen und -programme enthalten. Mithilfe dieser Beispielkonfigurationen und -programme kann eine Kommunikation zwischen dem Server-Programm und der Robotersteuerung hergestellt werden.

Komponenten	Verzeichnis
Server-Programm ■ EthernetKRL_Server.exe	DOC\Example\Application
Beispielprogramme in KRL ■ BinaryFixed.src ■ BinaryStream.src ■ XmlCallback.src ■ XmlServer.src ■ XmlTransmit.src	DOC\Example\Program
Beispielkonfigurationen in XML ■ BinaryFixed.xml ■ BinaryStream.xml ■ XmlCallBack.xml ■ XmlServer.xml ■ XmlTransmit.xml ■ XmlFullConfig.xml	DOC\Example\Config

7.1 Server-Programm und Beispiele einbinden

- Voraussetzung**
- Externes System:
- Windows-Betriebssystem mit installiertem .NET-Framework 3.5 oder höher
- Robotersteuerung:
- Benutzergruppe Experte
 - Betriebsart T1 oder T2
- Vorgehensweise**
1. Server-Programm auf externes System kopieren.
 2. Die XML-Beispielkonfigurationen in das Verzeichnis C:\KRC\ROBOTER\Config\User\Common\EthernetKRL der Robotersteuerung einfügen.
 3. Die KRL-Beispielprogramme in das Verzeichnis C:\KRC\ROBOTER\Program der Robotersteuerung einfügen.
 4. Server-Programm auf dem externen System starten.
(>>> 7.2 "Bedienoberfläche Server-Programm" Seite 48)
 5. Menü-Button drücken. Das Fenster **Communication Properties** öffnet sich.
(>>> 7.2.1 "Kommunikationsparameter im Server-Programm einstellen" Seite 49)
 6. Nur wenn am externen System mehrere Netzwerkschnittstellen zur Verfügung stehen: Nummer des Netzwerkadapters (= Netzwerkkartenindex) eingeben, der zur Kommunikation mit der Robotersteuerung genutzt wird.
 7. Fenster **Communication Properties** schließen und Start-Button drücken. Die zur Kommunikation verfügbare IP-Adresse wird im Meldungsfenster angezeigt.

8. Angezeigte IP-Adresse des externen Systems in der gewünschten XML-Datei einstellen.

7.2 Bedienoberfläche Server-Programm

Beschreibung

Das Server-Programm ermöglicht es, die Kommunikation zwischen einem externen System und der Robotersteuerung zu testen, indem eine stabile Verbindung zur Robotersteuerung hergestellt wird.

Das Server-Programm enthält folgende Funktionalitäten:

- Senden und Empfangen von Daten (automatisch oder manuell)
- Anzeige der empfangenen Daten
- Anzeige der gesendeten Daten

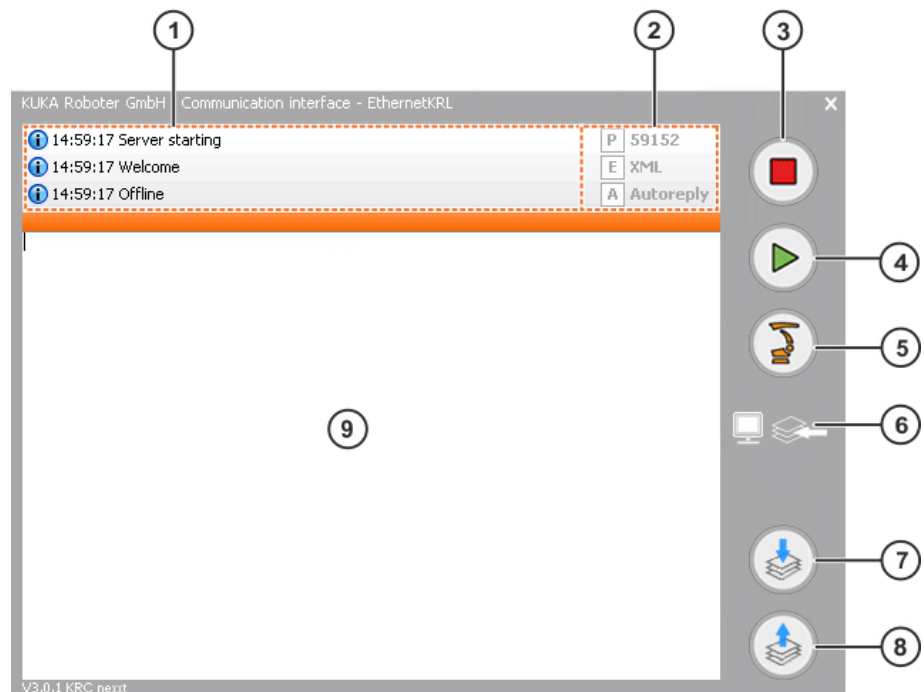


Abb. 7-1: Server-Programm Bedienoberfläche

Pos.	Beschreibung
1	Meldungsfenster
2	Anzeige der eingestellten Kommunikationsparameter (>>> 7.2.1 "Kommunikationsparameter im Server-Programm einstellen" Seite 49) ■ P: Port-Nummer ■ E: Beispieldaten ■ Xml: XML-Daten ■ BinaryFixed: Binärdaten mit fester Länge ■ BinaryStream: Binärdatenstrom variabel mit Endzeichenfolge ■ A: Kommunikationsmodus ■ Autoreply: Der Server beantwortet jedes empfangene Datenpaket automatisch. ■ Manual: Nur manueller Datenempfang oder Datenversand

Pos.	Beschreibung
3	Stopp-Button Die Kommunikation mit der Robotersteuerung wird beendet und der Server wird zurückgesetzt.
4	Start-Button Der Datenaustausch zwischen Server-Programm und Robotersteuerung wird ausgewertet. Die erste eingehende Verbindungsanfrage wird gebunden und als Kommunikationsadapter benutzt.
5	Menü-Button zum Einstellen der Kommunikationsparameter (>>> 7.2.1 "Kommunikationsparameter im Server-Programm einstellen" Seite 49)
6	Anzeigeoptionen ■ Pfeil zeigt nach links: Die empfangenen Daten werden angezeigt. (Default) ■ Pfeil zeigt nach rechts: Die gesendeten Daten werden angezeigt.
7	Button für den manuellen Datenempfang
8	Button für den manuellen Datenversand
9	Anzeigefenster Je nach eingestellter Anzeigeoption werden die gesendeten oder die empfangenen Daten angezeigt.

7.2.1 Kommunikationsparameter im Server-Programm einstellen

- Vorgehensweise**
1. Im Server-Programm auf den Menü-Button klicken.
Das Fenster **Communication Properties** öffnet sich.
 2. Kommunikationsparameter einstellen.
 3. Fenster schließen.

Beschreibung

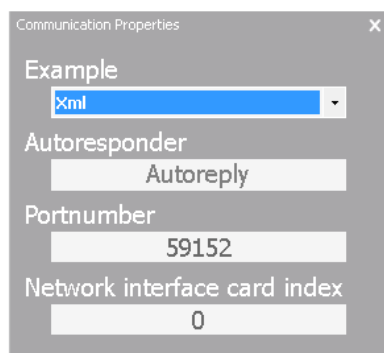


Abb. 7-2: Fenster Communication Properties

Parameter	Beschreibung
Example	Beispieldaten auswählen. ■ Xml: XML-Daten ■ BinaryFixed: Binärdaten mit fester Länge ■ BinaryStream: Binärdatenstrom variabel mit Endzeichenfolge Default-Wert: xml
Autoresponder	Kommunikationsmodus auswählen. ■ Autoreply: Der Server beantwortet jedes empfangene Datenpaket automatisch. ■ Manual: Nur manueller Datenempfang oder Datenversand Default-Wert: Autoreply
Portnumber	Port-Nummer der Socket-Verbindung eingeben. An diesem Port erwartet das externe System die Verbindungsanfrage der Robotersteuerung. Es muss eine freie Nummer gewählt werden, die nicht als Standarddienst belegt ist. Default-Wert: 59152 Hinweis: Bei der Wahl des Ports ist darauf zu achten, dass dieser nicht durch andere Dienste, z. B. vom Betriebssystem verwendet wird. In diesem Fall kann keine Verbindung über diesen Port aufgebaut werden.
Network interface card index	Nummer des Netzwerkadapters eingeben. Nur relevant, wenn das externe System mehrere Netzwerkkarten benutzt, z. B. WLAN und LAN. Default-Wert: 0

7.3 Beispiel "BinaryFixed"



Zur Kommunikation mit der Robotersteuerung müssen im Server-Programm die passenden Beispieldaten eingestellt sein, hier "BinaryFixed".

EKI ist als Client konfiguriert. Über die Verbindung können nur Binärdatensätze mit einer festen Länge von 10 Bytes und dem Elementnamen "Buffer" empfangen werden. Das Server-Programm sendet einen Datensatz. Wenn EKI externe Daten empfangen hat, wird \$FLAG[1] gesetzt.

XML-Datei

```
<ETHERNETKRL>
  <CONFIGURATION>
    <EXTERNAL>
      <IP>x.x.x.x</IP>
      <PORT>59152</PORT>
    </EXTERNAL>
  </CONFIGURATION>
  <RECEIVE>
    <RAW>
      <ELEMENT Tag="Buffer" Type="BYTE" Set_Flag="1" Size="10" />
    </RAW>
  </RECEIVE>
  <SEND/>
</ETHERNETKRL>
```

Binärdatensätze fester Länge müssen im KRL-Programm mit CAST_TO() und CAST_FROM() ein- und ausgelesen werden. Es sind nur Daten vom Typ REAL (4 Bytes) lesbar, kein Double.



Detaillierte Informationen zu den Befehlen CAST_TO() und CAST_FROM() sind in der Dokumentation CREAD/CWRITE zu finden.

Programm

```

1 DEF BinaryFixed( )
2 Declaration
3 INI
4 Initialize sample data
5
6 RET=EKI_Init("BinaryFixed")
7 RET=EKI_Open("BinaryFixed")
8
9 OFFSET=0
10 CAST_TO(Bytes[],OFFSET,34.425,674345,"R",TRUE)
11
12 RET=EKI_Send("BinaryFixed",Bytes[])
13
14 WAIT FOR $FLAG[1]
15 RET=EKI_GetString("BinaryFixed","Buffer",Bytes[])
16 $FLAG[1]=FALSE
17
18 OFFSET=0
19 CAST_FROM(Bytes[], OFFSET, valueReal, valueInt, valueChar[],
    valueBool)
20
21 RET=EKI_Close("BinaryFixed")
22 RET=EKI_Clear("BinaryFixed")
23 END

```

Zeile	Beschreibung
4	Initialisieren der KRL-Variablen durch Zuweisung von Werten
6	EKI_Init() initialisiert den Kanal, über den sich das EKI mit dem externen System verbindet.
7	EKI_Open() öffnet den Kanal und verbindet sich mit dem Server.
9, 10	CAST_TO schreibt die Werte in das CHAR-Feld Bytes[].
12	EKI_Send() sendet das CHAR-Feld Bytes[] an das externe System.
14 ... 16	\$FLAG[1] signalisiert den Empfang des konfigurierten Datenelements. EKI_GetString greift auf den Speicher zu und kopiert die Daten in das CHAR-Feld Bytes[]. \$FLAG[1] wird wieder zurückgesetzt.
18, 19	CAST_FROM liest die im CHAR-Feld Bytes[] enthaltenen Werte aus und kopiert sie typgerecht in die angegebenen Variablen.
21	EKI_Close() schließt den Kanal.
22	EKI_Clear() löscht den Kanal.

7.4 Beispiel "BinaryStream"



Zur Kommunikation mit der Robotersteuerung müssen im Server-Programm die passenden Beispieldaten eingestellt sein, hier "BinaryStream".

EKI ist als Client konfiguriert. Über die Verbindung können nur Binärdatensätze mit einer Länge von maximal 64 Bytes und dem Elementnamen "Buffer" empfangen werden. Das Ende des Binärdatensatzes muss mit der Endzeichenfolge CR, LF gekennzeichnet sein. Wenn EK1 dieses Element empfangen hat, wird \$FLAG[1] gesetzt.

XML-Datei

```
<ETHERNETKRL>
  <CONFIGURATION>
    <EXTERNAL>
      <IP>x.x.x.x</IP>
      <PORT>59152</PORT>
    </EXTERNAL>
  </CONFIGURATION>
  <RECEIVE>
    <RAW>
      <ELEMENT Tag="Buffer" Type="STREAM" Set_Flag="1"
        Size="64" EOS="13,10" />
    </RAW>
  </RECEIVE>
  <SEND/>
</ETHERNETKRL>
```

Programm

```
1 DEF BinaryStream( )
2 Declaration
3 INI
4 Initialize sample data
5
6 RET=EKI_Init("BinaryStream")
7 RET=EKI_Open("BinaryStream")
8
9 Bytes[]="Stream ends with CR,LF"
10
11 RET=EKI_Send("BinaryStream",Bytes[])
12
13 WAIT FOR $FLAG[1]
14 RET=EKI_GetString("BinaryStream","Buffer",Bytes[])
15 $FLAG[1]=FALSE
16
17 RET=EKI_Close("BinaryStream")
18 RET=EKI_Clear("BinaryStream")
19
20 END
```

Zeile	Beschreibung
4	Initialisieren der KRL-Variablen durch Zuweisung von Werten
6	EKI_Init() initialisiert den Kanal, über den sich das EK1 mit dem externen System verbindet.
7	EKI_Open() öffnet den Kanal und verbindet sich mit dem Server.
9	Das CHAR-Feld Bytes[] wird mit Daten beschrieben.
11	EKI_Send() sendet das CHAR-Feld Bytes[] an das externe System.

Zeile	Beschreibung
13 ... 15	\$FLAG[1] signalisiert den Empfang des konfigurierten Datenelements. EKI_GetString liest die Zeichenfolge im CHAR-Feld Bytes[] aus dem Speicher. \$FLAG[1] wird wieder zurückgesetzt.
17	EKI_Close() schließt den Kanal.
18	EKI_Clear() löscht den Kanal.

7.5 Beispiel "XmlTransmit"



Zur Kommunikation mit der Robotersteuerung müssen im Server-Programm die passenden Beispieldaten eingestellt sein, hier "Xml".

EKI ist als Client konfiguriert. Es werden Roboterdaten gesendet und nach einer Wartezeit von 1 sec die empfangenen Sensordaten aus dem Speicher gelesen.

XML-Datei

```

<ETHERNETKRL>
  <CONFIGURATION>
    <EXTERNAL>
      <IP>x.x.x.x</IP>
      <PORT>59152</PORT>
    </EXTERNAL>
  </CONFIGURATION>
  <RECEIVE>
    <XML>
      <ELEMENT Tag="Sensor/Message" Type="STRING" />
      <ELEMENT Tag="Sensor/Positions/Current/@X" Type="REAL" />
      <ELEMENT Tag="Sensor/Positions/Before/X" Type="REAL" />
      <ELEMENT Tag="Sensor/Nmb" Type="INT" />
      <ELEMENT Tag="Sensor/Status/IsActive" Type="BOOL" />
      <ELEMENT Tag="Sensor/Read/xyzabc" Type="FRAME" />
      <ELEMENT Tag="Sensor/Show/@error" Type="BOOL" />
      <ELEMENT Tag="Sensor/Show/@temp" Type="INT" />
      <ELEMENT Tag="Sensor/Show" Type="STRING" />
      <ELEMENT Tag="Sensor/Free" Type="INT" />
    </XML>
  </RECEIVE>
  <SEND>
    <XML>
      <ELEMENT Tag="Robot/Data/LastPos/@X" />
      <ELEMENT Tag="Robot/Data/LastPos/@Y" />
      <ELEMENT Tag="Robot/Data/LastPos/@Z" />
      <ELEMENT Tag="Robot/Data/LastPos/@A" />
      <ELEMENT Tag="Robot/Data/LastPos/@B" />
      <ELEMENT Tag="Robot/Data/LastPos/@C" />
      <ELEMENT Tag="Robot/Data/ActPos/@X" />
      <ELEMENT Tag="Robot/Status" />
      <ELEMENT Tag="Robot/Mode" />
      <ELEMENT Tag="Robot/RobotLamp/GrenLamp/LightOn" />
    </XML>
  </SEND>
</ETHERNETKRL>

```

Programm

```

1 DEF XmlTransmit( )
2 Declaration

```

```

3 Communicated data
4 INI
5 Initialize sample data
6
7 RET=EKI_Init("XmlTransmit")
8 RET=EKI_Open("XmlTransmit")
9
10 Write data to connection
11 Send data to external program
12 Get received sensor data
13
14 RET=EKI_Close("XmlTransmit")
15 RET=EKI_Clear("XmlTransmit")
16
17 END

```

Zeile	Beschreibung
5	Initialisieren der KRL-Variablen durch Zuweisung von Werten
7	EKI_Init() initialisiert den Kanal, über den sich das EKI mit dem externen System verbindet.
8	EKI_Open() öffnet den Kanal und verbindet sich mit dem externen System.
10	Schreibt Daten in das gespeicherte XML-Dokument für den Datenversand.
11	Sendet das beschriebene XML-Dokument an das externe System.
12	Liest die empfangenen Sensordaten aus dem Speicher.
14	EKI_Close() schließt den Kanal.
15	EKI_Clear() löscht den Kanal.

7.6 Beispiel "XmlServer"

EKI ist als Server konfiguriert. Solange eine Verbindung zum externen System besteht, ist \$FLAG[1] gesetzt.



Wenn das EKI als Server konfiguriert ist, kann das Server-Programm auf dem externen System nicht verwendet werden. Ein einfacher Client kann mit Windows HyperTerminal realisiert werden.

XML-Datei

```

<ETHERNETKRL>
  <CONFIGURATION>
    <EXTERNAL>
      <TYPE>Client</TYPE>
    </EXTERNAL>
    <INTERNAL>
      <IP>x.x.x.x</IP>
      <PORT>54600</PORT>
      <ALIVE Set_Flag="1" />
    </INTERNAL>
  </CONFIGURATION>
  <RECEIVE>
    <XML>
      <ELEMENT Tag="Sensor/A" Type="BOOL" />
    </XML>
  </RECEIVE>
  <SEND>
    <XML>

```

```

    <ELEMENT Tag="Robot/B" />
  </XML>
</SEND>
</ETHERNETKRL>

```

Programm

```

1 DEF XmlServer( )
2 Declaration
3 INI
4
5 RET=EKI_Init("XmlServer")
6 RET=EKI_Open("XmlServer")
7
8 ; wait until server is connected
9 wait for $FLAG[1]
10 ; wait until server is disconnected
11 wait for $FLAG[1]==FALSE
12
13 RET=EKI_Clear("XmlServer")
14 END

```

Zeile	Beschreibung
5	EKI_Init() initialisiert den Kanal, über den sich das externe System mit dem EKI verbindet.
6	EKI_Open() öffnet den Kanal.
9	Wenn sich der externe Client erfolgreich mit dem Server verbunden hat, wird \$FLAG[1] gesetzt.
11	Da das EKI als Server konfiguriert ist, erwartet die Robotersteuerung, dass der Kanal vom externen Client geschlossen wird. Wenn dies der Fall ist, wird \$FLAG[1] gelöscht.
13	EKI_Clear() löscht den Kanal.

7.7 Beispiel "XmlCallback"



Zur Kommunikation mit der Robotersteuerung müssen im Server-Programm die passenden Beispieldaten eingestellt sein, hier "Xml".

EKI ist als Client konfiguriert. Es werden Roboterdaten gesendet, Sensordaten empfangen und dann auf \$FLAG[1] gewartet. \$FLAG[1] signalisiert, dass die Sensordaten ausgelesen wurden.

In der XML-Datei ist konfiguriert, dass \$FLAG[998] gesetzt wird, wenn EKI alle Sensordaten empfangen hat. Dieses Flag löst einen Interrupt im Programm aus. Durch die Konfiguration des Tags "Sensor" als Ereignis-Tag wird sichergestellt, dass die Sensordaten erst abgeholt werden, wenn alle Daten in den Speichern liegen.

Wenn die Sensordaten ausgelesen sind, wird \$FLAG[998] wieder zurückgesetzt und \$FLAG[1] gesetzt.

XML-Datei

```

<ETHERNETKRL>
  <CONFIGURATION>
    <EXTERNAL>
      <IP>x.x.x.x</IP>
      <PORT>59152</PORT>
    </EXTERNAL>
  </CONFIGURATION>
  <RECEIVE>
    <XML>

```

```

<ELEMENT Tag="Sensor/Message" Type="STRING" />
<ELEMENT Tag="Sensor/Positions/Current/@X" Type="REAL" />
<ELEMENT Tag="Sensor/Positions/Before/X" Type="REAL" />
<ELEMENT Tag="Sensor/Nmb" Type="INT" />
<ELEMENT Tag="Sensor/Status/IsActive" Type="BOOL" />
<ELEMENT Tag="Sensor/Read/xyzabc" Type="FRAME" />
<ELEMENT Tag="Sensor/Show/@error" Type="BOOL" />
<ELEMENT Tag="Sensor/Show/@temp" Type="INT" />
<ELEMENT Tag="Sensor/Show" Type="STRING" />
<ELEMENT Tag="Sensor/Free" Type="INT" Set_Out="998" />
<ELEMENT Tag="Sensor" Set_Flag="998" />

</XML>
</RECEIVE>
<SEND>
  <XML>
    <ELEMENT Tag="Robot/Data/LastPos/@X" />
    <ELEMENT Tag="Robot/Data/LastPos/@Y" />
    <ELEMENT Tag="Robot/Data/LastPos/@Z" />
    <ELEMENT Tag="Robot/Data/LastPos/@A" />
    <ELEMENT Tag="Robot/Data/LastPos/@B" />
    <ELEMENT Tag="Robot/Data/LastPos/@C" />
    <ELEMENT Tag="Robot/Data/ActPos/@X" />
    <ELEMENT Tag="Robot/Status" />
    <ELEMENT Tag="Robot/Mode" />
    <ELEMENT Tag="Robot/RobotLamp/GrenLamp/LightOn" />
  </XML>
</SEND>
</ETHERNETKRL>

```

Programm

```

1 DEF XmlCallBack( )
2 Declaration
3 Communicated data
4 INI
5 Define callback
6
7 RET=EKI_Init("XmlCallBack")
8 RET=EKI_Open("XmlCallBack")
9
10 Write data to connection
11 RET=EKI_Send("XmlCallBack","Robot")
12
13 ;wait until data read
14 WAIT FOR $FLAG[1]
15
16 RET=EKI_Close("XmlCallBack")
17 RET=EKI_Clear("XmlCallBack")
18 END
19
20 DEF GET_DATA()
21 Declaration
22 Initialize sample data
23 Get received sensor data
24 Signal read

```

Zeile	Beschreibung
5	Deklaration und Einschalten des Interrupts
7	EKI_Init() initialisiert den Kanal, über den sich das EKI mit dem externen System verbindet.
8	EKI_Open() öffnet den Kanal.

Zeile	Beschreibung
10	Schreibt Daten in das gespeicherte XML-Dokument für den Datenversand.
11	Sendet die Daten.
14	Wartet auf \$FLAG[1]. Das Ereignis-Flag meldet, dass alle Daten gelesen wurden.
16	EKI_Close() schließt den Kanal.
17	EKI_Clear() löscht den Kanal.
20 ... 24	Initialisieren der KRL-Variablen durch Zuweisung von Werten und Auslesen der Daten Wenn alle Daten gelesen sind, wird \$FLAG[1] gesetzt.

Datenversand

Das XML-Dokument wird vom KRL-Programm mit Roboterdaten beschrieben und über das EKI an das externe System gesendet.

```
<Robot>
  <Data>
    <LastPos X="..." Y="..." Z="..." A="..." B="..." C="...">
    </LastPos>
    <ActPos X="1000.12">
    </ActPos>
  </Data>
  <Status>12345678</Status>
  <Mode>ConnectSensor</Mode>
  <RobotLamp>
    <GrenLamp>
      <LightOn>1</LightOn>
    </GrenLamp>
  </RobotLamp>
</Robot>
```

Datenempfang

Das XML-Dokument wird vom Server-Programm mit Sensordaten beschrieben und vom EKI empfangen.

```
<Sensor>
  <Message>Example message</Message>
  <Positions>
    <Current X="4645.2" />
    <Before>
      <X>0.9842</X>
    </Before>
  </Positions>
  <Nmb>8</Nmb>
  <Status>
    <IsActive>1</IsActive>
  </Status>
  <Read>
    <xyzabc X="210.3" Y="825.3" Z="234.3" A="84.2" B="12.3"
      C="43.5" />
  </Read>
  <Show error="0" temp="9929">Taginfo in attributes</Show>
  <Free>2912</Free>
</Sensor>
```


8 Diagnose

8.1 Diagnosedaten anzeigen

Voraussetzung ■ Benutzerrechte: Funktionsgruppe **Diagnose-Funktionen**

Vorgehensweise

1. Im Hauptmenü **Diagnose** > **Diagnosemonitor** wählen.
2. Im Feld **Modul** das Modul **EKI (EthernetKRL)** auswählen.

Beschreibung

Name	Beschreibung
Gesamtspeicher	Insgesamt verfügbarer Speicher (Bytes)
Verbrauchter Speicher	Benutzer Speicher (Bytes)
Verbindungen Roboterprogramm	Anzahl der vom Roboter-Interpreter initialisierten Verbindungen
Verbindungen Submit-Programm	Anzahl der vom Submit-Interpreter initialisierten Verbindungen
Verbindungen System	Anzahl der vom System initialisierten Verbindungen
Ethernet-Verbindungen	Anzahl offener Verbindungen
Verarbeitungszeit	Maximale Zeit, die benötigt wird, um empfangene Daten zu bearbeiten (Aktualisierung alle 5 sec)
Warnmeldungen	Anzahl der Warnmeldungen
Fehlermeldungen	Anzahl der Fehlermeldungen



Die Warn- und Fehlermeldungen werden auch dann gezählt, wenn die automatische Meldungsabgabe und das Loggen von Meldungen deaktiviert wurden.

9 Meldungen

9.1 Fehlerprotokoll (EKI-Logbuch)

Alle Fehlermeldungen des EKI werden in einer LOG-Datei unter C:\KRC\ROBOTER\LOG\EthernetKRL protokolliert.

9.2 Informationen zu den Meldungen

Wenn beim Aufruf einer Funktion oder beim Datenempfang ein Fehler aufgetreten ist, gibt KUKA.Ethernet KRL die Fehlernummer zurück. Den Fehlernummern ist ein Meldungstext zugeordnet, der auf der smartHMI angezeigt wird. Ist die automatische Ausgabe von Meldungen deaktiviert, kann über EKI_CHECK() die Meldung weiterhin auf der smartHMI angezeigt werden.

Das Kapitel "Meldungen" enthält ausgewählte Meldungen. Es behandelt nicht alle Meldungen, die im Meldungsfenster angezeigt werden.

9.3 Systemmeldungen aus Modul: EthernetKRL (EKI)

9.3.1 EKI00002

Meldungscode	■ EKI00002
Meldungstext	■ Der Systemspeicher ist verbraucht
Meldungstyp	■ Fehlermeldung
	
Auswirkung	■ Bahntreuer NOT-HALT ■ Die Eingabe von aktiven Kommandos (Roboterbewegungen, Programmstart) ist blockiert.
Mögliche Ursache(n)	■ Ursache: Reservierter Speicher nicht ausreichend (>>> Seite 61) Lösung: Speicher erhöhen (>>> Seite 62) ■ Ursache: Verbindungskonfiguration verbraucht zu viel Speicher (>>> Seite 62) Lösung: Verbindungskonfiguration anpassen, um weniger Speicher zu verbrauchen (>>> Seite 62) ■ Ursache: Mehrere Verbindungen mit hohem Datenaufkommen aktiv (>>> Seite 62) Lösung: Programmierung anpassen, um weniger Speicher zu verbrauchen (>>> Seite 63)

Ursache: Reservierter Speicher nicht ausreichend

Beschreibung Der für Ethernet KRL reservierte Speicher ist nicht ausreichend.

Prüfanweisung ■ Prüfen, ob der Speicherbedarf durch eine andere Konfiguration oder Programmierung reduziert werden kann.

Wenn dies nicht möglich ist, darf der Speicher nach Rücksprache mit der KUKA Roboter GmbH erhöht werden.

Lösung: Speicher erhöhen

Der Speicher darf nur nach Rücksprache mit der KUKA Roboter GmbH erhöht werden. (>>> 11 "KUKA Service" Seite 113)

Voraussetzung

- Benutzergruppe Experte
- Betriebsart T1, T2 oder AUT

Vorgehensweise

1. Im Verzeichnis C:\KRC\ROBOTER\Config\User\Common die Datei EthernetKRL.XML öffnen.
2. Im Abschnitt <EthernetKRL> im Element <MemSize> die gewünschte Speicherkapazität in Byte eintragen.

```
<EthernetKRL>
  <Interface>
    <MemSize>1048576</MemSize>
  ...
</EthernetKRL>
```

3. Datei schließen und die Sicherheitsabfrage, ob die Änderungen gespeichert werden sollen, mit **Ja** beantworten.
4. Die Robotersteuerung neu starten, mit den Einstellungen **Kaltstart** und **Dateien neu einlesen**.

Ursache: Verbindungskonfiguration verbraucht zu viel Speicher**Beschreibung**

Die konfigurierten Ethernet-Verbindungen verbrauchen zu viel Speicher.

Konfigurations-datei

Für jede Ethernet-Verbindung muss eine XML-Datei konfiguriert sein. Der Name der XML-Datei ist gleichzeitig der Zugriffsschlüssel in KRL.

Verzeichnis	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Datei	Beispiel: ... \EXT.XML → EKI_INIT("EXT")

Prüfanweisung

- XML-Datei/en daraufhin überprüfen, ob die Ethernet-Verbindungen so konfiguriert werden können, dass weniger Speicher verbraucht wird.

Lösung: Verbindungskonfiguration anpassen, um weniger Speicher zu verbrauchen**Voraussetzung**

- Benutzergruppe Experte
- Betriebsart T1, T2 oder AUT

Vorgehensweise

1. XML-Datei wie benötigt ändern.
2. Wenn XML-Datei offline geändert wurde, sie in das vorgesehene Verzeichnis kopieren und die alte XML-Datei überschreiben.
3. Die Robotersteuerung neu starten, mit den Einstellungen **Kaltstart** und **Dateien neu einlesen**.

Ursache: Mehrere Verbindungen mit hohem Datenaufkommen aktiv**Beschreibung**

Wenn mehrere Verbindungen mit hohem Datenaufkommen gleichzeitig aktiv sind, kann dies dazu führen, dass zu viel Speicher verbraucht wird.

Prüfanweisung

- Programmierung daraufhin überprüfen, ob sie so geändert werden kann, dass weniger Speicher verbraucht wird.

Lösung: Programmierung anpassen, um weniger Speicher zu verbrauchen

Beschreibung Die Programmierung muss so angepasst werden, dass weniger Speicher verbraucht wird.

9.3.2 EKI00003

Meldungscode	■ EKI00003
Meldungstext	■ Zugriff auf Datei fehlgeschlagen
Meldungstyp	■ Fehlermeldung
	
Auswirkung	■ Bahntreuer NOT-HALT ■ Die Eingabe von aktiven Kommandos (Roboterbewegungen, Programmstart) ist blockiert.
Mögliche Ursache(n)	■ Ursache: XML-Datei mit der Verbindungskonfiguration nicht vorhanden (>>> Seite 63) Lösung: XML-Datei mit der Verbindungskonfiguration wiederherstellen (>>> Seite 63) ■ Ursache: XML-Datei mit der Verbindungskonfiguration nicht lesbar (>>> Seite 64) Lösung: XML-Datei mit der Verbindungskonfiguration wiederherstellen (>>> Seite 64)

Ursache: XML-Datei mit der Verbindungskonfiguration nicht vorhanden

Beschreibung Die Ethernet-Verbindung wurde nicht initialisiert, da unter dem in der Funktion EKI_Init() angegebenen Namen keine XML-Datei gespeichert ist.

Konfigurationsdatei Für jede Ethernet-Verbindung muss eine XML-Datei konfiguriert sein. Der Name der XML-Datei ist gleichzeitig der Zugriffsschlüssel in KRL.

Verzeichnis	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Datei	Beispiel: ... \EXT.XML —> EKI_INIT("EXT")

So kann man prüfen, ob die XML-Datei vorhanden ist:

Voraussetzung

- Benutzergruppe Experte
- Programm ist angewählt oder geöffnet.

Prüfanweisung

1. Dateiname notieren, der in der Funktion EKI_Init() verwendet wird.

```
RET = EKI_Init("Dateiname")
```

 - *Dateiname:* Name der XML-Datei mit der Verbindungskonfiguration
2. Verzeichnis, in dem die XML-Dateien abgelegt sind, öffnen und prüfen, ob eine XML-Datei mit dem in der Funktion EKI_Init() angegebenen Namen vorhanden ist.

Lösung: XML-Datei mit der Verbindungskonfiguration wiederherstellen

Beschreibung Die XML-Datei mit der Verbindungskonfiguration muss wiederhergestellt und in das Verzeichnis C:\KRC\ROBOTER\Config\User\Common\EthernetKRL kopiert werden.

- Voraussetzung**
- Benutzergruppe Experte
 - Betriebsart T1, T2 oder AUT
- Vorgehensweise**
- XML-Datei mit der Verbindungskonfiguration in das vorgesehene Verzeichnis kopieren.
 - Die Robotersteuerung neu starten, mit den Einstellungen **Kaltstart** und **Dateien neu einlesen**.

Ursache: XML-Datei mit der Verbindungskonfiguration nicht lesbar

Beschreibung Die Ethernet-Verbindung wurde nicht initialisiert, da die in der Funktion EKI_Init() genannte XML-Datei nicht lesbar ist. Die XML-Datei ist beschädigt und lässt sich nicht öffnen.

Konfigurations-datei Für jede Ethernet-Verbindung muss eine XML-Datei konfiguriert sein. Der Name der XML-Datei ist gleichzeitig der Zugriffsschlüssel in KRL.

Verzeichnis	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Datei	Beispiel: ... \EXT.XML → EKI_INIT("EXT")

So kann man prüfen, ob die XML-Datei beschädigt ist:

- Voraussetzung**
- Benutzergruppe Experte
 - Programm ist angewählt oder geöffnet.
- Prüfanweisung**
1. Dateiname notieren, der in der Funktion EKI_Init() verwendet wird.

```
RET = EKI_Init("Dateiname")
```

 - *Dateiname*: Name der XML-Datei mit der Verbindungskonfiguration
 2. Verzeichnis, in dem die XML-Dateien abgelegt sind, öffnen und prüfen, ob sich die XML-Datei öffnen lässt.

Lösung: XML-Datei mit der Verbindungskonfiguration wiederherstellen

Beschreibung Die XML-Datei mit der Verbindungskonfiguration muss wiederhergestellt und in das Verzeichnis C:\KRC\ROBOTER\Config\User\Common\EthernetKRL kopiert werden.

- Voraussetzung**
- Benutzergruppe Experte
 - Betriebsart T1, T2 oder AUT
- Vorgehensweise**
- XML-Datei mit der Verbindungskonfiguration in das vorgesehene Verzeichnis kopieren.
 - Die Robotersteuerung neu starten, mit den Einstellungen **Kaltstart** und **Dateien neu einlesen**.

9.3.3 EKI00006

Meldungscode	■ EKI00006
Meldungstext	■ Interpretieren der Konfiguration fehlgeschlagen
Meldungstyp	■ Fehlermeldung
	
Auswirkung	■ Bahntreuer NOT-HALT ■ Die Eingabe von aktiven Kommandos (Roboterbewegungen, Programmstart) ist blockiert.

Mögliche Ursache(n)

- **Ursache:** Schemafehler in XML-Datei mit der Verbindungskonfiguration (>>> Seite 65)
- Lösung:** Fehler in XML-Datei beheben (>>> Seite 65)

Ursache: Schemafehler in XML-Datei mit der Verbindungskonfiguration

Beschreibung

Die XML-Datei mit der Verbindungskonfiguration kann wegen eines Schemafehlers nicht gelesen werden.

KUKA.Ethernet KRL verwendet das XPath-Schema. Die Syntax, die das Schema vorgibt, muss genau eingehalten werden. Z. B. dürfen keine Satzzeichen oder Strukturelemente fehlen.

Die Schreibweise der Elemente und Attribute in der XML-Datei, einschließlich Groß- und/oder Kleinschreibung, ist vorgegeben und muss genau eingehalten werden.



Weitere Informationen zum XPath-Schema sind in der Dokumentation zu KUKA.Ethernet KRL zu finden.

Konfigurationsdatei

Für jede Ethernet-Verbindung muss eine XML-Datei konfiguriert sein. Der Name der XML-Datei ist gleichzeitig der Zugriffsschlüssel in KRL.

Verzeichnis	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Datei	Beispiel: ...\EXT.XML → EKI_INIT("EXT")

Lösung: Fehler in XML-Datei beheben

Beschreibung

Die Fehler in der XML-Datei müssen behoben werden.

Voraussetzung

- Benutzergruppe Experte
- Betriebsart T1, T2 oder AUT

Vorgehensweise

1. XML-Datei wie benötigt ändern.
2. Wenn XML-Datei offline geändert wurde, sie in das vorgesehene Verzeichnis kopieren und die alte XML-Datei überschreiben.
3. Die Robotersteuerung neu starten, mit den Einstellungen **Kaltstart** und **Dateien neu einlesen**.

9.3.4 EKI00007

Meldungscode	■ EKI00007
Meldungstext	■ Schreiben der Daten zum Senden fehlgeschlagen
Meldungstyp	■ Fehlermeldung
	
Auswirkung	■ Bahntreuer NOT-HALT ■ Die Eingabe von aktiven Kommandos (Roboterbewegungen, Programmstart) ist blockiert.

Mögliche Ursache(n)

- **Ursache:** XML-Struktur für Datenversand falsch aufgebaut (>>> Seite 66)
Lösung: XML-Struktur entsprechend den zu sendenden XML-Dokumenten aufbauen (>>> Seite 66)

Ursache: XML-Struktur für Datenversand falsch aufgebaut

Beschreibung

Die XML-Struktur für den Datenversand kann nicht beschrieben werden, da die XML-Struktur nach einem anderen Schema aufgebaut ist als das zu sendende XML-Dokument. KUKA.Ethernet KRL verwendet das XPath-Schema.



Weitere Informationen zum XPath-Schema sind in der Dokumentation zu KUKA.Ethernet KRL zu finden.

Lösung: XML-Struktur entsprechend den zu sendenden XML-Dokumenten aufbauen

Beschreibung

Die XML-Struktur für den Datenversand ist entsprechend den zu sendenden XML-Dokumenten nach XPath-Schema aufzubauen.

9.3.5 EKI00009

Meldungscode

- EKI00009

Meldungstext

- Verbindung nicht vorhanden

Meldungstyp

- Fehlermeldung



Auswirkung

- Bahntreuer NOT-HALT
- Die Eingabe von aktiven Kommandos (Roboterbewegungen, Programmstart) ist blockiert.

Mögliche Ursache(n)

- **Ursache:** EKI_Init() nicht oder falsch programmiert (>>> Seite 66)
Lösung: Funktion richtig programmieren (>>> Seite 67)

Ursache: EKI_Init() nicht oder falsch programmiert

Beschreibung

Die Ethernet-Verbindung wurde nicht initialisiert, da die Funktion EKI_Init() nicht oder falsch programmiert ist.

Eine Verbindung muss immer mit der Funktion EKI_Init() angelegt und initialisiert werden. Die in der Funktion angegebene XML-Datei mit der Verbindungskonfiguration wird dabei eingelesen.

RET = EKI_Init(CHAR[])	
Funktion	Initialisiert einen Kanal für die Ethernet-Kommunikation Folgende Aktionen werden ausgeführt: ■ Einlesen der Verbindungskonfiguration ■ Erstellen der Datenspeicher ■ Vorbereiten der Ethernet-Verbindung
Parameter	Typ: CHAR Name des Kanals (= Name der XML-Datei mit der Verbindungskonfiguration)
RET	Typ: EKI_STATUS Rückgabewerte der Funktion
Beispiel	RET = EKI_Init("Channel_1")

Konfigurationsdatei

Für jede Ethernet-Verbindung muss eine XML-Datei konfiguriert sein. Der Name der XML-Datei ist gleichzeitig der Zugriffsschlüssel in KRL.

Verzeichnis	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Datei	Beispiel: ... \EXT.XML → EKI_INIT("EXT")

So kann man prüfen, ob die Funktion korrekt programmiert ist:

Voraussetzung

- Benutzergruppe Experte
- Programm ist angewählt oder geöffnet.

Prüfanweisung

1. Prüfen, ob folgende Zeile programmiert ist:

```
RET = EKI_Init("Dateiname")
```

 - *Dateiname*: Name der XML-Datei mit der Verbindungskonfiguration
2. Prüfen, ob der in der Funktion EKI_Init() angegebene Dateiname mit dem Namen der XML-Datei mit der Verbindungskonfiguration übereinstimmt.

Lösung: Funktion richtig programmieren

Beschreibung

Die Funktion muss richtig programmiert werden.

Voraussetzung

- Benutzergruppe Experte
- Betriebsart T1, T2 oder AUT

Vorgehensweise

1. Programm im Navigator markieren und **Öffnen** drücken. Das Programm wird im Editor angezeigt.
2. Die entsprechende Stelle im Programm suchen und bearbeiten.
3. Datei schließen und die Sicherheitsabfrage, ob die Änderungen gespeichert werden sollen, mit **Ja** beantworten.

9.3.6 EKI00010

Meldungscode	■ EKI00010
Meldungstext	■ Ethernet ist getrennt
Meldungstyp	■ Fehlermeldung
	

Auswirkung	■ Bahntreuer NOT-HALT ■ Die Eingabe von aktiven Kommandos (Roboterbewegungen, Programmstart) ist blockiert.
Mögliche Ursache(n)	■ Ursache: EKI_Open() nicht oder falsch programmiert (>>> Seite 68) ■ Lösung: Funktion richtig programmieren (>>> Seite 68)

Ursache: EKI_Open() nicht oder falsch programmiert

Beschreibung Die Ethernet-Verbindung ist initialisiert aber noch nicht geöffnet, da die Funktion EKI_Open() nicht oder falsch programmiert ist.

RET = EKI_Open(CHAR[])	
Funktion	Öffnet einen initialisierten Kanal Wenn EKI als Client konfiguriert ist, verbindet sich das EKI mit dem externen System (= Server). Wenn EKI als Server konfiguriert ist, wartet das EKI auf die Verbindungsanfrage des externen Systems (= Client).
Parameter	Typ: CHAR Name des Kanals
RET	Typ: EKI_STATUS Name des Kanals (= Name der XML-Datei mit der Verbindungskonfiguration)
Beispiel	RET = EKI_Open("Channel_1")

Konfigurations-datei Für jede Ethernet-Verbindung muss eine XML-Datei konfiguriert sein. Der Name der XML-Datei ist gleichzeitig der Zugriffsschlüssel in KRL.

Verzeichnis	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Datei	Beispiel: ... \EXT.XML → EKI_INIT("EXT")

So kann man prüfen, ob die Funktion korrekt programmiert ist:

Voraussetzung

- Benutzergruppe Experte
- Programm ist angewählt oder geöffnet.

Prüfanweisung

1. Prüfen, ob folgende Zeile programmiert ist:

```
RET = EKI_Open ("Dateiname")
```

 - *Dateiname:* Name der XML-Datei mit der Verbindungskonfiguration
2. Prüfen, ob der in der Funktion EKI_Open() angegebene Dateiname mit dem in der Funktion EKI_Init() verwendeten Dateinamen übereinstimmt.

Lösung: Funktion richtig programmieren

Beschreibung Die Funktion muss richtig programmiert werden.

Voraussetzung

- Benutzergruppe Experte
- Betriebsart T1, T2 oder AUT

Vorgehensweise

1. Programm im Navigator markieren und **Öffnen** drücken. Das Programm wird im Editor angezeigt.
2. Die entsprechende Stelle im Programm suchen und bearbeiten.

3. Datei schließen und die Sicherheitsabfrage, ob die Änderungen gespeichert werden sollen, mit **Ja** beantworten.

9.3.7 EKI00011

Meldungscode	■ EKI00011
Meldungstext	■ Ethernetverbindung zu externem System bereits vorhanden
Meldungstyp	■ Fehlermeldung
	
Auswirkung	■ Bahntreuer NOT-HALT ■ Die Eingabe von aktiven Kommandos (Roboterbewegungen, Programmstart) ist blockiert.
Mögliche Ursache(n)	■ Ursache: Ethernet-Verbindung bereits mit EKI_Open() geöffnet (>>> Seite 69) ■ Lösung: Zu viel programmierte Funktion löschen (>>> Seite 70)

Ursache: Ethernet-Verbindung bereits mit EKI_Open() geöffnet

Beschreibung Die Ethernet-Verbindung ist bereits mit der Funktion EKI_Open() geöffnet worden.

RET = EKI_Open(CHAR[])	
Funktion	Öffnet einen initialisierten Kanal Wenn EKI als Client konfiguriert ist, verbindet sich das EKI mit dem externen System (= Server). Wenn EKI als Server konfiguriert ist, wartet das EKI auf die Verbindungsanfrage des externen Systems (= Client).
Parameter	Typ: CHAR Name des Kanals
RET	Typ: EKI_STATUS Name des Kanals (= Name der XML-Datei mit der Verbindungskonfiguration)
Beispiel	RET = EKI_Open("Channel_1")

Konfigurations-datei Für jede Ethernet-Verbindung muss eine XML-Datei konfiguriert sein. Der Name der XML-Datei ist gleichzeitig der Zugriffsschlüssel in KRL.

Verzeichnis	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Datei	Beispiel: ... \EXT.XML → EKI_INIT("EXT")

So kann man prüfen, ob die Verbindung bereits geöffnet ist:

Voraussetzung

- Benutzergruppe Experte
- Programm ist angewählt oder geöffnet.

Prüfanweisung

- Prüfen, ob folgende Zeile zwischen dem Initialisieren und dem Schließen der Verbindung mehrmals programmiert ist:

```
RET = EKI_Open ("Dateiname")
```

- *Dateiname:* Name der XML-Datei mit der Verbindungskonfiguration

Lösung: Zu viel programmierte Funktion löschen

Beschreibung	Zu viel programmierte Funktion muss gelöscht werden.
Voraussetzung	- Benutzergruppe Experte - Betriebsart T1, T2 oder AUT
Vorgehensweise	1. Programm im Navigator markieren und Öffnen drücken. Das Programm wird im Editor angezeigt. 2. Die entsprechende Stelle im Programm suchen und bearbeiten. 3. Datei schließen und die Sicherheitsabfrage, ob die Änderungen gespeichert werden sollen, mit Ja beantworten.

9.3.8 EKI00012

Meldungscode	■ EKI00012
Meldungstext	■ Erstellen des Servers fehlgeschlagen
Meldungstyp	■ Fehlermeldung
	
Auswirkung	■ Bahntreuer NOT-HALT ■ Die Eingabe von aktiven Kommandos (Roboterbewegungen, Programmstart) ist blockiert.
Mögliche Ursache(n)	■ Ursache: IP-Adresse und/oder Port-Nummer für EKI nicht oder falsch angegeben (>>> Seite 70) Lösung: IP-Adresse und Port-Nummer richtig in XML-Datei eintragen (>>> Seite 71)

Ursache: IP-Adresse und/oder Port-Nummer für EKI nicht oder falsch angegeben

Beschreibung	EKI ist als Server und das externe System als Client konfiguriert. In der XML-Datei mit der Verbindungskonfiguration sind die IP-Adresse und/oder Port-Nummer für das EKI nicht oder formal falsch angegeben. Um die Netzwerksicherheit zu gewährleisten, kann festgelegt werden, von welchem Subnetz auf EKI zugegriffen werden kann.
---------------------	--

Beispiele	Verbindung nur über das Subnetz 192.168.X.X auf den Port 54600 möglich:
------------------	---

```
<INTERNAL>
<IP>192.168.23.1</IP>
<PORT>54600</PORT>
</INTERNAL>
```

Verbindung über alle im KLI konfigurierten Subnetze auf den Port 54600 möglich:

```
<INTERNAL>
<IP>0.0.0.0</IP>
<PORT>54600</PORT>
</INTERNAL>
```

Konfigurations-datei Für jede Ethernet-Verbindung muss eine XML-Datei konfiguriert sein. Der Name der XML-Datei ist gleichzeitig der Zugriffsschlüssel in KRL.

Verzeichnis	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Datei	Beispiel: ... \EXT.XML → EKI_INIT("EXT")

Prüfanweisung ■ In der XML-Datei mit der Verbindungskonfiguration prüfen, ob IP-Adresse und Port-Nummer richtig eingetragen sind.

Lösung: IP-Adresse und Port-Nummer richtig in XML-Datei eintragen

Voraussetzung ■ Benutzergruppe Experte
 ■ Betriebsart T1, T2 oder AUT

Vorgehensweise

1. XML-Datei wie benötigt ändern.
2. Wenn XML-Datei offline geändert wurde, sie in das vorgesehene Verzeichnis kopieren und die alte XML-Datei überschreiben.
3. Die Robotersteuerung neu starten, mit den Einstellungen **Kaltstart** und **Dateien neu einlesen**.

9.3.9 EKI00013

Meldungscode	■ EKI00013
Meldungstext	■ Ethernetparameter konnten nicht initialisiert werden
Meldungstyp	■ Fehlermeldung
	
Auswirkung	■ Bahntreuer NOT-HALT ■ Die Eingabe von aktiven Kommandos (Roboterbewegungen, Programmstart) ist blockiert.
Mögliche Ursache(n)	■ Ursache: IP-Adresse und/oder Port-Nummer für externes System nicht oder falsch angegeben (>>> Seite 71) Lösung: IP-Adresse und Port-Nummer richtig in XML-Datei eintragen (>>> Seite 72)

Ursache: IP-Adresse und/oder Port-Nummer für externes System nicht oder falsch angegeben

Beschreibung EKI ist als Client und das externe System als Server konfiguriert. In der XML-Datei mit der Verbindungskonfiguration sind die IP-Adresse und/oder Port-Nummer für das externe System nicht oder formal falsch angegeben.

IP-Adresse und Port-Nummer für das externe System sind im Abschnitt <EXTERNAL> ... </EXTERNAL> der XML-Datei wie folgt anzugeben:

- <IP>IP-Adresse</IP>
- <PORT>Port-Nummer</PORT>

Konfigurations-datei Für jede Ethernet-Verbindung muss eine XML-Datei konfiguriert sein. Der Name der XML-Datei ist gleichzeitig der Zugriffsschlüssel in KRL.

Verzeichnis	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Datei	Beispiel: ... \EXT.XML → EKI_INIT("EXT")

- Prüfanweisung**
- In der XML-Datei mit der Verbindungskonfiguration prüfen, ob IP-Adresse und Port-Nummer richtig eingetragen sind.

Lösung: IP-Adresse und Port-Nummer richtig in XML-Datei eintragen

- Voraussetzung**
- Benutzergruppe Experte
 - Betriebsart T1, T2 oder AUT

- Vorgehensweise**
1. XML-Datei wie benötigt ändern.
 2. Wenn XML-Datei offline geändert wurde, sie in das vorgesehene Verzeichnis kopieren und die alte XML-Datei überschreiben.
 3. Die Robotersteuerung neu starten, mit den Einstellungen **Kaltstart** und **Dateien neu einlesen**.

9.3.10 EKI00014

Meldungscode	■ EKI00014
Meldungstext	■ Ethernetverbindung zu externem System konnte nicht hergestellt werden
Meldungstyp	■ Fehlermeldung 
Auswirkung	■ Bahntreuer NOT-HALT ■ Die Eingabe von aktiven Kommandos (Roboterbewegungen, Programmstart) ist blockiert.
Mögliche Ursache(n)	■ Ursache: Falsche IP-Adresse und/oder Port-Nummer angegeben (>>> Seite 72) Lösung: IP-Adresse und Port-Nummer richtig in XML-Datei eintragen (>>> Seite 73) ■ Ursache: Netzkabel defekt oder nicht richtig angeschlossen (>>> Seite 73) Lösung: Netzkabel tauschen oder richtig einstecken (>>> Seite 73) ■ Ursache: Keine Ethernet-Verbindung wegen Software-Fehler, externes System (>>> Seite 74) Lösung: Fehler in Software des externen Systems beheben (>>> Seite 74) ■ Ursache: Keine Ethernet-Verbindung wegen Hardware-Fehler, externes System (>>> Seite 74) Lösung: Hardware-Fehler am externen System beheben (>>> Seite 74)

Ursache: Falsche IP-Adresse und/oder Port-Nummer angegeben

- Beschreibung**
- In der XML-Datei mit der Verbindungskonfiguration sind die falsche IP-Adresse und/oder Port-Nummer angegeben. Sie passen nicht zum externen System oder zum EKI.

Abhängig davon, ob das externe System als Server oder Client konfiguriert ist, sind IP-Adresse und Port-Nummer wie folgt anzugeben:

- Abschnitt <EXTERNAL> ... </EXTERNAL>
 - <TYPE>Server</TYPE> oder <TYPE> nicht angegeben: Externes System ist als Server konfiguriert.
 - <TYPE>Client</TYPE>: Externes System ist als Client konfiguriert.
 - Wenn TYPE = Server, müssen hier IP-Adresse und Port-Nummer des externen Systems eingetragen werden:
 - <IP>IP-Adresse</IP>
 - <PORT>Port-Nummer</PORT>

Wenn TYPE = Client, werden hier angegebene Verbindungsparameter ignoriert.
- Abschnitt <INTERNAL> ... </INTERNAL>
 - Wenn TYPE = Client, müssen hier IP-Adresse und Port-Nummer des EKI eingetragen werden:
 - <IP>IP-Adresse</IP>
 - <PORT>Port-Nummer</PORT>

Wenn TYPE = Server, werden hier angegebene Verbindungsparameter ignoriert.

Konfigurations-datei

Für jede Ethernet-Verbindung muss eine XML-Datei konfiguriert sein. Der Name der XML-Datei ist gleichzeitig der Zugriffsschlüssel in KRL.

Verzeichnis	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Datei	Beispiel: ... \EXT.XML → EKI_INIT("EXT")

Prüfanweisung

- In der XML-Datei mit der Verbindungskonfiguration prüfen, ob IP-Adresse und Port-Nummer richtig eingetragen sind.

Lösung: IP-Adresse und Port-Nummer richtig in XML-Datei eintragen

Voraussetzung

- Benutzergruppe Experte
- Betriebsart T1, T2 oder AUT

Vorgehensweise

1. XML-Datei wie benötigt ändern.
2. Wenn XML-Datei offline geändert wurde, sie in das vorgesehene Verzeichnis kopieren und die alte XML-Datei überschreiben.
3. Die Robotersteuerung neu starten, mit den Einstellungen **Kaltstart** und **Dateien neu einlesen**.

Ursache: Netzkabel defekt oder nicht richtig angeschlossen

Beschreibung

Das Netzkabel ist defekt oder die Steckverbindung ist fehlerhaft.

So kann man prüfen, ob Netzkabel und Verbindungsstecker korrekt angeschlossen sind:

Prüfanweisung

1. Die Netzkabel auf korrekte Steckverbindung und festen Sitz prüfen.
2. Die Netzkabel untereinander quer tauschen.

Lösung: Netzkabel tauschen oder richtig einstecken

Vorgehensweise

- Netzkabel tauschen oder richtig einstecken.

Ursache: Keine Ethernet-Verbindung wegen Software-Fehler, externes System

Beschreibung Wegen eines Fehlers in der Software des externen Systems liegt keine Ethernet-Verbindung vor.

Prüfanweisung ■ Externes System auf Software-Fehler überprüfen.

Lösung: Fehler in Software des externen Systems beheben

Beschreibung Der Fehler in der Software des externen Systems muss behoben werden.

Ursache: Keine Ethernet-Verbindung wegen Hardware-Fehler, externes System

Beschreibung Wegen eines Hardware-Fehlers am externen System, z. B. durch Wackelkontakt an Buchse, defekte Netzwerkkarte, etc., liegt keine Ethernet-Verbindung vor.

Prüfanweisung ■ Externes System auf Hardware-Fehler überprüfen.

Lösung: Hardware-Fehler am externen System beheben

Beschreibung Der Hardware-Fehler am externen System muss behoben werden.

9.3.11 EKI00015

Meldungscode	■ EKI00015
Meldungstext	■ Zugriff auf leeren Empfangsspeicher
Meldungstyp	■ Fehlermeldung
	
Auswirkung	■ Bahntreuer NOT-HALT ■ Die Eingabe von aktiven Kommandos (Roboterbewegungen, Programmstart) ist blockiert.
Mögliche Ursache(n)	■ Ursache: Leerer Datenspeicher bei Zugriff mit EKI_Get...() (>>> Seite 74) Lösung: Programm ändern (>>> Seite 75)

Ursache: Leerer Datenspeicher bei Zugriff mit EKI_Get...()

Beschreibung Mit einer EKI_Get...()-Funktion wurde auf einen leeren Datenspeicher zugegriffen.

Der Zugriff auf einen leeren Speicher kann verhindert werden, indem man den entsprechenden Rückgabewert der Zugriffsfunktion abfragt und auswertet.

EKI_STATUS ist die globale Strukturvariable, in die die Rückgabewerte der Funktion geschrieben werden. Relevant für die Auswertung ist das Element `Buff` von EKI_STATUS.

`Buff` enthält folgenden Wert:

- Anzahl der Elemente, die sich nach einem Zugriff noch im Speicher befinden

Syntax `GLOBAL STRUC EKI_STATUS INT Buff, Read, Msg_No, BOOL Connected, INT Counter`

Beispiel

```

...
REPEAT
    ret = EKI_GetInt("MyChannel", "Root/Number", value)
    DoSomething(value)
UNTIL (ret.Buff < 1)
...

```

Lösung: Programm ändern**Voraussetzung**

- Benutzergruppe Experte
- Betriebsart T1, T2 oder AUT

Vorgehensweise

1. Programm im Navigator markieren und **Öffnen** drücken. Das Programm wird im Editor angezeigt.
2. Die entsprechende Stelle im Programm suchen und bearbeiten.
3. Datei schließen und die Sicherheitsabfrage, ob die Änderungen gespeichert werden sollen, mit **Ja** beantworten.

9.3.12 EKI00016

Meldungscode	■ EKI00016
Meldungstext	■ Element konnte nicht gefunden werden
Meldungstyp	■ Fehlermeldung
	
Auswirkung	■ Bahntreuer NOT-HALT ■ Die Eingabe von aktiven Kommandos (Roboterbewegungen, Programmstart) ist blockiert.
Mögliche Ursache(n)	■ Ursache: Element in XML-Struktur für Datenempfang nicht oder falsch konfiguriert (>>> Seite 75) Lösung: Fehler in XML-Datei beheben (>>> Seite 76) ■ Ursache: Elementname in Zugriffsfunktion falsch programmiert (>>> Seite 77) Lösung: Funktion richtig programmieren (>>> Seite 78)

Ursache: Element in XML-Struktur für Datenempfang nicht oder falsch konfiguriert**Beschreibung**

Das in einer Zugriffsfunktion angegebene Element ist in der XML-Struktur für den Datenempfang nicht konfiguriert oder stimmt nicht mit dem konfigurierten Element überein.

Zugriffsfunktionen

Parameter 2 einer Zugriffsfunktion gibt jeweils das Element an, auf das zugegriffen werden soll.

Zugriffsfunktionen

EKI_STATUS = EKI_GetBool(CHAR[], CHAR[], BOOL)

EKI_STATUS = EKI_GetBoolArray(CHAR[], CHAR[], BOOL[])

EKI_STATUS = EKI_GetInt(CHAR[], CHAR[], INT)

EKI_STATUS = EKI_GetIntArray(CHAR[], CHAR[], INT[])

EKI_STATUS = EKI_GetReal(CHAR[], CHAR[], REAL)

EKI_STATUS = EKI_GetRealArray(CHAR[], CHAR[], REAL[])

Zugriffsfunktionen

EKI_STATUS = EKI_GetString(CHAR[], CHAR[], CHAR[])

EKI_STATUS = EKI_GetFrame(CHAR[], CHAR[], FRAME)

EKI_STATUS = EKI_GetFrameArray(CHAR[], CHAR[], FRAME[])

Konfigurations-datei

Für jede Ethernet-Verbindung muss eine XML-Datei konfiguriert sein. Der Name der XML-Datei ist gleichzeitig der Zugriffsschlüssel in KRL.

Verzeichnis	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Datei	Beispiel: ...\EXT.XML → EKI_INIT("EXT")

Die Empfangsstruktur ist im Abschnitt <RECEIVE> ... </RECEIVE> der XML-Datei konfiguriert. Das Attribut Tag definiert die Elemente, auf die zugegriffen werden kann.

Prüfanweisung

1. Prüfen, ob das Element, auf das versucht wurde zuzugreifen, in der Empfangsstruktur konfiguriert ist.
2. Prüfen, ob der programmierte Elementname mit dem konfigurierten Elementnamen übereinstimmt.

Beispiele**Auslesen von XML-Daten:**

XML-Struktur:

```
<RECEIVE>
  <XML>
    <ELEMENT Tag="Sensor/Message" Type="STRING" />
    <ELEMENT Tag="Sensor/Status/IsActive" Type="BOOL" />
  </XML>
</RECEIVE>
```

Programmierung:

```
RET=EKI_GetString("Channel_1", "Sensor/Message", valueChar[])
RET=EKI_GetBool("Channel_1", "Sensor/Status/IsActive", valueBOOL)
```

Auslesen eines Binärdatensatzes fester Länge (10 Byte):

■ Rohdaten:

```
<RECEIVE>
  <RAW>
    <ELEMENT Tag="Buffer" Type="BYTE" Size="10" />
  </RAW>
</RECEIVE>
```

■ Programmierung:

```
RET=EKI_GetString("Channel_1", "Buffer", Bytes[])
```

Lösung: Fehler in XML-Datei beheben**Beschreibung**

Die Fehler in der XML-Datei müssen behoben werden.

Voraussetzung

- Benutzergruppe Experte
- Betriebsart T1, T2 oder AUT

Vorgehensweise

1. XML-Datei wie benötigt ändern.
2. Wenn XML-Datei offline geändert wurde, sie in das vorgesehene Verzeichnis kopieren und die alte XML-Datei überschreiben.
3. Die Robotersteuerung neu starten, mit den Einstellungen **Kaltstart** und **Dateien neu einlesen**.

Ursache: Elementname in Zugriffsfunktion falsch programmiert

Beschreibung Der in einer Zugriffsfunktion angegebene Elementname stimmt nicht mit dem Elementnamen überein, der in der XML-Struktur für den Datenempfang konfiguriert ist.

Zugriffsfunktionen Parameter 2 einer Zugriffsfunktion gibt jeweils das Element an, auf das zugegriffen werden soll.

Zugriffsfunktionen
EKI_STATUS = EKI_GetBool(CHAR[], CHAR[], BOOL)
EKI_STATUS = EKI_GetBoolArray(CHAR[], CHAR[], BOOL[])
EKI_STATUS = EKI_GetInt(CHAR[], CHAR[], INT)
EKI_STATUS = EKI_GetIntArray(CHAR[], CHAR[], INT[])
EKI_STATUS = EKI_GetReal(CHAR[], CHAR[], REAL)
EKI_STATUS = EKI_GetRealArray(CHAR[], CHAR[], REAL[])
EKI_STATUS = EKI_GetString(CHAR[], CHAR[], CHAR[])
EKI_STATUS = EKI_GetFrame(CHAR[], CHAR[], FRAME)
EKI_STATUS = EKI_GetFrameArray(CHAR[], CHAR[], FRAME[])

Konfigurationsdatei Für jede Ethernet-Verbindung muss eine XML-Datei konfiguriert sein. Der Name der XML-Datei ist gleichzeitig der Zugriffsschlüssel in KRL.

Verzeichnis	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Datei	Beispiel: ...\EXT.XML → EKI_INIT("EXT")

Die Empfangsstruktur ist im Abschnitt <RECEIVE> ... </RECEIVE> der XML-Datei konfiguriert. Das Attribut Tag definiert die Elemente, auf die zugegriffen werden kann.

Prüfanweisung ■ Prüfen, ob der programmierte Elementname mit dem konfigurierten Elementnamen übereinstimmt.

Beispiele Auslesen von XML-Daten:

XML-Struktur:

```
<RECEIVE>
  <XML>
    <ELEMENT Tag="Sensor/Message" Type="STRING" />
    <ELEMENT Tag="Sensor/Status/IsActive" Type="BOOL" />
  </XML>
</RECEIVE>
```

Programmierung:

```
RET=EKI_GetString("Channel_1", "Sensor/Message", valueChar[])
RET=EKI_GetBool("Channel_1", "Sensor/Status/IsActive", valueBOOL)
```

Auslesen eines Binärdatensatzes fester Länge (10 Byte):

■ Rohdaten:

```
<RECEIVE>
  <RAW>
    <ELEMENT Tag="Buffer" Type="BYTE" Size="10" />
  </RAW>
</RECEIVE>
```

■ Programmierung:

```
RET=EKI_GetString("Channel_1", "Buffer", Bytes[])
```

Lösung: Funktion richtig programmieren

Beschreibung	Die Funktion muss richtig programmiert werden.
Voraussetzung	■ Benutzergruppe Experte ■ Betriebsart T1, T2 oder AUT
Vorgehensweise	1. Programm im Navigator markieren und Öffnen drücken. Das Programm wird im Editor angezeigt. 2. Die entsprechende Stelle im Programm suchen und bearbeiten. 3. Datei schließen und die Sicherheitsabfrage, ob die Änderungen gespeichert werden sollen, mit Ja beantworten.

9.3.13 EKI00017

Meldungscode	■ EKI00017
Meldungstext	■ Zusammenstellen der Daten zum Senden fehlgeschlagen
Meldungstyp	■ Fehlermeldung
	
Auswirkung	■ Bahntreuer NOT-HALT ■ Die Eingabe von aktiven Kommandos (Roboterbewegungen, Programmstart) ist blockiert.
Mögliche Ursache(n)	■ Ursache: XML-Struktur für Datenversand falsch konfiguriert (>>> Seite 78) Lösung: Fehler in XML-Datei beheben (>>> Seite 80) ■ Ursache: EKI_Send() falsch programmiert (>>> Seite 80) Lösung: Funktion richtig programmieren (>>> Seite 81)

Ursache: XML-Struktur für Datenversand falsch konfiguriert

Beschreibung	Die XML-Struktur für den Datenversand passt nicht zum XML-Dokument, das mit den XML-Daten beschrieben werden soll. Sie passt möglicherweise auch nicht zur Programmierung der Funktion EKI_Send().
---------------------	--

RET = EKI_Send(CHAR[], CHAR[], INT)	
Funktion	Sendet Daten über einen Kanal (>>> 6.2.4 "Senden von Daten" Seite 36)
Parameter 1	Typ: CHAR Name des geöffneten Kanals, über den gesendet werden soll

RET = EKI_Send(CHAR[], CHAR[], INT)	
Parameter 2	Typ: CHAR Definiert den Umfang der zu sendenden Daten. Bei Kommunikation über XML-Struktur: ■ XPath-Ausdruck des zu sendenden Elements aus der konfigurierten XML-Struktur für den Datenversand. Wird nur das Wurzelement angegeben, wird die gesamte XML-Struktur übertragen (siehe Beispiel 1). Soll nur ein Teil der XML-Struktur übertragen werden, d. h. das Element einer untergeordneten Ebene, muss ausgehend vom Wurzelement der Weg bis zu diesem Element angegeben werden (siehe Beispiel 2). ■ Alternativ: Beliebige Zeichenkette variabler Länge, die übertragen werden soll Bei Kommunikation über Rohdaten: ■ Beliebige Zeichenkette, die übertragen werden soll: ■ Bei Binärdatensätzen mit fester Länge (Attribut <code>Type = "BYTE"</code>): Beliebige Zeichenkette mit fester Länge Die Größe der Zeichenkette in Bytes muss genau dem konfigurierten Attribut <code>Size</code> entsprechen. Bei Überschreitung wird eine Fehlermeldung, bei Unterschreitung eine Warnmeldung ausgegeben. ■ Bei Binärdatensätzen mit variabler Endzeichenfolge (Attribut <code>Type = "STREAM"</code>): Beliebige Zeichenkette mit variabler Länge Die Endzeichenfolge wird automatisch mitgesendet. Hinweis: Enthält eine beliebige Zeichenkette variabler Länge ein ASCII NULL-Zeichen, wird nur der Teil bis zu diesem Zeichen übertragen. Ist dies nicht gewünscht, muss Parameter 3 passend definiert werden.
Parameter 3 (optional)	Typ: INT Nur relevant beim Senden von beliebigen Zeichenketten, deren Länge variabel ist (siehe Parameter 2): Anzahl der maximal zu sendenden Zeichen ASCII NULL-Zeichen werden ignoriert. Ist die Zeichenkette zu lang, wird diese abgeschnitten.
RET	Typ: EKI_STATUS Rückgabewerte der Funktion
Beispiel 1	RET = EKI_Send("Channel_1", "Root")
Beispiel 2	RET = EKI_Send("Channel_1", "Root/Test")
Beispiel 3	RET = EKI_Send("Channel_1", MyBytes[])
Beispiel 4	RET = EKI_Send("Channel_1", MyBytes[], 6)

Konfigurations-datei

Für jede Ethernet-Verbindung muss eine XML-Datei konfiguriert sein. Der Name der XML-Datei ist gleichzeitig der Zugriffsschlüssel in KRL.

Verzeichnis	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Datei	Beispiel: ... \EXT.XML → EKI_INIT("EXT")

Die Sendestruktur ist im Abschnitt <SEND> ... </SEND> der XML-Datei konfiguriert.

Prüfanweisung

1. Konfiguration der Sendestruktur in der XML-Datei prüfen.
2. Prüfen, ob die Programmierung der gesendeten Daten zur Konfiguration passt.

Lösung: Fehler in XML-Datei beheben

Beschreibung	Die Fehler in der XML-Datei müssen behoben werden.
Voraussetzung	■ Benutzergruppe Experte ■ Betriebsart T1, T2 oder AUT
Vorgehensweise	1. XML-Datei wie benötigt ändern. 2. Wenn XML-Datei offline geändert wurde, sie in das vorgesehene Verzeichnis kopieren und die alte XML-Datei überschreiben. 3. Die Robotersteuerung neu starten, mit den Einstellungen Kaltstart und Dateien neu einlesen.

Ursache: EKI_Send() falsch programmiert

Beschreibung	Die zu sendenden Daten sind in der Funktion EKI_Send() falsch angegeben. Sie passen möglicherweise nicht zur konfigurierten XML-Struktur für den Datenversand.
---------------------	--

RET = EKI_Send(CHAR[], CHAR[], INT)	
Funktion	Sendet Daten über einen Kanal (>>> 6.2.4 "Senden von Daten" Seite 36)
Parameter 1	Typ: CHAR Name des geöffneten Kanals, über den gesendet werden soll
Parameter 2	Typ: CHAR Definiert den Umfang der zu sendenden Daten. Bei Kommunikation über XML-Struktur: ■ XPath-Ausdruck des zu sendenden Elements aus der konfigurierten XML-Struktur für den Datenversand. Wird nur das Wurzelement angegeben, wird die gesamte XML-Struktur übertragen (siehe Beispiel 1). Soll nur ein Teil der XML-Struktur übertragen werden, d. h. das Element einer untergeordneten Ebene, muss ausgehend vom Wurzelement der Weg bis zu diesem Element angegeben werden (siehe Beispiel 2). ■ Alternativ: Beliebige Zeichenkette variabler Länge, die übertragen werden soll Bei Kommunikation über Rohdaten: ■ Beliebige Zeichenkette, die übertragen werden soll: ■ Bei Binärdatensätzen mit fester Länge (Attribut <code>Type = "BYTE"</code>): Beliebige Zeichenkette mit fester Länge Die Größe der Zeichenkette in Bytes muss genau dem konfigurierten Attribut <code>Size</code> entsprechen. Bei Überschreitung wird eine Fehlermeldung, bei Unterschreitung eine Warnmeldung ausgegeben. ■ Bei Binärdatensätzen mit variabler Endzeichenfolge (Attribut <code>Type = "STREAM"</code>): Beliebige Zeichenkette mit variabler Länge Die Endzeichenfolge wird automatisch mitgesendet. Hinweis: Enthält eine beliebige Zeichenkette variabler Länge ein ASCII NULL-Zeichen, wird nur der Teil bis zu diesem Zeichen übertragen. Ist dies nicht gewünscht, muss Parameter 3 passend definiert werden.

RET = EKI_Send(CHAR[], CHAR[], INT)	
Parameter 3 (optional)	Typ: INT Nur relevant beim Senden von beliebigen Zeichenketten, deren Länge variabel ist (siehe Parameter 2): Anzahl der maximal zu sendenden Zeichen ASCII NULL-Zeichen werden ignoriert. Ist die Zeichenkette zu lang, wird diese abgeschnitten.
RET	Typ: EKI_STATUS Rückgabewerte der Funktion
Beispiel 1	RET = EKI_Send("Channel_1", "Root")
Beispiel 2	RET = EKI_Send("Channel_1", "Root/Test")
Beispiel 3	RET = EKI_Send("Channel_1", MyBytes[])
Beispiel 4	RET = EKI_Send("Channel_1", MyBytes[], 6)

Konfigurationsdatei Für jede Ethernet-Verbindung muss eine XML-Datei konfiguriert sein. Der Name der XML-Datei ist gleichzeitig der Zugriffsschlüssel in KRL.

Verzeichnis	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Datei	Beispiel: ... \EXT.XML → EKI_INIT("EXT")

Die Sendestruktur ist im Abschnitt <SEND> ... </SEND> der XML-Datei konfiguriert.

Prüfanweisung

1. Konfiguration der Sendestruktur in der XML-Datei prüfen.
2. Prüfen, ob die Programmierung der gesendeten Daten zur Konfiguration passt.

Lösung: Funktion richtig programmieren

Beschreibung Die Funktion muss richtig programmiert werden.

Voraussetzung

- Benutzergruppe Experte
- Betriebsart T1, T2 oder AUT

Vorgehensweise

1. Programm im Navigator markieren und **Öffnen** drücken. Das Programm wird im Editor angezeigt.
2. Die entsprechende Stelle im Programm suchen und bearbeiten.
3. Datei schließen und die Sicherheitsabfrage, ob die Änderungen gespeichert werden sollen, mit **Ja** beantworten.

9.3.14 EKI00018

Meldungscode	■ EKI00018
Meldungstext	■ Senden von Daten fehlgeschlagen
Meldungstyp	■ Fehlermeldung
	
Auswirkung	■ Bahntreuer NOT-HALT ■ Die Eingabe von aktiven Kommandos (Roboterbewegungen, Programmstart) ist blockiert.

Mögliche Ursache(n)	■ Ursache: Netzkabel defekt oder nicht richtig angeschlossen (>>> Seite 82) Lösung: Netzkabel tauschen oder richtig einstecken (>>> Seite 82)
	■ Ursache: Keine Ethernet-Verbindung wegen Software-Fehler, externes System (>>> Seite 82) Lösung: Fehler in Software des externen Systems beheben (>>> Seite 82)
	■ Ursache: Keine Ethernet-Verbindung wegen Hardware-Fehler, externes System (>>> Seite 82) Lösung: Hardware-Fehler am externen System beheben (>>> Seite 82)

Ursache: Netzkabel defekt oder nicht richtig angeschlossen

Beschreibung	Das Netzkabel ist defekt oder die Steckverbindung ist fehlerhaft. So kann man prüfen, ob Netzkabel und Verbindungsstecker korrekt angeschlossen sind:
Prüfanweisung	1. Die Netzkabel auf korrekte Steckverbindung und festen Sitz prüfen. 2. Die Netzkabel untereinander quer tauschen.

Lösung: Netzkabel tauschen oder richtig einstecken

Vorgehensweise	■ Netzkabel tauschen oder richtig einstecken.
-----------------------	---

Ursache: Keine Ethernet-Verbindung wegen Software-Fehler, externes System

Beschreibung	Wegen eines Fehlers in der Software des externen Systems liegt keine Ethernet-Verbindung vor.
Prüfanweisung	■ Externes System auf Software-Fehler überprüfen.

Lösung: Fehler in Software des externen Systems beheben

Beschreibung	Der Fehler in der Software des externen Systems muss behoben werden.
---------------------	--

Ursache: Keine Ethernet-Verbindung wegen Hardware-Fehler, externes System

Beschreibung	Wegen eines Hardware-Fehlers am externen System, z. B. durch Wackelkontakt an Buchse, defekte Netzwerkkarte, etc., liegt keine Ethernet-Verbindung vor.
Prüfanweisung	■ Externes System auf Hardware-Fehler überprüfen.

Lösung: Hardware-Fehler am externen System beheben

Beschreibung	Der Hardware-Fehler am externen System muss behoben werden.
---------------------	---

9.3.15 EKI00019

Meldungscode	■ EKI00019
Meldungstext	■ Keine Daten zum Senden vorhanden
Meldungstyp	■ Fehlermeldung
	
Auswirkung	■ Bahntreuer NOT-HALT ■ Die Eingabe von aktiven Kommandos (Roboterbewegungen, Programmstart) ist blockiert.
Mögliche Ursache(n)	■ Ursache: EKI_Send() enthält keine zu sendenden Daten (>>> Seite 83) ■ Lösung: Funktion richtig programmieren (>>> Seite 84)

Ursache: EKI_Send() enthält keine zu sendenden Daten

Beschreibung Eine Sendefunktion EKI_Send() enthält keine zu sendenden Daten (Parameter 2).

RET = EKI_Send(CHAR[], CHAR[], INT)	
Funktion	Sendet Daten über einen Kanal (>>> 6.2.4 "Senden von Daten" Seite 36)
Parameter 1	Typ: CHAR Name des geöffneten Kanals, über den gesendet werden soll
Parameter 2	Typ: CHAR Definiert den Umfang der zu sendenden Daten. Bei Kommunikation über XML-Struktur: ■ XPath-Ausdruck des zu sendenden Elements aus der konfigurierten XML-Struktur für den Datenversand. Wird nur das Wurzelement angegeben, wird die gesamte XML-Struktur übertragen (siehe Beispiel 1). Soll nur ein Teil der XML-Struktur übertragen werden, d. h. das Element einer untergeordneten Ebene, muss ausgehend vom Wurzelement der Weg bis zu diesem Element angegeben werden (siehe Beispiel 2). ■ Alternativ: Beliebige Zeichenkette variabler Länge, die übertragen werden soll Bei Kommunikation über Rohdaten: ■ Beliebige Zeichenkette, die übertragen werden soll: ■ Bei Binärdatensätzen mit fester Länge (Attribut <code>Type = "BYTE"</code>): Beliebige Zeichenkette mit fester Länge Die Größe der Zeichenkette in Bytes muss genau dem konfigurierten Attribut <code>Size</code> entsprechen. Bei Überschreitung wird eine Fehlermeldung, bei Unterschreitung eine Warnmeldung ausgegeben. ■ Bei Binärdatensätzen mit variabler Endzeichenfolge (Attribut <code>Type = "STREAM"</code>): Beliebige Zeichenkette mit variabler Länge Die Endzeichenfolge wird automatisch mitgesendet. Hinweis: Enthält eine beliebige Zeichenkette variabler Länge ein ASCII NULL-Zeichen, wird nur der Teil bis zu diesem Zeichen übertragen. Ist dies nicht gewünscht, muss Parameter 3 passend definiert werden.

RET = EKI_Send(CHAR[], CHAR[], INT)	
Parameter 3 (optional)	Typ: INT Nur relevant beim Senden von beliebigen Zeichenketten, deren Länge variabel ist (siehe Parameter 2): Anzahl der maximal zu sendenden Zeichen ASCII NULL-Zeichen werden ignoriert. Ist die Zeichenkette zu lang, wird diese abgeschnitten.
RET	Typ: EKI_STATUS Rückgabewerte der Funktion
Beispiel 1	RET = EKI_Send("Channel_1", "Root")
Beispiel 2	RET = EKI_Send("Channel_1", "Root/Test")
Beispiel 3	RET = EKI_Send("Channel_1", MyBytes[])
Beispiel 4	RET = EKI_Send("Channel_1", MyBytes[], 6)

Prüfanweisung ■ Im KRL-Programm prüfen, ob die zu sendenden Daten angegeben sind.

Lösung: Funktion richtig programmieren

Beschreibung Die Funktion muss richtig programmiert werden.

Voraussetzung ■ Benutzergruppe Experte
 ■ Betriebsart T1, T2 oder AUT

Vorgehensweise

1. Programm im Navigator markieren und **Öffnen** drücken. Das Programm wird im Editor angezeigt.
2. Die entsprechende Stelle im Programm suchen und bearbeiten.
3. Datei schließen und die Sicherheitsabfrage, ob die Änderungen gespeichert werden sollen, mit **Ja** beantworten.

9.3.16 EKI00020

Meldungscode	■ EKI00020
Meldungstext	■ Datentypen passen nicht zusammen
Meldungstyp	■ Fehlermeldung
	
Auswirkung	■ Bahntreuer NOT-HALT ■ Die Eingabe von aktiven Kommandos (Roboterbewegungen, Programmstart) ist blockiert.
Mögliche Ursache(n)	■ Ursache: Datentyp in XML-Struktur für Datenempfang falsch konfiguriert (>>> Seite 85) Lösung: Fehler in XML-Datei beheben (>>> Seite 86) ■ Ursache: Datentyp in Zugriffsfunktion falsch programmiert (>>> Seite 86) Lösung: Funktion richtig programmieren (>>> Seite 87)

Ursache: Datentyp in XML-Struktur für Datenempfang falsch konfiguriert

Beschreibung Der in einer Zugriffsfunktion angegebene Datentyp eines Elements stimmt nicht mit dem Datentyp überein, der in der XML-Struktur für den Datenempfang dafür konfiguriert ist.

Zugriffsfunktionen Parameter 3 einer Zugriffsfunktion gibt jeweils den Datentyp des Elements an, auf das zugegriffen werden soll.

Zugriffsfunktionen
EKI_STATUS = EKI_GetBool(CHAR[], CHAR[], BOOL)
EKI_STATUS = EKI_GetBoolArray(CHAR[], CHAR[], BOOL[])
EKI_STATUS = EKI_GetInt(CHAR[], CHAR[], INT)
EKI_STATUS = EKI_GetIntArray(CHAR[], CHAR[], INT[])
EKI_STATUS = EKI_GetReal(CHAR[], CHAR[], REAL)
EKI_STATUS = EKI_GetRealArray(CHAR[], CHAR[], REAL[])
EKI_STATUS = EKI_GetString(CHAR[], CHAR[], CHAR[])
EKI_STATUS = EKI_GetFrame(CHAR[], CHAR[], FRAME)
EKI_STATUS = EKI_GetFrameArray(CHAR[], CHAR[], FRAME[])

Konfigurationsdatei Für jede Ethernet-Verbindung muss eine XML-Datei konfiguriert sein. Der Name der XML-Datei ist gleichzeitig der Zugriffsschlüssel in KRL.

Verzeichnis	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Datei	Beispiel: ... \EXT.XML → EKI_INIT("EXT")

Die Empfangsstruktur ist im Abschnitt <RECEIVE> ... </RECEIVE> der XML-Datei konfiguriert. Das Attribut `Type` definiert den Datentyp eines Elements.

Prüfanweisung ■ Prüfen, ob der programmierte Datentyp mit dem in der Empfangsstruktur konfigurierten Datentyp des Elements übereinstimmt.

Beispiele Auslesen von XML-Daten:

XML-Struktur:

```
<RECEIVE>
  <XML>
    <ELEMENT Tag="Sensor/Message" Type="STRING" />
    <ELEMENT Tag="Sensor/Status/IsActive" Type="BOOL" />
  </XML>
</RECEIVE>
```

Programmierung:

```
...
CHAR valueChar[256]
BOOL valueBOOL
...
RET=EKI_GetString("Channel_1", "Sensor/Message", valueChar[])
RET=EKI_GetBool("Channel_1", "Sensor/Status/IsActive", valueBOOL)
```

Auslesen eines Binärdatensatzes fester Länge (10 Byte):

■ Rohdaten:

```
<RECEIVE>
  <RAW>
    <ELEMENT Tag="Buffer" Type="BYTE" Size="10" />
  </RAW>
</RECEIVE>
```

■ Programmierung:

```
...
CHAR Bytes[10]
...
RET=EKI_GetString("Channel_1", "Buffer", Bytes[])
```

Lösung: Fehler in XML-Datei beheben

Beschreibung Die Fehler in der XML-Datei müssen behoben werden.

Voraussetzung

- Benutzergruppe Experte
- Betriebsart T1, T2 oder AUT

Vorgehensweise

1. XML-Datei wie benötigt ändern.
2. Wenn XML-Datei offline geändert wurde, sie in das vorgesehene Verzeichnis kopieren und die alte XML-Datei überschreiben.
3. Die Robotersteuerung neu starten, mit den Einstellungen **Kaltstart** und **Dateien neu einlesen**.

Ursache: Datentyp in Zugriffsfunktion falsch programmiert

Beschreibung Der in einer Zugriffsfunktion angegebene Datentyp eines Elements stimmt nicht mit dem Datentyp überein, der in der XML-Struktur für den Datenempfang dafür konfiguriert ist.

Zugriffsfunktionen Parameter 3 einer Zugriffsfunktion gibt jeweils den Datentyp des Elements an, auf das zugegriffen werden soll.

Zugriffsfunktionen	
EKI_STATUS =	EKI_GetBool(CHAR[], CHAR[], BOOL)
EKI_STATUS =	EKI_GetBoolArray(CHAR[], CHAR[], BOOL[])
EKI_STATUS =	EKI_GetInt(CHAR[], CHAR[], INT)
EKI_STATUS =	EKI_GetIntArray(CHAR[], CHAR[], INT[])
EKI_STATUS =	EKI_GetReal(CHAR[], CHAR[], REAL)
EKI_STATUS =	EKI_GetRealArray(CHAR[], CHAR[], REAL[])
EKI_STATUS =	EKI_GetString(CHAR[], CHAR[], CHAR[])
EKI_STATUS =	EKI_GetFrame(CHAR[], CHAR[], FRAME)
EKI_STATUS =	EKI_GetFrameArray(CHAR[], CHAR[], FRAME[])

Konfigurationsdatei Für jede Ethernet-Verbindung muss eine XML-Datei konfiguriert sein. Der Name der XML-Datei ist gleichzeitig der Zugriffsschlüssel in KRL.

Verzeichnis	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Datei	Beispiel: ... \EXT.XML → EKI_INIT("EXT")

Die Empfangsstruktur ist im Abschnitt <RECEIVE> ... </RECEIVE> der XML-Datei konfiguriert. Das Attribut `Type` definiert den Datentyp eines Elements.

Prüfanweisung

- Prüfen, ob der programmierte Datentyp mit dem in der Empfangsstruktur konfigurierten Datentyp des Elements übereinstimmt.

Beispiele **Auslesen von XML-Daten:**

XML-Struktur:

```
<RECEIVE>
<XML>
```

```

<ELEMENT Tag="Sensor/Message" Type="STRING" />
<ELEMENT Tag="Sensor/Status/IsActive" Type="BOOL" />
</XML>
</RECEIVE>

```

Programmierung:

```

...
CHAR valueChar[256]
BOOL valueBOOL
...
RET=EKI_GetString("Channel_1", "Sensor/Message", valueChar[])
RET=EKI_GetBool("Channel_1", "Sensor/Status/IsActive", valueBOOL)

```

Auslesen eines Binärdatensatzes fester Länge (10 Byte):

■ Rohdaten:

```

<RECEIVE>
<RAW>
<ELEMENT Tag="Buffer" Type="BYTE" Size="10" />
</RAW>
</RECEIVE>

```

■ Programmierung:

```

...
CHAR Bytes[10]
...
RET=EKI_GetString("Channel_1", "Buffer", Bytes[])

```

Lösung: Funktion richtig programmieren

Beschreibung	Die Funktion muss richtig programmiert werden.
Voraussetzung	■ Benutzergruppe Experte ■ Betriebsart T1, T2 oder AUT
Vorgehensweise	1. Programm im Navigator markieren und Öffnen drücken. Das Programm wird im Editor angezeigt. 2. Die entsprechende Stelle im Programm suchen und bearbeiten. 3. Datei schließen und die Sicherheitsabfrage, ob die Änderungen gespeichert werden sollen, mit Ja beantworten.

9.3.17 EKI00021

Meldungscode	■ EKI00021
Meldungstext	■ Mit maximaler Datenhaltung Systemspeicher nicht ausreichend
Meldungstyp	■ Fehlermeldung
	
Auswirkung	■ Bahntreuer NOT-HALT ■ Die Eingabe von aktiven Kommandos (Roboterbewegungen, Programmstart) ist blockiert.

Mögliche Ursache(n)	■ Ursache: Reservierter Speicher nicht ausreichend (>>> Seite 88) Lösung: Speicher erhöhen (>>> Seite 88)
	■ Ursache: Verbindungskonfiguration verbraucht zu viel Speicher (>>> Seite 88) Lösung: Verbindungskonfiguration anpassen, um weniger Speicher zu verbrauchen (>>> Seite 89)

Ursache: Reservierter Speicher nicht ausreichend

Beschreibung Der für Ethernet KRL reservierte Speicher ist nicht ausreichend.

Prüfanweisung

- Prüfen, ob der Speicherbedarf durch eine andere Konfiguration oder Programmierung reduziert werden kann.

Wenn dies nicht möglich ist, darf der Speicher nach Rücksprache mit der KUKA Roboter GmbH erhöht werden.

Lösung: Speicher erhöhen

 Der Speicher darf nur nach Rücksprache mit der KUKA Roboter GmbH erhöht werden. (>>> 11 "KUKA Service" Seite 113)

Voraussetzung

- Benutzergruppe Experte
- Betriebsart T1, T2 oder AUT

Vorgehensweise

1. Im Verzeichnis C:\KRC\ROBOTER\Config\User\Common die Datei EthernetKRL.XML öffnen.
2. Im Abschnitt <EthernetKRL> im Element <MemSize> die gewünschte Speicherkapazität in Byte eintragen.

```
<EthernetKRL>
  <Interface>
    <MemSize>1048576</MemSize>
    ...
  </EthernetKRL>
```

3. Datei schließen und die Sicherheitsabfrage, ob die Änderungen gespeichert werden sollen, mit **Ja** beantworten.
4. Die Robotersteuerung neu starten, mit den Einstellungen **Kaltstart** und **Dateien neu einlesen**.

Ursache: Verbindungskonfiguration verbraucht zu viel Speicher

Beschreibung Die konfigurierten Ethernet-Verbindungen verbrauchen zu viel Speicher.

Konfigurationsdatei Für jede Ethernet-Verbindung muss eine XML-Datei konfiguriert sein. Der Name der XML-Datei ist gleichzeitig der Zugriffsschlüssel in KRL.

Verzeichnis	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Datei	Beispiel: ... \EXT.XML —> EKI_INIT("EXT")

Prüfanweisung

- XML-Datei/en daraufhin überprüfen, ob die Ethernet-Verbindungen so konfiguriert werden können, dass weniger Speicher verbraucht wird.

Lösung: Verbindungskonfiguration anpassen, um weniger Speicher zu verbrauchen

- Voraussetzung**
- Benutzergruppe Experte
 - Betriebsart T1, T2 oder AUT
- Vorgehensweise**
1. XML-Datei wie benötigt ändern.
 2. Wenn XML-Datei offline geändert wurde, sie in das vorgesehene Verzeichnis kopieren und die alte XML-Datei überschreiben.
 3. Die Robotersteuerung neu starten, mit den Einstellungen **Kaltstart** und **Dateien neu einlesen**.

9.3.18 EKI00022

Meldungscode	■ EKI00022
Meldungstext	■ Fehler bei Lesen der Konfiguration. XML Fehler.
Meldungstyp	■ Fehlermeldung
	
Auswirkung	■ Bahntreuer NOT-HALT ■ Die Eingabe von aktiven Kommandos (Roboterbewegungen, Programmstart) ist blockiert.
Mögliche Ursache(n)	■ Ursache: Schemafehler in XML-Datei mit der Verbindungskonfiguration (>>> Seite 89) ■ Lösung: Fehler in XML-Datei beheben (>>> Seite 89)

Ursache: Schemafehler in XML-Datei mit der Verbindungskonfiguration

Beschreibung Die XML-Datei mit der Verbindungskonfiguration kann wegen eines Schemafehlers nicht gelesen werden.

KUKA.Ethernet KRL verwendet das XPath-Schema. Die Syntax, die das Schema vorgibt, muss genau eingehalten werden. Z. B. dürfen keine Satzzeichen oder Strukturelemente fehlen.

Die Schreibweise der Elemente und Attribute in der XML-Datei, einschließlich Groß- und/oder Kleinschreibung, ist vorgegeben und muss genau eingehalten werden.



Weitere Informationen zum XPath-Schema sind in der Dokumentation zu KUKA.Ethernet KRL zu finden.

Konfigurationsdatei Für jede Ethernet-Verbindung muss eine XML-Datei konfiguriert sein. Der Name der XML-Datei ist gleichzeitig der Zugriffsschlüssel in KRL.

Verzeichnis	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Datei	Beispiel: ... \EXT.XML → EKI_INIT("EXT")

Lösung: Fehler in XML-Datei beheben

Beschreibung Die Fehler in der XML-Datei müssen behoben werden.

- Voraussetzung**
- Benutzergruppe Experte
 - Betriebsart T1, T2 oder AUT

- Vorgehensweise**
1. XML-Datei wie benötigt ändern.
 2. Wenn XML-Datei offline geändert wurde, sie in das vorgesehene Verzeichnis kopieren und die alte XML-Datei überschreiben.
 3. Die Robotersteuerung neu starten, mit den Einstellungen **Kaltstart** und **Dateien neu einlesen**.

9.3.19 EKI00023

Meldungscode	■ EKI00023
Meldungstext	■ Initialisierung bereits durchgeführt.
Meldungstyp	■ Fehlermeldung
	
Auswirkung	■ Bahntreuer NOT-HALT ■ Die Eingabe von aktiven Kommandos (Roboterbewegungen, Programmstart) ist blockiert.
Mögliche Ursache(n)	■ Ursache: Ethernet-Verbindung bereits mit EKI_Init() initialisiert (>>> Seite 90) Lösung: Zu viel programmierte Funktion löschen (>>> Seite 91)

Ursache: Ethernet-Verbindung bereits mit EKI_Init() initialisiert

Beschreibung Die Ethernet-Verbindung ist bereits mit der Funktion EKI_Init() initialisiert worden.

RET = EKI_Init(CHAR[])	
Funktion	Initialisiert einen Kanal für die Ethernet-Kommunikation Folgende Aktionen werden ausgeführt: ■ Einlesen der Verbindungskonfiguration ■ Erstellen der Datenspeicher ■ Vorbereiten der Ethernet-Verbindung
Parameter	Typ: CHAR Name des Kanals (= Name der XML-Datei mit der Verbindungskonfiguration)
RET	Typ: EKI_STATUS Rückgabewerte der Funktion
Beispiel	RET = EKI_Init("Channel_1")

So kann man prüfen, ob die Verbindung bereits initialisiert ist:

- Voraussetzung**
- Benutzergruppe Experte
 - Programm ist angewählt oder geöffnet.
- Prüfanweisung**
- Prüfen, ob folgende Zeile zwischen der Erstinitialisierung und dem Schließen der Verbindung weitere Male programmiert ist:


```
RET = EKI_Init("Dateiname")
```

 - *Dateiname:* Name der XML-Datei mit der Verbindungskonfiguration

Lösung: Zu viel programmierte Funktion löschen

Beschreibung	Zu viel programmierte Funktion muss gelöscht werden.
Voraussetzung	■ Benutzergruppe Experte ■ Betriebsart T1, T2 oder AUT
Vorgehensweise	1. Programm im Navigator markieren und Öffnen drücken. Das Programm wird im Editor angezeigt. 2. Die entsprechende Stelle im Programm suchen und bearbeiten. 3. Datei schließen und die Sicherheitsabfrage, ob die Änderungen gespeichert werden sollen, mit Ja beantworten.

9.3.20 EKI00024

Meldungscode	■ EKI00024
Meldungstext	■ Bindung an interne Parameter (Port,IP) fehlgeschlagen
Meldungstyp	■ Fehlermeldung
	
Auswirkung	■ Bahntreuer NOT-HALT ■ Die Eingabe von aktiven Kommandos (Roboterbewegungen, Programmstart) ist blockiert.
Mögliche Ursache(n)	■ Ursache: IP-Adresse und/oder Port-Nummer für EKI nicht oder falsch angegeben (>>> Seite 91) ■ Lösung: IP-Adresse und Port-Nummer richtig in XML-Datei eintragen (>>> Seite 92)

Ursache: IP-Adresse und/oder Port-Nummer für EKI nicht oder falsch angegeben

Beschreibung	EKI ist als Server und das externe System als Client konfiguriert. In der XML-Datei mit der Verbindungskonfiguration sind die IP-Adresse und/oder Port-Nummer für das EKI nicht oder formal falsch angegeben. Um die Netzwerksicherheit zu gewährleisten, kann festgelegt werden, von welchem Subnetz auf EKI zugegriffen werden kann.
---------------------	--

Beispiele	Verbindung nur über das Subnetz 192.168.X.X auf den Port 54600 möglich:
------------------	---

```
<INTERNAL>
<IP>192.168.23.1</IP>
<PORT>54600</PORT>
</INTERNAL>
```

Verbindung über alle im KLI konfigurierten Subnetze auf den Port 54600 möglich:

```
<INTERNAL>
<IP>0.0.0.0</IP>
<PORT>54600</PORT>
</INTERNAL>
```

Konfigurations-datei

Für jede Ethernet-Verbindung muss eine XML-Datei konfiguriert sein. Der Name der XML-Datei ist gleichzeitig der Zugriffsschlüssel in KRL.

Verzeichnis	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Datei	Beispiel: ... \EXT.XML → EKI_INIT("EXT")

Prüfanweisung

- In der XML-Datei mit der Verbindungskonfiguration prüfen, ob IP-Adresse und Port-Nummer richtig eingetragen sind.

Lösung: IP-Adresse und Port-Nummer richtig in XML-Datei eintragen**Voraussetzung**

- Benutzergruppe Experte
- Betriebsart T1, T2 oder AUT

Vorgehensweise

1. XML-Datei wie benötigt ändern.
2. Wenn XML-Datei offline geändert wurde, sie in das vorgesehene Verzeichnis kopieren und die alte XML-Datei überschreiben.
3. Die Robotersteuerung neu starten, mit den Einstellungen **Kaltstart** und **Dateien neu einlesen**.

9.3.21 EK100027

Meldungscode	■ EK100027
Meldungstext	■ Das KRL CHAR[] Feld ist zu klein.
Meldungstyp	■ Fehlermeldung
	
Auswirkung	■ Bahntreuer NOT-HALT ■ Die Eingabe von aktiven Kommandos (Roboterbewegungen, Programmstart) ist blockiert.
Mögliche Ursache(n)	■ Ursache: CHAR-Feld zu klein für empfangene Daten (>>> Seite 92) ■ Lösung: Speichergröße des CHAR-Felds erhöhen (>>> Seite 92)

Ursache: CHAR-Feld zu klein für empfangene Daten**Beschreibung**

Das im KRL-Programm definierte CHAR-Feld ist zu klein für die empfangenen Daten.

Prüfanweisung

- Prüfen, welche Länge die vom externen System gesendeten Datensätze maximal annehmen.

Lösung: Speichergröße des CHAR-Felds erhöhen**Beschreibung**

Die Speichergröße des CHAR-Felds muss erhöht werden.

Voraussetzung

- Benutzergruppe Experte
- Betriebsart T1, T2 oder AUT

Vorgehensweise

1. Programm im Navigator markieren und **Öffnen** drücken. Das Programm wird im Editor angezeigt.
2. Die entsprechende Stelle im Programm suchen und bearbeiten.
3. Datei schließen und die Sicherheitsabfrage, ob die Änderungen gespeichert werden sollen, mit **Ja** beantworten.

9.3.22 EKI00512

Meldungscode	■ EKI00512
Meldungstext	■ Ethernetverbindung gestört
Meldungstyp	■ Fehlermeldung
	
Auswirkung	■ Bahntreuer NOT-HALT ■ Die Eingabe von aktiven Kommandos (Roboterbewegungen, Programmstart) ist blockiert.
Mögliche Ursache(n)	■ Ursache: Netzkabel defekt oder nicht richtig angeschlossen (>>> Seite 93) Lösung: Netzkabel tauschen oder richtig einstecken (>>> Seite 93) ■ Ursache: Keine Ethernet-Verbindung wegen Software-Fehler, externes System (>>> Seite 93) Lösung: Fehler in Software des externen Systems beheben (>>> Seite 93) ■ Ursache: Keine Ethernet-Verbindung wegen Hardware-Fehler, externes System (>>> Seite 94) Lösung: Hardware-Fehler am externen System beheben (>>> Seite 94)

Ursache: Netzkabel defekt oder nicht richtig angeschlossen

Beschreibung	Das Netzkabel ist defekt oder die Steckverbindung ist fehlerhaft. So kann man prüfen, ob Netzkabel und Verbindungsstecker korrekt angeschlossen sind:
Prüfanweisung	1. Die Netzkabel auf korrekte Steckverbindung und festen Sitz prüfen. 2. Die Netzkabel untereinander quer tauschen.

Lösung: Netzkabel tauschen oder richtig einstecken

Vorgehensweise	■ Netzkabel tauschen oder richtig einstecken.
-----------------------	---

Ursache: Keine Ethernet-Verbindung wegen Software-Fehler, externes System

Beschreibung	Wegen eines Fehlers in der Software des externen Systems liegt keine Ethernet-Verbindung vor.
Prüfanweisung	■ Externes System auf Software-Fehler überprüfen.

Lösung: Fehler in Software des externen Systems beheben

Beschreibung	Der Fehler in der Software des externen Systems muss behoben werden.
---------------------	--

Ursache: Keine Ethernet-Verbindung wegen Hardware-Fehler, externes System

Beschreibung Wegen eines Hardware-Fehlers am externen System, z. B. durch Wackelkontakt an Buchse, defekte Netzwerkkarte, etc., liegt keine Ethernet-Verbindung vor.

Prüfanweisung ■ Externes System auf Hardware-Fehler überprüfen.

Lösung: Hardware-Fehler am externen System beheben

Beschreibung Der Hardware-Fehler am externen System muss behoben werden.

9.3.23 EK100768

Meldungscode	■ EK100768
Meldungstext	■ Ping meldet kein Kontakt
Meldungstyp	■ Fehlermeldung
	
Auswirkung	■ Bahntreuer NOT-HALT ■ Die Eingabe von aktiven Kommandos (Roboterbewegungen, Programmstart) ist blockiert.
Mögliche Ursache(n)	■ Ursache: Netzkabel defekt oder nicht richtig angeschlossen (>>> Seite 94) Lösung: Netzkabel tauschen oder richtig einstecken (>>> Seite 94) ■ Ursache: Keine Ethernet-Verbindung wegen Hardware-Fehler, externes System (>>> Seite 94) Lösung: Hardware-Fehler am externen System beheben (>>> Seite 95)

Ursache: Netzkabel defekt oder nicht richtig angeschlossen

Beschreibung Das Netzkabel ist defekt oder die Steckverbindung ist fehlerhaft.
So kann man prüfen, ob Netzkabel und Verbindungsstecker korrekt angeschlossen sind:

Prüfanweisung

1. Die Netzkabel auf korrekte Steckverbindung und festen Sitz prüfen.
2. Die Netzkabel untereinander quer tauschen.

Lösung: Netzkabel tauschen oder richtig einstecken

Vorgehensweise ■ Netzkabel tauschen oder richtig einstecken.

Ursache: Keine Ethernet-Verbindung wegen Hardware-Fehler, externes System

Beschreibung Wegen eines Hardware-Fehlers am externen System, z. B. durch Wackelkontakt an Buchse, defekte Netzwerkkarte, etc., liegt keine Ethernet-Verbindung vor.

Prüfanweisung ■ Externes System auf Hardware-Fehler überprüfen.

Lösung: Hardware-Fehler am externen System beheben

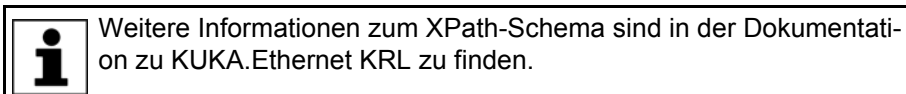
Beschreibung Der Hardware-Fehler am externen System muss behoben werden.

9.3.24 EKI01024

Meldungscode	■ EKI01024
Meldungstext	■ Fehler bei Lesen empfangener XML-Daten
Meldungstyp	■ Fehlermeldung
	
Auswirkung	■ Bahntreuer NOT-HALT ■ Die Eingabe von aktiven Kommandos (Roboterbewegungen, Programmstart) ist blockiert.
Mögliche Ursache(n)	■ Ursache: Gesendetes XML-Dokument entspricht nicht dem XPath-Schema (>>> Seite 95) ■ Lösung: Gesendetes XML-Dokument nach XPath-Schema aufbauen (>>> Seite 95)

Ursache: Gesendetes XML-Dokument entspricht nicht dem XPath-Schema

Beschreibung Das vom externen System gesendete XML-Dokument entspricht nicht dem XPath-Schema.

**Lösung: Gesendetes XML-Dokument nach XPath-Schema aufbauen**

Beschreibung Das vom externen System gesendete XML-Dokument ist nach XPath-Schema aufzubauen.

9.3.25 EKI01280

Meldungscode	■ EKI01280
Meldungstext	■ Grenze speicherbarer Elemente erreicht
Meldungstyp	■ Fehlermeldung
	
Auswirkung	■ Bahntreuer NOT-HALT ■ Die Eingabe von aktiven Kommandos (Roboterbewegungen, Programmstart) ist blockiert.
Mögliche Ursache(n)	■ Ursache: Speicherlimit erreicht (Anzahl Datenelemente pro Speicher) (>>> Seite 96) ■ Lösung: Speicherlimit in XML-Datei erhöhen (>>> Seite 96)

- **Ursache:** Speicherlimit erreicht (Anzahl Datenelemente pro Speicher) (>>> Seite 96)
Lösung: Datenspeicher zyklisch prüfen und vor Erreichen des Limits sperren (>>> Seite 97)
- **Ursache:** EKI: EKI Speicher wird nicht ausgelesen (>>> Seite 98)
Lösung: Programm ändern (>>> Seite 98)

Ursache: Speicherlimit erreicht (Anzahl Datenelemente pro Speicher)

Beschreibung Die Anzahl speicherbarer Datenelemente ist begrenzt. Dieses Speicherlimit wurde erreicht. Die Ethernet-Verbindung wurde geschlossen, um den Empfang weiterer Daten zu verhindern.

Die maximale Anzahl speicherbarer Elemente ist in der XML-Datei mit der Verbindungskonfiguration definiert:

- Abschnitt <INTERNAL> ... </INTERNAL>
- Element <BUFFERING ... Limit="..."/>
 - Maximal mögliche Anzahl: 512
- Ist das Element BUFFERING nicht konfiguriert, gilt der Default-Wert:
 - <BUFFERING Mode="FIFO" Limit="16"/>

Konfigurations-datei Für jede Ethernet-Verbindung muss eine XML-Datei konfiguriert sein. Der Name der XML-Datei ist gleichzeitig der Zugriffsschlüssel in KRL.

Verzeichnis	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Datei	Beispiel: ... \EXT.XML → EKI_INIT("EXT")

Prüfanweisung ■ In der XML-Datei mit der Verbindungskonfiguration prüfen, auf welchen Wert die Anzahl speicherbarer Elemente begrenzt ist.

Lösung: Speicherlimit in XML-Datei erhöhen

Beschreibung Das Speicherlimit kann in der XML-Datei erhöht werden.

Voraussetzung ■ Benutzergruppe Experte
■ Betriebsart T1, T2 oder AUT

Vorgehensweise

1. XML-Datei wie benötigt ändern.
2. Wenn XML-Datei offline geändert wurde, sie in das vorgesehene Verzeichnis kopieren und die alte XML-Datei überschreiben.
3. Die Robotersteuerung neu starten, mit den Einstellungen **Kaltstart** und **Dateien neu einlesen**.

Ursache: Speicherlimit erreicht (Anzahl Datenelemente pro Speicher)

Beschreibung Die Anzahl speicherbarer Datenelemente ist begrenzt. Dieses Speicherlimit wurde erreicht. Die Ethernet-Verbindung wurde geschlossen, um den Empfang weiterer Daten zu verhindern.

Die maximale Anzahl speicherbarer Elemente ist in der XML-Datei mit der Verbindungskonfiguration definiert:

- Abschnitt <INTERNAL> ... </INTERNAL>
- Element <BUFFERING ... Limit="..."/>

- Maximal mögliche Anzahl: 512
- Ist das Element BUFFERING nicht konfiguriert, gilt der Default-Wert:
 - <BUFFERING Mode="FIFO" Limit="16"/>

Konfigurations-datei

Für jede Ethernet-Verbindung muss eine XML-Datei konfiguriert sein. Der Name der XML-Datei ist gleichzeitig der Zugriffsschlüssel in KRL.

Verzeichnis	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Datei	Beispiel: ... \EXT.XML → EKI_INIT("EXT")

Prüfanweisung

- In der XML-Datei mit der Verbindungskonfiguration prüfen, auf welchen Wert die Anzahl speicherbarer Elemente begrenzt ist.

Lösung: Datenspeicher zyklisch prüfen und vor Erreichen des Limits sperren

Beschreibung

Über die Programmierung kann verhindert werden, dass das Speicherlimit erreicht wird. Dazu prüft man zyklisch, wie viele Datenelemente sich aktuell im Datenspeicher befinden und sperrt den Speicher vor Erreichen des Limits vorübergehend.

Folgende Funktionen werden benötigt:

- `EKI_STATUS = EKI_CheckBuffer(CHAR[], CHAR[])`
Prüft, wie viele Daten sich noch im Speicher befinden. Der Speicher wird nicht verändert.
- `EKI_STATUS = EKI_Lock(CHAR[])`
Sperrt das Verarbeiten empfangener Daten. Die Daten können nicht mehr im Speicher abgelegt werden.
- `EKI_STATUS = EKI_Unlock(CHAR[])`
Entsperrt das Verarbeiten empfangener Daten. Die Daten werden wieder im Speicher abgelegt.

`EKI_STATUS` ist die globale Strukturvariable, in die die Rückgabewerte der Funktion geschrieben werden. Relevant für die Auswertung ist das Element `Buff` von `EKI_STATUS`.

`Buff` enthält folgenden Wert:

- Anzahl der Elemente, die sich nach einem Zugriff noch im Speicher befinden

Syntax

```
GLOBAL STRUC EKI_STATUS INT Buff, Read, Msg_No, BOOL Connected, INT Counter
```

Beispiel

```
...
DECL INT limit
DECL EKI_STATUS ret
...
limit = 16

ret = EKI_CheckBuffer("MyChannel", "MyData")

IF(ret.Buff >= limit - 1) THEN
  ret = EKI_Lock("MyChannel")
ELSE
  ret = EKI_Unlock("MyChannel")
ENDIF
```

Ursache: EKI: EKI Speicher wird nicht ausgelesen

Beschreibung Das externe System schreibt Daten in den EKI Speicher. Wenn diese Daten nicht über KRL-Befehle abgerufen werden, füllt sich der EKI Speicher. Um Datenverlust zu vermeiden, wird die Verbindung bei vollem EKI Speicher geschlossen. Das TCP Protokoll erkennt die geschlossene Verbindung und unterbricht den Datenaustausch. Die maximale Anzahl von Datenelementen wurde in den EKI Speicher geschrieben ohne diese auszulesen.

So kann man prüfen, ob der EKI Speicher ausgelesen wird:

- Prüfanweisung**
1. Programm wählen und **Öffnen** klicken.
 2. Prüfen, ob der KRL-Befehl **EKI_Get** vorhanden ist.

Lösung: Programm ändern

- Voraussetzung**
- Benutzergruppe Experte
 - Betriebsart T1, T2 oder AUT

- Vorgehensweise**
1. Programm im Navigator markieren und **Öffnen** drücken. Das Programm wird im Editor angezeigt.
 2. Die entsprechende Stelle im Programm suchen und bearbeiten.
 3. Datei schließen und die Sicherheitsabfrage, ob die Änderungen gespeichert werden sollen, mit **Ja** beantworten.

9.3.26 EKI01536

Meldungscode	■ EKI01536
Meldungstext	■ Empfangene Zeichenkette zu lang
Meldungstyp	■ Fehlermeldung
	
Auswirkung	■ Bahntreuer NOT-HALT ■ Die Eingabe von aktiven Kommandos (Roboterbewegungen, Programmstart) ist blockiert.
Mögliche Ursache(n)	■ Ursache: Vom externen System gesendete Zeichenfolge zu lang (>>> Seite 98) ■ Lösung: Programmierung auf externem System anpassen (>>> Seite 98)

Ursache: Vom externen System gesendete Zeichenfolge zu lang

Beschreibung Eine vom externen System gesendete Zeichenfolge überschreitet die maximal zulässige Länge. Bis zu 3600 Zeichen sind erlaubt.

- Prüfanweisung**
- Länge der vom externen System gesendeten Daten überprüfen.

Lösung: Programmierung auf externem System anpassen

Beschreibung Die Programmierung auf dem externen System muss so angepasst werden, dass keine zu langen Zeichenfolgen mehr gesendet werden.

9.3.27 EKI01792

Meldungscode	■ EKI01792
Meldungstext	■ Limit Empfangsspeicher erreicht
Meldungstyp	■ Fehlermeldung
	
Auswirkung	■ Bahntreuer NOT-HALT ■ Die Eingabe von aktiven Kommandos (Roboterbewegungen, Programmstart) ist blockiert.
Mögliche Ursache(n)	■ Ursache: Speicherlimit erreicht (Anzahl Bytes Gesamtspeicher) (>>> Seite 99) ■ Lösung: Speicherlimit in XML-Datei erhöhen (>>> Seite 99)

Ursache: Speicherlimit erreicht (Anzahl Bytes Gesamtspeicher)

Beschreibung Die Anzahl speicherbarer Bytes ist begrenzt. Dieses Speicherlimit wurde erreicht. Die Ethernet-Verbindung wurde geschlossen, um den Empfang weiterer Daten zu verhindern.

Die maximale Anzahl speicherbarer Bytes ist in der XML-Datei mit der Verbindungskonfiguration definiert:

- Abschnitt <INTERNAL> ... </INTERNAL>
- Element <BUFFSIZE Limit="..."/>
 - Maximal mögliche Anzahl: 65534
- Ist das Element BUFFSIZE nicht konfiguriert, gilt der Default-Wert:
 - <BUFFSIZE Limit="16384"/>

Konfigurations-datei Für jede Ethernet-Verbindung muss eine XML-Datei konfiguriert sein. Der Name der XML-Datei ist gleichzeitig der Zugriffsschlüssel in KRL.

Verzeichnis	C:\KRC\ROBOTER\Config\User\Common\EthernetKRL
Datei	Beispiel: ... \EXT.XML —> EKI_INIT("EXT")

Prüfanweisung ■ In der XML-Datei mit der Verbindungskonfiguration prüfen, auf welchen Wert die Byte-Anzahl begrenzt ist.

Lösung: Speicherlimit in XML-Datei erhöhen

Beschreibung Das Speicherlimit kann in der XML-Datei erhöht werden.

Voraussetzung

- Benutzergruppe Experte
- Betriebsart T1, T2 oder AUT

Vorgehensweise

1. XML-Datei wie benötigt ändern.
2. Wenn XML-Datei offline geändert wurde, sie in das vorgesehene Verzeichnis kopieren und die alte XML-Datei überschreiben.
3. Die Robotersteuerung neu starten, mit den Einstellungen **Kaltstart** und **Dateien neu einlesen**.

9.3.28 EKI02048

Meldungscode	■ EKI02048
Meldungstext	■ Server Zeitgrenze erreicht
Meldungstyp	■ Fehlermeldung
	
Auswirkung	■ Bahntreuer NOT-HALT ■ Die Eingabe von aktiven Kommandos (Roboterbewegungen, Programmstart) ist blockiert.
Mögliche Ursache(n)	■ Ursache: Software-Fehler auf externem System führt zu Timeout (>>> Seite 100) Lösung: Fehler in Software des externen Systems beheben (>>> Seite 100)

Ursache: Software-Fehler auf externem System führt zu Timeout

Beschreibung EKI ist als Server konfiguriert und wartet auf die Verbindungsanfrage des externen Systems.

In der XML-Datei mit der Verbindungskonfiguration ist ein Wert für <TIMEOUT Connect="..."/> angegeben. Diese Zeit ist abgelaufen, ohne dass sich das externe System mit dem Server verbunden hat. Die Ursache ist ein Fehler in der Software des externen Systems.

Prüfanweisung ■ Externes System auf Software-Fehler überprüfen.

Lösung: Fehler in Software des externen Systems beheben

Beschreibung Der Fehler in der Software des externen Systems muss behoben werden.

10 Anhang

10.1 Erweiterte XML-Struktur für Verbindungseigenschaften



Die erweiterte XML-Struktur darf nur nach Rücksprache mit der KUKA Roboter GmbH verwendet werden. (>>> 11 "KUKA Service" Seite 113)

Beschreibung Im Abschnitt <INTERNAL> ... </INTERNAL> der Konfigurationsdatei können weitere Eigenschaften für das EKI konfiguriert werden:

Element	Attribut	Beschreibung
TIMEOUT	Receive	Zeit, nach der der Versuch Daten zu empfangen abgebrochen wird (optional) ■ 0 ... 65 534 ms Default-Wert: 0 ms
	Send	Zeit, nach der der Versuch Daten zu senden abgebrochen wird (optional) ■ 0 ... 65 534 ms Default-Wert: 0 ms
BUFFSIZE	Receive	Größe des verwendeten Sockets beim Datenempfang (optional) ■ 1 ... 65 534 Bytes Default-Wert: Vom System vorgegeben
	Send	Größe des verwendeten Sockets beim Datenversand (optional) ■ 1 ... 65 534 Bytes Default-Wert: Vom System vorgegeben

10.2 Speicher erhöhen



Der Speicher darf nur nach Rücksprache mit der KUKA Roboter GmbH erhöht werden. (>>> 11 "KUKA Service" Seite 113)

Beschreibung Wenn der zur Verfügung stehende Speicher nicht ausreicht, wird empfohlen die Programmierweise in KRL und die Konfiguration zu überprüfen.

- Überprüfen, ob eine Verbindung so konfiguriert ist, dass der Speicher vollständig mit empfangenen Daten belegt wird.
- Überprüfen, ob mehrere Verbindungen mit hohem Datenaufkommen definiert und aktiviert sind.

Voraussetzung

- Benutzergruppe Experte
- Betriebsart T1, T2 oder AUT

Vorgehensweise

1. Im Verzeichnis C:\KRC\ROBOTER\Config\User\Common die Datei EthernetKRL.XML öffnen.
2. Im Abschnitt <EthernetKRL> im Element <MemSize> die gewünschte Speicherkapazität in Byte eintragen.

```
<EthernetKRL>
  <Interface>
```

```
<MemSize>1048576</MemSize>
...
</EthernetKRL>
```

3. Datei schließen und die Sicherheitsabfrage, ob die Änderungen gespeichert werden sollen, mit **Ja** beantworten.
4. Die Robotersteuerung neu starten, mit den Einstellungen **Kaltstart** und **Dateien neu einlesen**.

10.3 Meldungsausgabe und Loggen von Meldungen deaktivieren

Beschreibung In folgenden Fällen wird empfohlen, die automatische Meldungsausgabe auf der smartHMI zu deaktivieren:

- Es treten Laufzeitfehler auf.
- EKI wird im Submit-Interpreter verwendet.

Wenn die automatische Meldungsausgabe deaktiviert ist, werden defaultmäßig weiterhin alle Fehlermeldungen protokolliert. Wenn diese Fehler oder Warnungen bewusst ignoriert werden sollen, z. B. weil das Loggen der Meldungen zu einer hohen Systembelastung und -verlangsamung führt, kann dieser Mechanismus ebenfalls deaktiviert werden.

Voraussetzung ■ Benutzergruppe Experte

Vorgehensweise

1. Im Verzeichnis C:\KRC\ROBOTER\Config\User\Common\EthernetKRL der Robotersteuerung die Konfigurationsdatei der Ethernet-Verbindung öffnen.
2. Im Abschnitt <INTERNAL> ... </INTERNAL> der XML-Datei folgende Zeile eintragen:

```
<MESSAGES Logging="disabled" Display="disabled"/>
```

3. Änderung speichern und Datei schließen.



Wenn die automatische Meldungsausgabe deaktiviert ist, kann die Funktion EKI_CHECK() verwendet werden, um einzelne EKI-Anweisungen auf Fehler zu überprüfen.

10.4 Befehlsreferenz

10.4.1 Verbindung initialisieren, öffnen, schließen und löschen

RET = EKI_Init(CHAR[])	
Funktion	Initialisiert einen Kanal für die Ethernet-Kommunikation Folgende Aktionen werden ausgeführt: ■ Einlesen der Verbindungskonfiguration ■ Erstellen der Datenspeicher ■ Vorbereiten der Ethernet-Verbindung
Parameter	Typ: CHAR Name des Kanals (= Name der XML-Datei mit der Verbindungskonfiguration)
RET	Typ: EKI_STATUS Rückgabewerte der Funktion
Beispiel	RET = EKI_Init("Channel_1")

RET = EKI_Open(CHAR[])	
Funktion	Öffnet einen initialisierten Kanal Wenn EKI als Client konfiguriert ist, verbindet sich das EKI mit dem externen System (= Server). Wenn EKI als Server konfiguriert ist, wartet das EKI auf die Verbindungsanfrage des externen Systems (= Client).
Parameter	Typ: CHAR Name des Kanals
RET	Typ: EKI_STATUS Name des Kanals (= Name der XML-Datei mit der Verbindungskonfiguration)
Beispiel	RET = EKI_Open("Channel_1")

RET = EKI_Close(CHAR[])	
Funktion	Schließt einen geöffneten Kanal
Parameter	Typ: CHAR Name des Kanals (= Name der XML-Datei mit der Verbindungskonfiguration)
RET	Typ: EKI_STATUS Rückgabewerte der Funktion
Beispiel	RET = EKI_Close("Channel_1")

RET = EKI_Clear(CHAR[])	
Funktion	Löscht einen Kanal sowie alle zugehörigen Datenspeicher und beendet die Ethernet-Verbindung.
Parameter	Typ: CHAR Name des Kanals (= Name der XML-Datei mit der Verbindungskonfiguration)
RET	Typ: EKI_STATUS Rückgabewerte der Funktion
Beispiel	RET = EKI_Clear("Channel_1")

Informationen zu den Rückgabewerten der jeweiligen Funktion sind hier zu finden: (>>> "Rückgabewerte" Seite 42)

10.4.2 Daten senden

RET = EKI_Send(CHAR[], CHAR[], INT)	
Funktion	Sendet Daten über einen Kanal (>>> 6.2.4 "Senden von Daten" Seite 36)
Parameter 1	Typ: CHAR Name des geöffneten Kanals, über den gesendet werden soll

RET = EKI_Send(CHAR[], CHAR[], INT)	
Parameter 2	Typ: CHAR Definiert den Umfang der zu sendenden Daten. Bei Kommunikation über XML-Struktur: ■ XPath-Ausdruck des zu sendenden Elements aus der konfigurierten XML-Struktur für den Datenversand. Wird nur das Wurzelement angegeben, wird die gesamte XML-Struktur übertragen (siehe Beispiel 1). Soll nur ein Teil der XML-Struktur übertragen werden, d. h. das Element einer untergeordneten Ebene, muss ausgehend vom Wurzelement der Weg bis zu diesem Element angegeben werden (siehe Beispiel 2). ■ Alternativ: Beliebige Zeichenkette variabler Länge, die übertragen werden soll Bei Kommunikation über Rohdaten: ■ Beliebige Zeichenkette, die übertragen werden soll: ■ Bei Binärdatensätzen mit fester Länge (Attribut <code>Type = "BYTE"</code>): Beliebige Zeichenkette mit fester Länge Die Größe der Zeichenkette in Bytes muss genau dem konfigurierten Attribut <code>Size</code> entsprechen. Bei Überschreitung wird eine Fehlermeldung, bei Unterschreitung eine Warnmeldung ausgegeben. ■ Bei Binärdatensätzen mit variabler Endzeichenfolge (Attribut <code>Type = "STREAM"</code>): Beliebige Zeichenkette mit variabler Länge Die Endzeichenfolge wird automatisch mitgesendet. Hinweis: Enthält eine beliebige Zeichenkette variabler Länge ein ASCII NULL-Zeichen, wird nur der Teil bis zu diesem Zeichen übertragen. Ist dies nicht gewünscht, muss Parameter 3 passend definiert werden.
Parameter 3 (optional)	Typ: INT Nur relevant beim Senden von beliebigen Zeichenketten, deren Länge variabel ist (siehe Parameter 2): Anzahl der maximal zu sendenden Zeichen ASCII NULL-Zeichen werden ignoriert. Ist die Zeichenkette zu lang, wird diese abgeschnitten.
RET	Typ: EKI_STATUS Rückgabewerte der Funktion
Beispiel 1	RET = EKI_Send("Channel_1", "Root")
Beispiel 2	RET = EKI_Send("Channel_1", "Root/Test")
Beispiel 3	RET = EKI_Send("Channel_1", MyBytes[])
Beispiel 4	RET = EKI_Send("Channel_1", MyBytes[], 6)

Informationen zu den Rückgabewerten der jeweiligen Funktion sind hier zu finden: (>>> "Rückgabewerte" Seite 42)

10.4.3 Daten schreiben

RET = EKI_SetBool(CHAR[], CHAR[], BOOL)	
Funktion	Schreibt einen booleschen Wert in einen Speicher
Parameter 1	Typ: CHAR Name des geöffneten Kanals
Parameter 2	Typ: CHAR Name der Position in der XML-Struktur

RET = EKI_SetBool(CHAR[], CHAR[], BOOL)	
Parameter 3	Typ: BOOL Wert, der in den Speicher geschrieben wird
RET	Typ: EKI_STATUS Rückgabewerte der Funktion
Beispiel	RET = EKI_SetBool("Channel_1", "Root/Activ", true)

RET = EKI_SetFrame(CHAR[], CHAR[], FRAME)	
Funktion	Schreibt einen Wert vom Typ FRAME in einen Speicher
Parameter 1	Typ: CHAR Name des geöffneten Kanals
Parameter 2	Typ: CHAR Name der Position in der XML-Struktur
Parameter 3	Typ: FRAME Wert, der in den Speicher geschrieben wird
RET	Typ: EKI_STATUS Rückgabewerte der Funktion
Beispiel	RET= EKI_SetFrame("Channel_1", "Root/BASE", {X 0.0, Y 0.0, Z 0.0, A 0.0, B 0.0, C 0.0})

RET = EKI_SetInt(CHAR[], CHAR[], INT)	
Funktion	Schreibt einen ganzzahligen Wert in einen Speicher
Parameter 1	Typ: CHAR Name des geöffneten Kanals
Parameter 2	Typ: CHAR Name der Position in der XML-Struktur
Parameter 3	Typ: INT Wert, der in den Speicher geschrieben wird
RET	Typ: EKI_STATUS Rückgabewerte der Funktion
Beispiel	RET = EKI_SetInt("Channel_1", "Root/List", 67234)

RET = EKI_SetReal(CHAR[], CHAR[], REAL)	
Funktion	Schreibt einen Gleitkommawert in einen Speicher
Parameter 1	Typ: CHAR Name des geöffneten Kanals
Parameter 2	Typ: CHAR Name der Position in der XML-Struktur
Parameter 3	Typ: REAL Wert, der in den Speicher geschrieben wird
RET	Typ: EKI_STATUS Rückgabewerte der Funktion
Beispiel	RET = EKI_SetReal("Channel_1", "Root/Number", 1.234)

RET = EKI_SetString(CHAR[], CHAR[], CHAR[])	
Funktion	Schreibt eine Zeichenfolge in einen Speicher
Parameter 1	Typ: CHAR Name des geöffneten Kanals
Parameter 2	Typ: CHAR Name der Position in der XML-Struktur
Parameter 3	Typ: CHAR Zeichenfolge, die in den Speicher geschrieben wird Maximale Zeichenanzahl: ■ 3 600
RET	Typ: EKI_STATUS Rückgabewerte der Funktion
Beispiel	RET = EKI_SetString("Channel_1", "Root/Message", "Hello")

Informationen zu den Rückgabewerten der jeweiligen Funktion sind hier zu finden: (>>> "Rückgabewerte" Seite 42)

10.4.4 Daten auslesen

RET = EKI_GetBool(CHAR[], CHAR[], BOOL)	
Funktion	Liest einen booleschen Wert aus einem Speicher
Parameter 1	Typ: CHAR Name des geöffneten Kanals
Parameter 2	Typ: CHAR Name der Position in der XML-Struktur
Parameter 3	Typ: BOOL Wert, der aus dem Speicher gelesen wird
RET	Typ: EKI_STATUS Rückgabewerte der Funktion
Beispiel	RET = EKI_GetBool("Channel_1", "Root/Activ", MyBool)

RET = EKI_GetBoolArray(CHAR[], CHAR[], BOOL[])	
Funktion	Liest einen booleschen Wert aus einem Speicher und kopiert den Wert in das vom KRL-Programm übergebene Feld Es werden so lange Werte gelesen, bis das Feld voll ist oder kein Element mehr vorhanden ist.
Parameter 1	Typ: CHAR Name des geöffneten Kanals
Parameter 2	Typ: CHAR Name der Position in der XML-Struktur
Parameter 3	Typ: BOOL Feld, das aus dem Speicher gelesen wird Maximale Anzahl lesbarer Feldelemente: ■ 512

RET = EKI_GetBoolArray(Char[], Char[], Bool[])	
RET	Typ: EKI_STATUS Rückgabewerte der Funktion
Beispiel	RET = EKI_GetBoolArray("Channel_1", "Root/Activ", MyBool[])

RET = EKI_GetInt(Char[], Char[], Int)	
Funktion	Liest einen ganzzahligen Wert aus einem Speicher
Parameter 1	Typ: CHAR Name des geöffneten Kanals
Parameter 2	Typ: CHAR Name der Position in der XML-Struktur
Parameter 3	Typ: INT Wert, der aus dem Speicher gelesen wird
RET	Typ: EKI_STATUS Rückgabewerte der Funktion
Beispiel	RET = EKI_GetInt("Channel_1", "Root/Numbers/One", MyInteger)

RET = EKI_GetIntArray(Char[], Char[], Int[])	
Funktion	Liest einen ganzzahligen Wert aus einem Speicher und kopiert den Wert in das vom KRL-Programm übergebene Feld Es werden so lange Werte gelesen, bis das Feld voll ist oder kein Element mehr vorhanden ist.
Parameter 1	Typ: CHAR Name des geöffneten Kanals
Parameter 2	Typ: CHAR Name der Position in der XML-Struktur
Parameter 3	Typ: INT Feld, das aus dem Speicher gelesen wird Maximale Anzahl lesbarer Feldelemente: ■ 512
RET	Typ: EKI_STATUS Rückgabewerte der Funktion
Beispiel	RET = EKI_GetIntArray("Channel_1", "Root/Numbers/One", MyInteger[])

RET = EKI_GetReal(Char[], Char[], Real)	
Funktion	Liest einen Gleitkommawert aus einem Speicher
Parameter 1	Typ: CHAR Name des geöffneten Kanals
Parameter 2	Typ: CHAR Name der Position in der XML-Struktur
Parameter 3	Typ: REAL Wert, der aus dem Speicher gelesen wird

RET = EKI_GetReal(CHAR[], CHAR[], REAL)

RET	Typ: EKI_STATUS Rückgabewerte der Funktion
Beispiel	RET = EKI_GetReal("Channel_1", "Root/Position", MyReal)

RET = EKI_GetRealArray(CHAR[], CHAR[], REAL[])

Funktion	Liest einen Gleitkommawert aus einem Speicher und kopiert den Wert in das vom KRL-Programm übergebene Feld Es werden so lange Werte gelesen, bis das Feld voll ist oder kein Element mehr vorhanden ist.
Parameter 1	Typ: CHAR Name des geöffneten Kanals
Parameter 2	Typ: CHAR Name der Position in der XML-Struktur
Parameter 3	Typ: REAL Feld, das aus dem Speicher gelesen wird Maximale Anzahl lesbarer Feldelemente: ■ 512
RET	Typ: EKI_STATUS Rückgabewerte der Funktion
Beispiel	RET = EKI_GetRealArray("Channel_1", "Root/Position", MyReal[])

RET = EKI_GetString(CHAR[], CHAR[], CHAR[])

Funktion	Liest eine Zeichenfolge aus einem Speicher
Parameter 1	Typ: CHAR Name des geöffneten Kanals
Parameter 2	Typ: CHAR Name der Position in der XML-Struktur oder Name des Elements in den Rohdaten
Parameter 3	Typ: CHAR Zeichenfolge, die aus dem Speicher gelesen wird Maximale Zeichenanzahl: ■ 3 600
RET	Typ: EKI_STATUS Rückgabewerte der Funktion
XML-Beispiel	RET = EKI_GetString("Channel_1", "Root/Message", MyChars[])
Binär-Beispiel	RET = EKI_GetString("Channel_1", "Streams", MyStream[])

RET = EKI_GetFrame(CHAR[], CHAR[], FRAME)	
Funktion	Liest einen Wert vom Typ FRAME aus einem Speicher
Parameter 1	Typ: CHAR Name des geöffneten Kanals
Parameter 2	Typ: CHAR Name der Position in der XML-Struktur
Parameter 3	Typ: FRAME Wert, der aus dem Speicher gelesen wird
RET	Typ: EKI_STATUS Rückgabewerte der Funktion
Beispiel	RET = EKI_GetFrame("Channel_1", "Root/TCP", MyFrame)

RET = EKI_GetFrameArray(CHAR[], CHAR[], FRAME[])	
Funktion	Liest einen Wert vom Typ FRAME aus einem Speicher und kopiert den Wert in das vom KRL-Programm übergebene Feld Es werden so lange Werte gelesen, bis das Feld voll ist oder kein Element mehr vorhanden ist.
Parameter 1	Typ: CHAR Name des geöffneten Kanals
Parameter 2	Typ: CHAR Name der Position in der XML-Struktur
Parameter 3	Typ: FRAME Feld, das aus dem Speicher gelesen wird Maximale Anzahl lesbarer Feldelemente: ■ 512
RET	Typ: EKI_STATUS Rückgabewerte der Funktion
Beispiel	RET = EKI_GetFrameArray("Channel_1", "Root/TCP", MyFrame[])

Informationen zu den Rückgabewerten der jeweiligen Funktion sind hier zu finden: (>>> "Rückgabewerte" Seite 42)

10.4.5 Funktion auf Fehler prüfen

EKI_CHECK(EKI_STATUS, EKrlMsgType, CHAR[])	
Funktion	Prüft, ob beim Ausführen einer Funktion ein Fehler aufgetreten ist: ■ Die Fehlernummer wird ausgelesen und die zugehörige Meldung auf der smartHMI ausgegeben. (siehe Parameter 1) ■ Optional: Wird der Kanalname angegeben, wird beim Datenempfang abgefragt, ob Fehler vorliegen (siehe Parameter 3)
Parameter 1	EKI_STATUS Rückgabewerte der geprüften Funktion

EKI_CHECK(EKI_STATUS, EKrlMsgType, CHAR[])	
Parameter 2	Typ: ENUM Meldungstyp, der auf der smartHMI ausgegeben wird: ■ #NOTIFY: Hinweismeldung ■ #STATE: Zustandsmeldung ■ #QUIT: Quittiermeldung ■ #WAITING: Wartemeldung
Parameter 3 (optional)	Typ: CHAR Name des geöffneten Kanals, der geprüft werden soll
Beispiel 1	EKI_CHECK(RET,#QUIT)
Beispiel 2	EKI_CHECK(RET,#NOTIFY,"MyChannelName")

Informationen zu den Rückgabewerten der jeweiligen Funktion sind hier zu finden: (>>> "Rückgabewerte" Seite 42)

10.4.6 Datenspeicher löschen, sperren, entsperren und prüfen

RET = EKl_Clear(CHAR[])	
Funktion	Löscht einen Kanal sowie alle zugehörigen Datenspeicher und beendet die Ethernet-Verbindung.
Parameter	Typ: CHAR Name des Kanals (= Name der XML-Datei mit der Verbindungskonfiguration)
RET	Typ: EKl_STATUS Rückgabewerte der Funktion
Beispiel	RET = EKl_Clear("Channel_1")

RET = EKl_ClearBuffer(CHAR[], CHAR[])	
Funktion	Löscht die definierten Datenspeicher, ohne die Ethernet-Verbindung zu beenden (>>> 6.2.6 "Löschen von Datenspeichern" Seite 41)
Parameter 1	Typ: CHAR Name des Kanals
Parameter 2	Typ: CHAR Definiert die zu löschenden Datenspeicher Bei Kommunikation über Rohdaten: ■ Name der zu löschenden Rohdaten aus der konfigurierten XML-Struktur für den Datenempfang Bei Kommunikation über XML-Struktur: ■ XPATH-Ausdruck des Elements aus der konfigurierten XML-Struktur für den Datenempfang oder der XML-Struktur für den Datenversand, dessen Datenspeicher gelöscht werden soll
RET	Typ: EKl_STATUS Rückgabewerte der Funktion
Beispiel 1	RET = EKl_ClearBuffer("Channel_1", "RawData")
Beispiel 2	RET = EKl_ClearBuffer("Channel_1", "Ext/Activ/Flag")

RET = EKI_Lock(CHAR[])	
Funktion	Sperrt das Verarbeiten empfangener Daten. Die Daten können nicht mehr im Speicher abgelegt werden.
Parameter	Typ: CHAR Name des Kanals
RET	Typ: EKI_STATUS Rückgabewerte der Funktion

RET = EKI_Unlock(CHAR[])	
Funktion	Entsperrt das Verarbeiten empfangener Daten. Die Daten werden wieder im Speicher abgelegt.
Parameter	Typ: CHAR Name des Kanals
RET	Typ: EKI_STATUS Rückgabewerte der Funktion

RET = EKI_CheckBuffer(CHAR[], CHAR[])	
Funktion	Prüft, wie viele Daten sich noch im Speicher befinden. Der Speicher wird nicht verändert. Zusätzlich wird der Zeitstempel des Datenelements zurückgegeben, das als nächstes zur Entnahme aus dem Speicher bereitsteht.
Parameter 1	Typ: CHAR Name des Kanals
Parameter 2	Typ: CHAR Definiert den zu prüfenden Datenspeicher Bei Kommunikation über Rohdaten: ■ Name der zu prüfenden Rohdaten aus der konfigurierten XML-Struktur für den Datenempfang Bei Kommunikation über XML-Struktur: ■ XPATH-Ausdruck des Elements aus der konfigurierten XML-Struktur für den Datenempfang oder der XML-Struktur für den Datenversand, dessen Datenspeicher geprüft werden soll
RET	Typ: EKI_STATUS Rückgabewerte der Funktion
Beispiel 1	RET = EKI_CheckBuffer("Channel_1", "RawData")
Beispiel 2	RET = EKI_CheckBuffer("Channel_1", "Ext/Activ/Flag")

Informationen zu den Rückgabewerten der jeweiligen Funktion sind hier zu finden: (>>> "Rückgabewerte" Seite 42)

11 KUKA Service

11.1 Support-Anfrage

Einleitung Diese Dokumentation bietet Informationen zu Betrieb und Bedienung und unterstützt Sie bei der Behebung von Störungen. Für weitere Anfragen steht Ihnen die lokale Niederlassung zur Verfügung.

Informationen Zur **Abwicklung einer Anfrage werden folgende Informationen benötigt:**

- Problembeschreibung inkl. Angaben zu Dauer und Häufigkeit der Störung
- Möglichst umfassende Informationen zu den Hardware- und Software-Komponenten des Gesamtsystems

Die folgende Liste gibt Anhaltspunkte, welche Informationen häufig relevant sind:

- Typ und Seriennummer der Kinematik, z. B. des Manipulators
- Typ und Seriennummer der Steuerung
- Typ und Seriennummer der Energiezuführung
- Bezeichnung und Version der System Software
- Bezeichnungen und Versionen weiterer/anderer Software-Komponenten oder Modifikationen
- Diagnosepaket KRCDiag

Für KUKA Sunrise zusätzlich: Vorhandene Projekte inklusive Applikationen

Für Versionen der KUKA System Software älter als V8: Archiv der Software (KRCDiag steht hier noch nicht zur Verfügung.)

- Vorhandene Applikation
- Vorhandene Zusatzachsen

11.2 KUKA Customer Support

Verfügbarkeit Der KUKA Customer Support ist in vielen Ländern verfügbar. Bei Fragen stehen wir gerne zur Verfügung.

Argentinien Ruben Costantini S.A. (Agentur)
Luis Angel Huergo 13 20
Parque Industrial
2400 San Francisco (CBA)
Argentinien
Tel. +54 3564 421033
Fax +54 3564 428877
ventas@costantini-sa.com

Australien KUKA Robotics Australia Pty Ltd
45 Fennell Street
Port Melbourne VIC 3207
Australien
Tel. +61 3 9939 9656
info@kuka-robotics.com.au
www.kuka-robotics.com.au

Belgien	KUKA Automatisering + Robots N.V. Centrum Zuid 1031 3530 Houthalen Belgien Tel. +32 11 516160 Fax +32 11 526794 info@kuka.be www.kuka.be
Brasilien	KUKA Roboter do Brasil Ltda. Travessa Claudio Armando, nº 171 Bloco 5 - Galpões 51/52 Bairro Assunção CEP 09861-7630 São Bernardo do Campo - SP Brasilien Tel. +55 11 4942-8299 Fax +55 11 2201-7883 info@kuka-roboter.com.br www.kuka-roboter.com.br
Chile	Robotec S.A. (Agency) Santiago de Chile Chile Tel. +56 2 331-5951 Fax +56 2 331-5952 robotec@robotec.cl www.robotec.cl
China	KUKA Robotics China Co., Ltd. No. 889 Kungang Road Xiaokunshan Town Songjiang District 201614 Shanghai P. R. China Tel. +86 21 5707 2688 Fax +86 21 5707 2603 info@kuka-robotics.cn www.kuka-robotics.com
Deutschland	KUKA Roboter GmbH Zugspitzstr. 140 86165 Augsburg Deutschland Tel. +49 821 797-1926 Fax +49 821 797-41 1926 Hotline.robotics.de@kuka.com www.kuka-roboter.de

Frankreich KUKA Automatismes + Robotique SAS
 Techvallée
 6, Avenue du Parc
 91140 Villebon S/Yvette
 Frankreich
 Tel. +33 1 6931660-0
 Fax +33 1 6931660-1
commercial@kuka.fr
www.kuka.fr

Indien KUKA Robotics India Pvt. Ltd.
 Office Number-7, German Centre,
 Level 12, Building No. - 9B
 DLF Cyber City Phase III
 122 002 Gurgaon
 Haryana
 Indien
 Tel. +91 124 4635774
 Fax +91 124 4635773
info@kuka.in
www.kuka.in

Italien KUKA Roboter Italia S.p.A.
 Via Pavia 9/a - int.6
 10098 Rivoli (TO)
 Italien
 Tel. +39 011 959-5013
 Fax +39 011 959-5141
kuka@kuka.it
www.kuka.it

Japan KUKA Robotics Japan K.K.
 YBP Technical Center
 134 Godo-cho, Hodogaya-ku
 Yokohama, Kanagawa
 240 0005
 Japan
 Tel. +81 45 744 7691
 Fax +81 45 744 7696
info@kuka.co.jp

Kanada KUKA Robotics Canada Ltd.
 6710 Maritz Drive - Unit 4
 Mississauga
 L5W 0A1
 Ontario
 Kanada
 Tel. +1 905 670-8600
 Fax +1 905 670-8604
info@kukarobotics.com
www.kuka-robotics.com/canada

Korea	KUKA Robotics Korea Co. Ltd. RIT Center 306, Gyeonggi Technopark 1271-11 Sa 3-dong, Sangnok-gu Ansan City, Gyeonggi Do 426-901 Korea Tel. +82 31 501-1451 Fax +82 31 501-1461 info@kukakorea.com
Malaysia	KUKA Robot Automation (M) Sdn Bhd South East Asia Regional Office No. 7, Jalan TPP 6/6 Taman Perindustrian Puchong 47100 Puchong Selangor Malaysia Tel. +60 (03) 8063-1792 Fax +60 (03) 8060-7386 info@kuka.com.my
Mexiko	KUKA de México S. de R.L. de C.V. Progreso #8 Col. Centro Industrial Puente de Vigas Tlalnepantla de Baz 54020 Estado de México Mexiko Tel. +52 55 5203-8407 Fax +52 55 5203-8148 info@kuka.com.mx www.kuka-robotics.com/mexico
Norwegen	KUKA Sveiseanlegg + Roboter Sentrumsvegen 5 2867 Hov Norwegen Tel. +47 61 18 91 30 Fax +47 61 18 62 00 info@kuka.no
Österreich	KUKA Roboter CEE GmbH Gruberstraße 2-4 4020 Linz Österreich Tel. +43 7 32 78 47 52 Fax +43 7 32 79 38 80 office@kuka-roboter.at www.kuka.at

Polen
KUKA Roboter CEE GmbH Poland
Spółka z ograniczoną odpowiedzialnością
Oddział w Polsce
Ul. Porcelanowa 10
40-246 Katowice
Polen
Tel. +48 327 30 32 13 or -14
Fax +48 327 30 32 26
ServicePL@kuka-roboter.de

Portugal
KUKA Robots IBÉRICA, S.A.
Rua do Alto da Guerra n° 50
Armazém 04
2910 011 Setúbal
Portugal
Tel. +351 265 729 780
Fax +351 265 729 782
info.portugal@kukapt.com
www.kuka.com

Russland
KUKA Robotics RUS
Werbnaja ul. 8A
107143 Moskau
Russland
Tel. +7 495 781-31-20
Fax +7 495 781-31-19
info@kuka-robotics.ru
www.kuka-robotics.ru

Schweden
KUKA Svetsanläggningar + Robotar AB
A. Odhners gata 15
421 30 Västra Frölunda
Schweden
Tel. +46 31 7266-200
Fax +46 31 7266-201
info@kuka.se

Schweiz
KUKA Roboter Schweiz AG
Industriestr. 9
5432 Neuenhof
Schweiz
Tel. +41 44 74490-90
Fax +41 44 74490-91
info@kuka-roboter.ch
www.kuka-roboter.ch

Spanien	KUKA Robots IBÉRICA, S.A. Pol. Industrial Torrent de la Pastera Carrer del Bages s/n 08800 Vilanova i la Geltrú (Barcelona) Spanien Tel. +34 93 8142-353 Fax +34 93 8142-950 comercial@kukarob.es www.kuka.es
Südafrika	Jendamark Automation LTD (Agentur) 76a York Road North End 6000 Port Elizabeth Südafrika Tel. +27 41 391 4700 Fax +27 41 373 3869 www.jendamark.co.za
Taiwan	KUKA Robot Automation Taiwan Co., Ltd. No. 249 Pujong Road Jungli City, Taoyuan County 320 Taiwan, R. O. C. Tel. +886 3 4331988 Fax +886 3 4331948 info@kuka.com.tw www.kuka.com.tw
Thailand	KUKA Robot Automation (M)SdnBhd Thailand Office c/o Maccall System Co. Ltd. 49/9-10 Soi Kingkaew 30 Kingkaew Road Tt. Rachatheva, A. Bangpli Samutprakarn 10540 Thailand Tel. +66 2 7502737 Fax +66 2 6612355 atika@ji-net.com www.kuka-roboter.de
Tschechien	KUKA Roboter Austria GmbH Organisation Tschechien und Slowakei Sezemická 2757/2 193 00 Praha Horní Počernice Tschechische Republik Tel. +420 22 62 12 27 2 Fax +420 22 62 12 27 0 support@kuka.cz

Ungarn KUKA Robotics Hungaria Kft.
Fő út 140
2335 Taksony
Ungarn
Tel. +36 24 501609
Fax +36 24 477031
info@kuka-robotics.hu

USA KUKA Robotics Corporation
51870 Shelby Parkway
Shelby Township
48315-1787
Michigan
USA
Tel. +1 866 873-5852
Fax +1 866 329-5852
info@kukarobotics.com
www.kukarobotics.com

Vereinigtes Königreich KUKA Robotics UK Ltd
Great Western Street
Wednesbury West Midlands
WS10 7LL
Vereinigtes Königreich
Tel. +44 121 505 9970
Fax +44 121 505 6589
service@kuka-robotics.co.uk
www.kuka-robotics.co.uk

Index

A

Anhang 101

B

Befehlsreferenz 102
Begriffe, verwendet 8
Beispiele, einbinden 47
Beispielkonfigurationen 47
Beispielprogramme 47
Bestimmungsgemäße Verwendung 16

C

CAST_FROM() 39, 51
CAST_TO() 37, 51
Client-Betrieb 14, 36

D

Datenaustausch 12
Datenspeicherung 13
Datenstrom 8
Defragmentierung 43
Deinstallation, Ethernet KRL 19, 21
Diagnose 59
Diagnosedaten, anzeigen 59
Diagnosemonitor (Menüpunkt) 59
Dokumentation, Industrieroboter 7

E

Eigenschaften 11
Einleitung 7
EKI 8
EKI_CHECK() 42, 44, 109
EKI_CheckBuffer() 34, 111
EKI_Clear() 41, 103, 110
EKI_ClearBuffer() 41, 110
EKI_Close() 36, 103
EKI_GetBool() 106
EKI_GetBoolArray() 106
EKI_GetFrame() 109
EKI_GetFrameArray() 109
EKI_GetInt() 107
EKI_GetIntArray() 107
EKI_GetReal() 107
EKI_GetRealArray() 108
EKI_GetString() 39, 108
EKI_Init() 34
EKI_Lock() 111
EKI_Open() 36, 68, 69, 103
EKI_Send() 36, 78, 80, 83, 103
EKI_SetBool() 104
EKI_SetFrame() 105
EKI_SetInt() 105
EKI_SetReal() 105
EKI_SetString() 106
EKI_STATUS 42
EKI_Unlock() 111
Endzeichenfolge 8
EOS 8

Ereignismeldungen 15, 43
Ethernet 8
Ethernet KRL, Übersicht 11
Ethernet-Verbindung, Konfiguration 11, 25
Ethernet, Schnittstellen 23
EthernetKRL_Server.exe 47

F

Fehlerbehandlung 15
Fehlerprotokoll 61
Fehlerreaktion, programmieren 44
FIFO 8, 13
Fragmentierung 43
Funktionen 11
Funktionen, Übersicht 32

H

Hardware 19
Hinweise 7

I

Installation 19
Installation, über smartHMI 20
Installation, über WorkVisual 19
IP 9

K

Kenntnisse, benötigt 7
KLI 8, 23
Kommunikation 11
Konfiguration 23
Konfiguration, Ethernet-Verbindung 11, 25
KR C 8
KRL 8
KUKA Customer Support 113

L

LIFO 8, 13, 43
Lizenzen 9
Logbuch 61

M

Marken 9
Meldungen 61
Meldungen, deaktivieren 102

N

Netzwerkverbindung 23

O

Open-Source 9

P

Ping 12
Produktbeschreibung 11
Programmiertipps 33
Programmierung 25
Protokollarten 14

S

Schulungen 7
Server-Betrieb 14, 36
Server-Programm 47
Server-Programm, Bedienoberfläche 48
Server-Programm, Parameter einstellen 49
Service, KUKA Roboter GmbH 113
Sicherheit 17
Sicherheitshinweise 7
smarHMI 8
Socket 8
Software 19
Speicher, erhöhen 101
Support-Anfrage 113
Systemvoraussetzungen 19

T

TCP/IP 9

U

UDP/IP 9
Updaten, Ethernet KRL 19, 20

Ü

Übersicht, Ethernet KRL 11
Übersicht, Funktionen 32
Überwachen, Verbindung 12

V

Verbindung, überwachen 12
Verbindungsverlust 11
Verwendete Begriffe 8
Verwendung, bestimmungsgemäß 16

X

XML 9
XPath 9, 29, 31

Z

Zeitstempel 42
Zielgruppe 7

